

**LAPORAN AKHR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

ERISCHA MARSELA

NIM. 2410817120022

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Akhir Praktikum Pemrograman Mobile Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman II. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : ERISCHA MARSELA

NIM : 2410817120022

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR TABEL.....	5
DAFTAR GAMBAR.....	7
MODUL 1 ANDROID BASIC WITH KOTLIN	8
SOAL 1	8
A. SOURCE CODE.....	10
B. OUTPUT CODE.....	19
C. PEMBAHASAN.....	20
ADDITIONAL.....	22
MODUL 2 : Android Layout With Compose	25
SOAL 1	25
A. SOURCE CODE.....	26
B. OUTPUT CODE.....	38
C. PEMBAHASAN.....	41
SOAL 2	42
A. PEMBAHASAN	42
ADDITIONAL.....	44
MODUL 3 : Build a Scrollable List.....	48
SOAL 1	48
A. SOURCE CODE.....	48
B. OUTPUT CODE.....	107
C. PEMBAHASAN	110
SOAL 2	114
A. PEMBAHASAN.....	114
SOAL 3	115
A. PEMBAHASAN	115
MODUL 4 : VIEWMODEL AND DEBUGGING.....	116
SOAL 1	116
A. OUTPUT CODE.....	166
B. PEMBAHASAN	167
MODUL 5 : CONNECT TO THE INTERNET	170

SOAL 1	170
A. SOURCE CODE.....	170
B. OUTPUT CODE.....	273
C. PEMBAHASAN.....	278
TAUTAN GIT	283

DAFTAR TABEL

MODUL 1

Tabel 1. Source Code MainActivity.kt XML Soal 1	10
Tabel 2. Source Code Activity_main.xml Soal 1	12
Tabel 3. Source Code MainActivity.kt Compose Soal 1	15
Tabel 4. Source Code MainActivity.kt XML Soal 1	26

MODUL 2

Tabel 5. Source Code Activity_main.xml Soal 1	28
Tabel 6. Source Code MainActivity.kt Compose Soal 1	31
Tabel 7. Source Code Testcase	46

MODUL 3

Tabel 8. Source Code MainActivity Compose	49
Tabel 9. Source Code AnimeViewModel Compose.....	51
Tabel 10. Source Code AnimeData Compose	52
Tabel 11. Source Code AnimeCharacter Compose	57
Tabel 12. SourceCode NavGraph Compose	58
Tabel 13. Source Code Screen Compose	61
Tabel 14. Source Code MainActivity XML	61
Tabel 15. Source Code AnimeAdapter XML	64
Tabel 16. Source Code AnimeRepository XML.....	67
Tabel 17. Source Code AnimeItem XML.....	75
Tabel 18. Source Code HomeFragment.....	76
Tabel 19. Source Code DetailFragment XML	82
Tabel 20. Source Code AnimeViewModel XML	86
Tabel 21. Source Code Activity_main.xml.....	87
Tabel 22. Source Code item_anime.xml.....	89
Tabel 23. Source Code fragment_home.xml	95
Tabel 24. Source Code fragment_detail.xml	97

MODUL 4

Tabel 25. Source Code MainActivity Compose	116
Tabel 26. AnimeApp.kt Compose	118
Tabel 27. AnimeListViewModel.kt Compose	119
Tabel 28. NavGraph.kt Compose.....	121
Tabel 29. SearchScreen.kt Compose	124
Tabel 30. MainActivity.kt XML.....	126
Tabel 31. MyApplication.kt XML	128
Tabel 32,AnimeViewModel.kt XML	129
Tabel 33. AnimeViewModelFactory.kt XML	132

Tabel 34. AnimeAdapter.kt XML.....	132
Tabel 35. AnimeItem.kt XML	136
Tabel 36. DetailFragment.kt XML	136
Tabel 37. activity_main.xml	142
Tabel 38. fragment_detail.xml.....	144
Tabel 39. item_anime.xml	155
Tabel 40. nav_graph.xml	161

MODUL 5

Tabel 41. Source Code MainActivity.kt	170
Tabel 42. Source Code MovieApp.kt	172
Tabel 43. Source Code MovieDao.kt.....	173
Tabel 44. Source Code MovieEntity.kt.....	176
Tabel 45. Source Code AppPreferences.kt	178
Tabel 46. Source Code MovieDatabase.kt.....	179
Tabel 47. Source Code MovieMapper.kt.....	180
Tabel 48. Source Code TmdbResponse.kt.....	184
Tabel 49. Source Code ApiHandler.kt	187
Tabel 50. Source Code NetworkClient.kt.....	188
Tabel 51. Source Code TmdbApiService.kt	190
Tabel 52. Source Code MovieRepository.kt.....	192
Tabel 53. Source Code AppContainer.kt	197
Tabel 54. Source Code Movie.kt	198
Tabel 55. Source Code MovieUiState.kt	200
Tabel 56. Source Code MovieViewModel.kt	201
Tabel 57. Source Code NavGraph.kt	207
Tabel 58. Source Code Screen.kt	210
Tabel 59. Source Code HomeScreen.kt	211
Tabel 60. Source Code DetailScreen.kt	223
Tabel 61. Source Code LanguageScreen.kt	232
Tabel 62. Source Code FavoritesScreen.kt.....	238
Tabel 63. Source Code SearchScreen.kt.....	247
Tabel 64. Source Code Settings Screen.kt.....	254
Tabel 65. Source Code strings.xml English.....	265
Tabel 66. Source Code strings.xml Indonesia	269

DAFTAR GAMBAR

MODUL 1

Gambar 1. Tampilan Awal Aplikasi	8
Gambar 2. Tampilan Dadu Setelah Di-Roll.....	9
Gambar 3. Tampilan Roll Dadu Double	9
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose.....	19
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose.....	19
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML	20

MODUL 2

Gambar 7. Tampilan Awalan Aplikasi	25
Gambar 8. Tampilan Pilihan Presentase Tip.....	26
Gambar 9. Tampilan Aplikasi Setelah Dijalankan	26
Gambar 10. Screenshot Hasil Jawaban Soal 1 XML	38
Gambar 11. Screenshot Hasil Jawaban Soal 1 XML	39
Gambar 12. Screenshot Hasil Jawaban Soal 1 XML	39
Gambar 13. Screenshot Hasil Jawaban Soal 1 Compose.....	40
Gambar 14. Screenshot Hasil Jawaban Soal 1 Compose.....	40
Gambar 15. Screenshot Hasil Jawaban Soal 1 Compose.....	41
Gambar 16. Output TestCase Extension Functions	46

MODUL 3

Gambar 17. Screenshot Hasil Jawaban Soal 1 XML	107
Gambar 18. Screenshot Hasil Jawaban Soal 1 XML	107
Gambar 19. Screenshot Hasil Jawaban Soal 1 XML	108
Gambar 20. Screenshot Hasil Jawaban Soal 1 Compose.....	108
Gambar 21. Screenshot Hasil Jawaban Soal 1 Compose.....	109
Gambar 22. Screenshot Hasil Jawaban Soal 1 Compose.....	109

MODUL 4

Gambar 23. Screenshot Hasil Jawaban Soal 1 Compose.....	166
Gambar 24. Screenshot Hasil Jawaban Soal 1 XML	167

MODUL 5

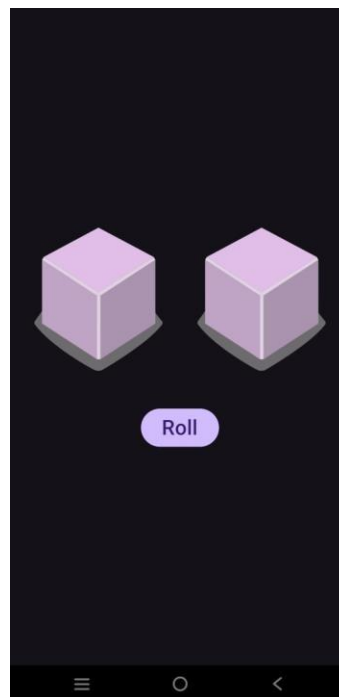
Gambar 25. Homepage English	273
Gambar 26. Home Page Indonesia.....	274
Gambar 27. Detail Page Indonesia.....	275
Gambar 28. Favorites Page	276
Gambar 29. Search Page	277

MODUL 1 ANDROID BASIC WITH KOTLIN

SOAL 1

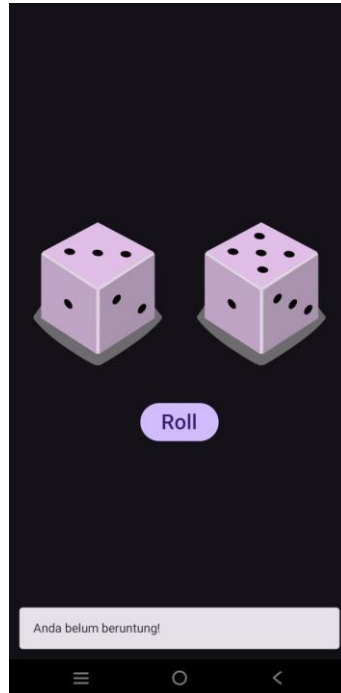
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



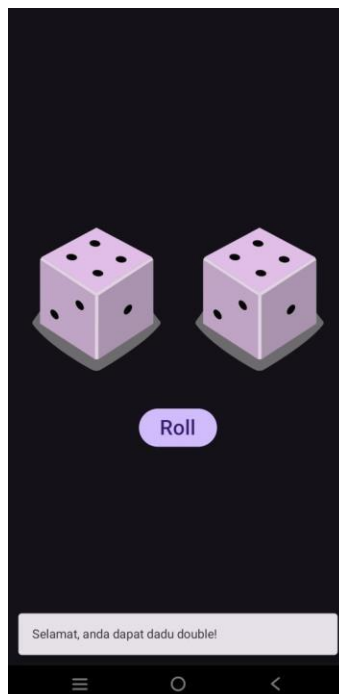
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder**

Modul

1 dalam bentuk Project. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.

6. Untuk gambar dadu dapat didownload pada link berikut:

https://drive.google.com/file/d/14V3qXGdFnuaYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**

*** = 1 sumber**

A. SOURCE CODE

MainActivity.Kt XML

Tabel 1. Source Code MainActivity.kt XML Soal 1

1.	package com.example.module1xml
2.	
3.	import android.os.Bundle
4.	import android.widget.Button
5.	import android.widget.ImageView
6.	import android.widget.Toast
7.	import androidx.appcompat.app.AppCompatActivity
8.	

9.	class MainActivity : AppCompatActivity() {	
10.	private lateinit var leftDiceView: ImageView	
11.	private lateinit var rightDiceView: ImageView	
12.	private lateinit var btnActionRoll: Button	
13.		
14.	override fun onCreate(savedInstanceState: Bundle?) {	
15.	super.onCreate(savedInstanceState)	
16.	setContentView(R.layout.activity_main)	
17.		
18.	initViewViews()	
19.	setupListener()	
20.	}	
21.		
22.	private fun initViewViews() {	
23.	leftDiceView	=
	findViewById(R.id.iv_left_dice)	
24.	rightDiceView	=
	findViewById(R.id.iv_right_dice)	
25.	btnActionRoll	=
	findViewById(R.id.btn_action_roll)	
26.	}	
27.		
28.	private fun setupListener() {	
29.	btnActionRoll.setOnClickListener {	
30.	executeDiceRoll()	
31.	}	
32.	}	
33.		
34.	private fun executeDiceRoll() {	
35.	val firstValue = (1..6).random()	
36.	val secondValue = (1..6).random()	
37.		

38.	<code>leftDiceView.setImageResource(resolveDiceDrawable (firstValue))</code>
39.	<code>rightDiceView.setImageResource(resolveDiceDrawabl e(secondValue))</code>
40.	
41.	<code>val resultMessage = if (firstValue == secondValue) {</code>
42.	<code>getString(R.string.double_message)</code>
43.	<code>} else {</code>
44.	<code>getString(R.string.bad_message)</code>
45.	<code>}</code>
46.	
47.	<code>Toast.makeText(this, resultMessage, Toast.LENGTH_SHORT).show()</code>
48.	<code>}</code>
49.	
50.	<code>private fun resolveDiceDrawable(value: Int): Int {</code>
51.	<code>return when (value) {</code>
52.	<code>1 -> R.drawable.dice_1</code>
53.	<code>2 -> R.drawable.dice_2</code>
54.	<code>3 -> R.drawable.dice_3</code>
55.	<code>4 -> R.drawable.dice_4</code>
56.	<code>5 -> R.drawable.dice_5</code>
57.	<code>else -> R.drawable.dice_6</code>
58.	<code>}</code>
59.	<code>}</code>
60.	<code>}</code>

Activity_main.xml

Tabel 2. Source Code Activity_main.xml Soal 1

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<androidx.constraintlayout.widget.ConstraintLayout
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	android:layout_width="match_parent"
6.	android:layout_height="match_parent"
7.	android:padding="16dp">
8.	
9.	<ImageView
10.	android:id="@+id/iv_left_dice"
11.	android:layout_width="120dp"
12.	android:layout_height="120dp"
13.	android:layout_marginEnd="8dp"
14.	android:src="@drawable/dice_1"
15.	app:layout_constraintBottom_toTopOf="@+id/btn_action_roll"
16.	app:layout_constraintEnd_toStartOf="@+id/iv_right_dice"
17.	app:layout_constraintHorizontal_chainStyle="packed"
18.	app:layout_constraintStart_toStartOf="parent"
19.	app:layout_constraintTop_toTopOf="parent"
20.	app:layout_constraintVertical_chainStyle="packed" />
21.	
22.	<ImageView

23.	android:id="@+id/iv_right_dice"
24.	android:layout_width="120dp"
25.	android:layout_height="120dp"
26.	android:layout_marginStart="8dp"
27.	android:src="@drawable/dice_1"
28.	app:layout_constraintBottom_toBottomOf="@+id/iv_left_dice"
29.	app:layout_constraintEnd_toEndOf="parent"
30.	app:layout_constraintStart_toEndOf="@+id/iv_left_dice"
31.	app:layout_constraintTop_toTopOf="@+id/iv_left_dice" />
32.	
33.	<Button
34.	android:id="@+id/btn_action_roll"
35.	android:layout_width="wrap_content"
36.	android:layout_height="wrap_content"
37.	android:layout_marginTop="32dp"
38.	android:text="@string/roll"
39.	app:layout_constraintBottom_toBottomOf="parent"
40.	app:layout_constraintEnd_toEndOf="parent"
41.	app:layout_constraintStart_toStartOf="parent"
42.	app:layout_constraintTop_toBottomOf="@+id/iv_left_dice" />
43.	
44.	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 3. Source Code MainActivity.kt Compose Soal 1

1.	package com.example.module1
2.	
3.	import android.os.Bundle
4.	import androidx.activity.ComponentActivity
5.	import androidx.activity.compose.setContent
6.	import androidx.compose.foundation.Image
7.	import androidx.compose.foundation.layout.Arrangement
8.	import androidx.compose.foundation.layout.Column
9.	import androidx.compose.foundation.layout.Row
10.	import androidx.compose.foundation.layout.Spacer
11.	import androidx.compose.foundation.layout.fillMaxSize
12.	import androidx.compose.foundation.layout.height
13.	import androidx.compose.foundation.layout.width
14.	import androidx.compose.foundation.layout.wrapContentSize
15.	import androidx.compose.material3.Button
16.	import androidx.compose.material3.Text
17.	import androidx.compose.runtime.Composable
18.	import androidx.compose.runtime.getValue
19.	import androidx.compose.runtime.mutableStateOf
20.	import androidx.compose.runtime.remember
21.	import androidx.compose.runtime.setValue
22.	import androidx.compose.ui.Alignment
23.	import androidx.compose.ui.Modifier
24.	import androidx.compose.ui.res.painterResource
25.	import androidx.compose.ui.res.stringResource
26.	import androidx.compose.ui.tooling.preview.Preview
27.	import androidx.compose.ui.unit.dp
28.	
29.	class MainActivity : ComponentActivity() {

```

30.     override fun onCreate(savedInstanceState:
Bundle?) {
31.         super.onCreate(savedInstanceState)
32.         setContent {
33.             TwoDiceRollerScreen()
34.         }
35.     }
36. }
37.
38. fun getDiceDrawable(diceValue: Int): Int {
39.     return when (diceValue) {
40.         1 -> R.drawable.dice_1
41.         2 -> R.drawable.dice_2
42.         3 -> R.drawable.dice_3
43.         4 -> R.drawable.dice_4
44.         5 -> R.drawable.dice_5
45.         else -> R.drawable.dice_6
46.     }
47. }
48.
49. @Preview(showBackground = true)
50. @Composable
51. fun DiceAppPreview() {
52.     TwoDiceRollerScreen()
53. }
54.
55. @Composable
56. fun TwoDiceRollerScreen(
57.     modifier: Modifier = Modifier
58.         .fillMaxSize()
59.         .wrapContentSize(Alignment.Center)
60. ) {

```

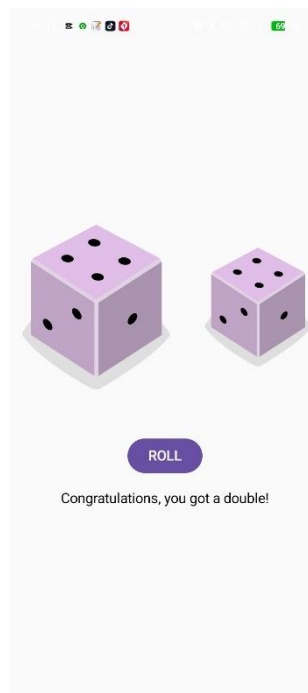
61.	var firstDiceValue by remember { mutableStateOf(1) }	
62.	var secondDiceValue by remember { mutableStateOf(1) }	
63.	var statusText by remember { mutableStateOf("") }	
64.		
65.	val leftDiceImage = getDiceDrawable(firstDiceValue)	=
66.	val rightDiceImage = getDiceDrawable(secondDiceValue)	=
67.		
68.	val winMessage = stringResource(R.string.double_message)	=
69.	val loseMessage = stringResource(R.string.bad_message)	=
70.		
71.	Column(	
72.	modifier = modifier,	
73.	horizontalAlignment Alignment.CenterHorizontally	=
74.	) {	
75.	Row(	
76.	verticalAlignment Alignment.CenterVertically,	=
77.	horizontalArrangement Arrangement.Center	=
78.	) {	
79.	Image(	
80.	painter painterResource(leftDiceImage),	=
81.	contentDescription = "Left Dice: \$firstDiceValue"	
82.	)	
83.		
84.	Spacer(modifier Modifier.width(16.dp))	=

```

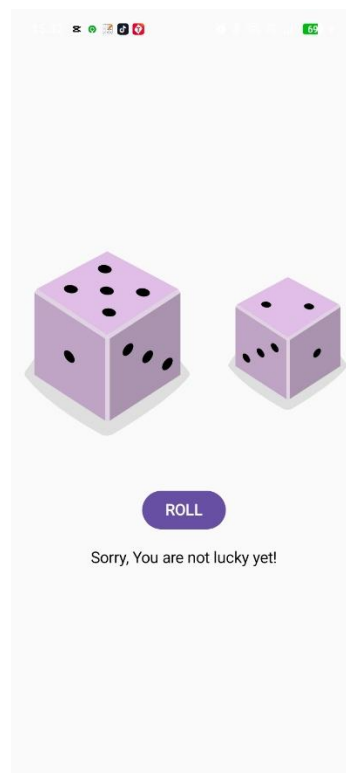
85.
86.         Image(
87.             painter                                     =
painterResource(rightDiceImage),
88.             contentDescription = "Right Dice:
$secondDiceValue"
89.         )
90.     }
91.
92.     Spacer(modifier = Modifier.height(24.dp))
93.
94.     Button(
95.         onClick = {
96.             firstDiceValue = (1..6).random()
97.             secondDiceValue = (1..6).random()
98.
99.             statusText = if (firstDiceValue ==
secondDiceValue) {
100.                 winMessage
101.             } else {
102.                 loseMessage
103.             }
104.         }
105.     ) {
106.         Text(text                                     =
stringResource(R.string.roll))
107.     }
108.
109.     Spacer(modifier = Modifier.height(16.dp))
110.
111.     Text(text = statusText)
112. }
113. }

```

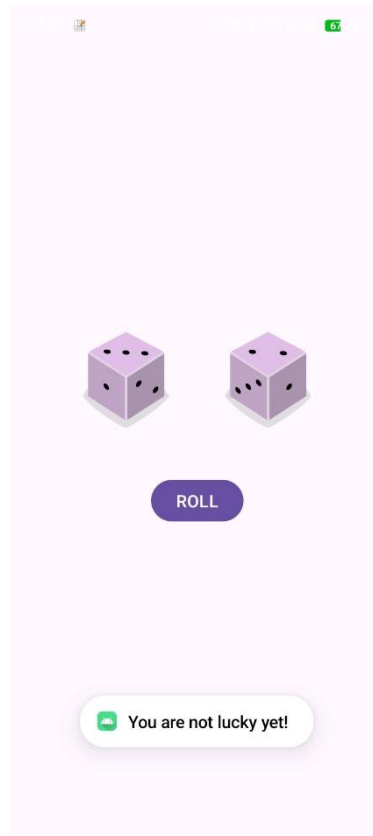
B. OUTPUT CODE



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

C. PEMBAHASAN

MainActivity.ky XML

Pada MainActivity.kt versi XML, kita membangun logika sederhana yang menampilkan dua gambar dadu beserta sebuah tombol untuk melakukan roll (mengacak) angka dadu. Aplikasi ini berjalan dengan menggunakan AppCompatActivity. Secara alur kerja, saat aplikasi pertama kali dibuka (di dalam onCreate), kita memanggil method initView() untuk menghubungkan variabel dengan komponen UI di XML menggunakan findViewById(). Setelah itu, setupListener() dipanggil untuk mendengarkan aksi dari user ketika tombol btn_action_roll diklik.

Ketika tombol tersebut ditekan, fungsi executeDiceRoll() akan dieksekusi. Nah, di sinilah logika utamanya berjalan yaitu, kita menggunakan fungsi (1..6).random() untuk menghasilkan angka acak antara 1 sampai 6 untuk masing-masing dadu. Angka acak tersebut kemudian dilempar ke dalam fungsi resolveDiceDrawable() yang berisi blok when. Fungsinya adalah untuk menentukan gambar drawable mana yang harus dipasang ke ImageView sesuai dengan angkanya. Terakhir, kita menggunakan kondisional if-else untuk mengecek apakah nilai dadu pertama sama dengan dadu kedua. Jika nilainya kembar

(double), sistem akan mengambil pesan dari `double_message`. Jika beda, pesan yang diambil adalah `bad_message`. Pesan tersebut kemudian ditampilkan ke user melalui Toast.

Activity_main.xml

Pemilihan `ConstraintLayout` ini sangat membantu karena memungkinkan kita menata elemen-elemen UI secara lebih fleksibel dengan menetapkan posisi relatif antar elemen, sehingga UI bisa lebih efisien dan adaptif di berbagai ukuran layar. Di dalamnya terdapat tiga komponen utama yaitu, dua buah `ImageView` (`iv_left_dice` dan `iv_right_dice`) yang digunakan untuk menampilkan gambar dadu dengan ukuran statis 120dp. Agar posisinya rapi berjejeran di tengah layar, kita memanfaatkan properti `layout_constraintHorizontal_chainStyle="packed"`. Serta, satu buah `Button` (`btn_action_roll`): Tombol pelempar dadu ini diposisikan di bawah chain kedua dadu tadi dengan margin top sebesar 32dp agar tidak terlalu menempel dengan gambar dadu di atasnya.

MainActivity.kt Compose

Class utama kita mewarisi `ComponentActivity`, yang dimana fungsi `setContent` langsung memanggil UI utama kita yaitu composable function `TwoDiceRollerScreen()`. Dalam merakit UI, kita menggunakan `Column` agar tata letaknya berjejer ke bawah (vertikal), dan menempatkan kedua gambar dadu di dalam `Row` agar berbaris ke samping (horizontal). Kita juga memakai `Spacer` untuk memberikan jarak (`padding`) antar komponen.

Hal yang paling penting di `Compose` adalah manajemen state. Nilai dadu pertama, dadu kedua, dan teks pesan dibungkus di dalam `remember` dan `mutableStateOf`. Hal ini memastikan bahwa UI akan recompose atau merespons secara otomatis ketika nilainya berubah saat tombol ditekan. Pada event `onClick` di komponen `Button`, dadu kembali diacak menggunakan `(1..6).random()`. Sama seperti di versi XML, hasil pengecekan apakah dadunya double atau tidak akan menentukan pesan (`stringResource`) yang diupdate ke state `statusText`. Penggunaan `stringResource()` ini sangat baik karena membuat aplikasi lebih fleksibel jika ke depannya ingin ditambahkan dukungan ragam bahasa (`internationalization`).

ADDITIONAL

1. Riset dan *cite* artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

Pemrograman imperatif dan deklaratif memiliki pendekatan logika yang sangat berbeda, terutama saat kita mengimplementasikannya pada pembuatan antarmuka pengguna (UI). Pemrograman imperatif mengandalkan pendekatan step-by-step, di mana kita sebagai developer harus memberikan instruksi spesifik mengenai bagaimana UI tersebut harus di-render sekaligus mengontrol state-nya secara manual. Sebaliknya, pemrograman deklaratif jauh lebih simpel karena berfokus pada apa yang ingin kita tampilkan; kita cukup mendeskripsikan kondisi akhir atau state dari antarmuka tersebut tanpa perlu menyusun urutan operasinya.

Opini saya lebih prefer menggunakan paradigma deklaratif untuk development UI. Alasannya karena UI deklaratif secara signifikan meningkatkan kemudahan maintenance dan testing pada kode kita. Pendekatan ini juga sangat mendukung separation of concerns dengan memisahkan logika antarmuka dari detail implementasi di belakangnya. Terlebih lagi, rata-rata framework modern untuk mobile development saat ini memang sudah difokuskan ke arah deklaratif.

Sumber =

* **Smith, J., Guo, X., & Bansal, A. (2022).** *A Predicate Construct for Declarative Programming in Imperative Languages.* ([A Predicate Construct for Declarative Programming in Imperative Languages](#))

* **Zhou, T., Zhao, Y., Hou, X., Sun, X., Chen, K., & Wang, H. (2024).** *Bridging Design and Development with Automated Declarative UI Code Generation.* ([Bridging Design and Development with Automated Declarative UI Code Generation](#))

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

Jika membandingkan konsep Object-Oriented Programming (OOP) antara Kotlin dan Java, perbedaan yang paling terasa ada pada penulisan sintaks dan jumlah boilerplate code-nya. Java menggunakan pendekatan OOP klasik yang cenderung sanga

panjang, sehingga kita diharuskan melakukan deklarasi secara ketat. Walaupun aman, hal ini sering kali membuat basis kode kita menjadi sangat besar. Berbeda halnya dengan Kotlin, yang sejak awal didesain dengan sintaks yang jauh lebih ringkas dan ekspresif. Selain itu, Kotlin tidak hanya membatasi kita pada OOP, tetapi juga menggabungkannya dengan paradigma pemrograman fungsional, sehingga arsitektur yang kita bangun bisa lebih fleksibel.

Sumber =

* **Petriv, Nicolas (2024).** *A Comparative Study of Java and Kotlin for Android Mobile Application Development.* ([COMPARATIVE-STUDY-OF-JAVA-AND-KOTLIN-FOR-ANDROID-DEVELOPMENT/Comparative study of Java and Kotlin for Android development \(scientific article\).pdf at main · NicolasPetrov/COMPARATIVE-STUDY-OF-JAVA-AND-KOTLIN-FOR-ANDROID-DEVELOPMENT](#))

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

Prinsip SOLID ibarat guideline wajib yang harus kita pegang agar kode yang kita tulis menjadi lebih rapi, scalable, dan mudah dikelola ke depannya. Prinsip ini mencakup lima aturan utama. Pertama ada Single Responsibility Principle (SRP), yang menegaskan bahwa sebuah kelas hanya boleh memiliki satu tanggung jawab atau satu alasan saja untuk berubah. Kedua, Open/Closed Principle (OCP), yang berarti entitas program kita harus selalu terbuka kalau mau ditambihin fitur baru, tapi tertutup dari modifikasi kode intinya. Ketiga, Liskov Substitution Principle (LSP), di mana objek dari sebuah subclass harus bisa menggantikan superclass-nya tanpa mengacaukan fungsionalitas sistem. Keempat, Interface Segregation Principle (ISP), yang mengingatkan kita untuk memecah interface yang terlalu besar menjadi bagian-bagian yang lebih kecil dan spesifik, agar klien tidak perlu bergantung pada method yang sebenarnya tidak mereka pakai. Terakhir adalah Dependency Inversion Principle (DIP), yang intinya menekankan bahwa modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah tapi keduanya harus sama-sama bergantung pada abstraksi.

Sumber =

* **Martin, Robert C. (2002).** *Agile Software Development: Principles, Patterns, and Practices.* ([Software_book/Agile.Principles.Patterns.and.Practices.pdf at main · rahff/Software_book](#))

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

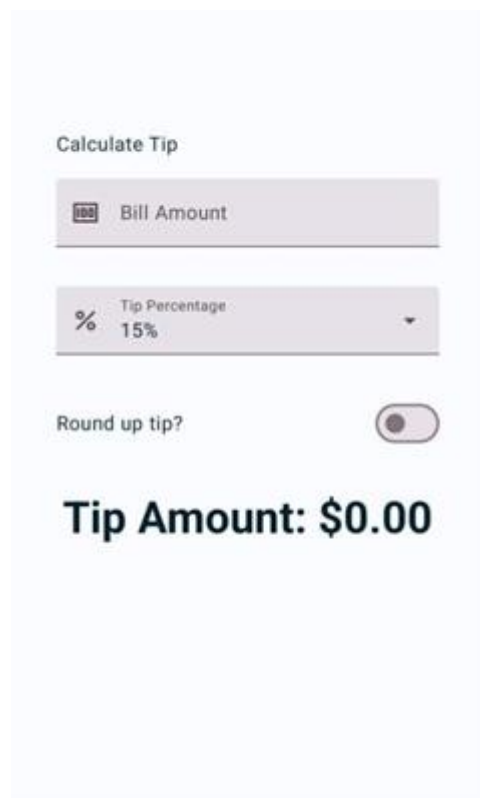
Class biasa adalah fondasi standar dari OOP di mana kita punya kendali penuh tapi harus menuliskan semuanya secara manual, mulai dari properti, metode, sampai membuat fungsi-fungsi getter dan setter. Nah, Kotlin menyediakan Data Class, yaitu tipe kelas spesial yang memang dirancang khusus hanya untuk merepresentasikan dan menyimpan objek data sederhana. Keuntungan utama memakai Data Class adalah Kotlin bakal otomatis ngenerate fungsi-fungsi standar atau kode boilerplate di belakang layar. Efeknya, kode kita tidak hanya menjadi lebih cepat untuk ditulis.

MODUL 2 : Android Layout With Compose

SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

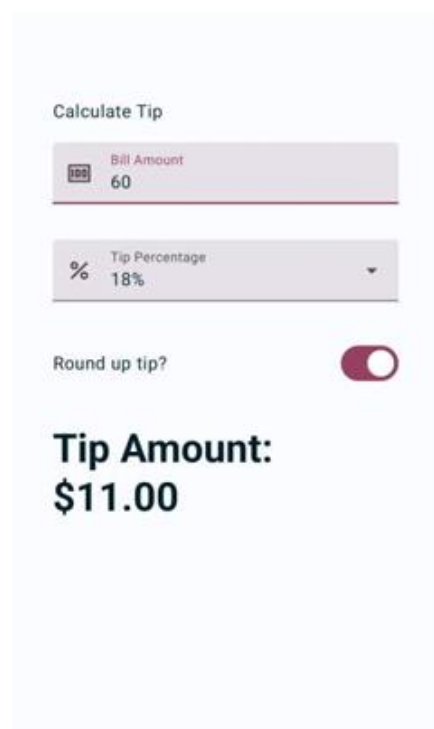
- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 7. Tampilan Awal Aplikasi



Gambar 8. Tampilan Pilihan Presentase Tip



Gambar 9. Tampilan Aplikasi Setelah Dijalankan

A. SOURCE CODE

MainActivity.Kt XML

Tabel 4. Source Code MainActivity.kt XML Soal 1

1.	package com.example.modul2xml	
2.		
3.	import android.os.Bundle	
4.	import android.widget.AdapterView	
5.	import android.widget.Button	
6.	import android.widget.EditText	
7.	import android.widget.Spinner	
8.	import android.widget.Switch	
9.	import android.widget.TextView	
10.	import androidx.appcompat.app.AppCompatActivity	
11.	import kotlin.math.ceil	
12.		
13.	class MainActivity : AppCompatActivity() {	
14.		
15.	override fun onCreate(savedInstanceState: Bundle?) {	
16.	super.onCreate(savedInstanceState)	
17.	setContentView(R.layout.activity_main)	
18.		
19.	val etTagihan	=
	findViewById<EditText>(R.id.etTagihan)	
20.	val spinnerTip	=
	findViewById<Spinner>(R.id.spinnerTip)	
21.	val switchBulat	=
	findViewById<Switch>(R.id.switchBulat)	
22.	val btnHitung	=
	findViewById<Button>(R.id.btnHitung)	
23.	val tvHasil	=
	findViewById<TextView>(R.id.tvHasil)	
24.		
25.	val pilihanPersentase = arrayOf("15%",	
	"18%", "20%")	
26.	val nilaiPersentase = arrayOf(0.15,	
	0.18, 0.20)	
27.		

28.	val adapter = ArrayAdapter(this, android.R.layout.simple_spinner_dropdown_item, pilihanPersentase)
29.	spinnerTip.adapter = adapter
30.	
31.	btnHitung.setOnClickListener {
32.	val inputAngka = etTagihan.text.toString()
33.	val tagihan = inputAngka.toDoubleOrNull() ?: 0.0
34.	
35.	val posisiTerpilih = spinnerTip.selectedItemPosition
36.	
37.	val persentaseTip = nilaiPersentase[posisiTerpilih]
38.	
39.	var jumlahTip = tagihan * persentaseTip
40.	
41.	if (switchBulat.isChecked) {
42.	jumlahTip = ceil(jumlahTip)
43.	}
44.	
45.	tvHasil.text = String.format("Jumlah Tip: \$%.2f", jumlahTip)
46.	}
47.	}
48.	}

Activity_main.xml*Tabel 5. Source Code Activity_main.xml Soal 1*

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<LinearLayout

3.	<code>xmlns:android="http://schemas.android.com/apk/res/android"</code>
4.	<code>android:layout_width="match_parent"</code>
5.	<code>android:layout_height="match_parent"</code>
6.	<code>android:orientation="vertical"</code>
7.	<code>android:gravity="center_vertical"</code>
8.	<code>android:padding="32dp"></code>
9.	<code><TextView</code>
10.	<code>android:layout_width="wrap_content"</code>
11.	<code>android:layout_height="wrap_content"</code>
12.	<code>android:text="Calculate Tip"</code>
13.	<code>android:textSize="20sp"</code>
14.	<code>android:textStyle="bold"</code>
15.	<code>android:layout_marginBottom="24dp"/></code>
16.	
17.	<code><EditText</code>
18.	<code>android:id="@+id/etTagihan"</code>
19.	<code>android:layout_width="match_parent"</code>
20.	<code>android:layout_height="60dp"</code>
21.	<code>android:hint="Bill Amount"</code>
22.	<code>android:inputType="numberDecimal"</code>
23.	<code>android:background="#F3F3F6"</code>
24.	<code>android:paddingStart="16dp"</code>
25.	<code>android:paddingEnd="16dp"</code>
26.	<code>android:layout_marginBottom="24dp" /></code>
27.	
28.	<code><LinearLayout</code>
29.	<code>android:layout_width="match_parent"</code>
30.	<code>android:layout_height="60dp"</code>
31.	<code>android:orientation="horizontal"</code>
32.	<code>android:background="#F3F3F6"</code>
33.	<code>android:gravity="center_vertical"</code>

```

34.         android:layout_marginBottom="32dp">
35.
36.         <TextView
37.
38.         android:layout_width="wrap_content"
39.
40.         android:layout_height="wrap_content"
41.
42.         android:text="%"
43.         android:textSize="18sp"
44.         android:textStyle="bold"
45.         android:paddingStart="16dp"
46.         android:paddingEnd="16dp"/>
47.
48.         <Spinner
49.
50.         android:id="@+id/spinnerTip"
51.
52.         android:layout_width="match_parent"
53.
54.         android:layout_height="match_parent"
55.         android:background="@null" />
56.     </LinearLayout>
57.
58.     <LinearLayout
59.
60.         android:layout_width="match_parent"
61.         android:layout_height="wrap_content"
62.         android:orientation="horizontal"
63.         android:gravity="center_vertical"
64.         android:layout_marginBottom="32dp">
65.
66.         <TextView
67.
68.         android:layout_width="0dp"
69.
70.         android:layout_height="wrap_content"
71.         android:layout_weight="1"

```

63.	android:text="Round up tip?"
64.	android:textSize="16sp"/>
65.	
66.	<Switch
67.	android:id="@+id/switchBulat"
68.	android:layout_width="wrap_content"
69.	android:layout_height="wrap_content" />
70.	</LinearLayout>
71.	
72.	<Button
73.	android:id="@+id/btnHitung"
74.	android:layout_width="match_parent"
75.	android:layout_height="wrap_content"
76.	android:text="Tip Amount"/>
77.	
78.	<TextView
79.	android:id="@+id/tvHasil"
80.	android:layout_width="wrap_content"
81.	android:layout_height="wrap_content"
82.	android:text="Tip Amount: \$0.00"
83.	android:textSize="32sp"
84.	android:textStyle="bold"
85.	android:layout_marginTop="16dp"/>
86.	
87.	</LinearLayout>

MainActivity.Kt Compose

Tabel 6. Source Code MainActivity.kt Compose Soal 1

1.	package com.example.modul2compose
2.	
3.	import android.os.Bundle

```

4. import androidx.activity.ComponentActivity
5. import androidx.activity.compose.setContent
6. import androidx.compose.foundation.layout.*
7. import
  androidx.compose.foundation.text.KeyboardOptions
8. import androidx.compose.material3.*
9. import androidx.compose.runtime.*
10. import androidx.compose.ui.Alignment
11. import androidx.compose.ui.Modifier
12. import androidx.compose.ui.graphics.Color
13. import androidx.compose.ui.text.font.FontWeight
14. import
  androidx.compose.ui.text.input.KeyboardType
15. import androidx.compose.ui.unit.dp
16. import androidx.compose.ui.unit.sp
17. import kotlin.math.ceil
18.
19. class MainActivity : ComponentActivity() {
20.     override fun onCreate(savedInstanceState:
  Bundle?) {
21.         super.onCreate(savedInstanceState)
22.         setContent {
23.             MaterialTheme {
24.                 Surface(
25.                     modifier =
  Modifier.fillMaxSize(),
26.                     color =
  MaterialTheme.colorScheme.background
27.                 ) {
28.                     KalkulatorTipCompose()
29.                 }
30.             }
31.         }
32.     }

```

```

33. }
34.
35. @OptIn(ExperimentalMaterial3Api::class)
36. @Composable
37. fun KalkulatorTipCompose() {
38.     var inputTagihan by remember {
39.         mutableStateOf("") }
40.
41.     var dropdownExpanded by remember {
42.         mutableStateOf(false) }
43.
44.     val daftarPilihan = listOf("15%" to 0.15,
45.         "18%" to 0.18, "20%" to 0.20)
46.
47.     var pilihanTerpilih by remember {
48.         mutableStateOf(daftarPilihan[0]) }
49.
50.     var switchBulat by remember {
51.         mutableStateOf(false) }
52.
53.     var teksHasil by remember {
54.         mutableStateOf("Tip Amount: $0.00") }
55.
56.     Column(
57.         modifier = Modifier
58.             .fillMaxSize()
59.             .padding(32.dp),
60.         verticalArrangement = Arrangement.Center
61.     ) {
62.
63.         Text(
64.             text = "Calculate Tip",
65.             fontSize = 20.sp,
66.             fontWeight = FontWeight.Bold,
67.             modifier = Modifier.padding(bottom =
68.                 24.dp)
69.         )
70.     }

```

```

60.
61.         TextField(
62.             value = inputTagihan,
63.             onChange = { inputTagihan = it },
64.             placeholder = { Text( text = "Bill
Amount", fontSize = 18.sp,) },
65.             keyboardOptions                                =
KeyboardOptions(keyboardType                                =
KeyboardType.Number),
66.             modifier = Modifier
67.                 .fillMaxWidth()
68.                 .height(60.dp)
69.                 .padding(bottom = 8.dp),
70.             leadingIcon = {
71.                 Text(
72.                     text = "$",
73.                     fontSize = 18.sp,
74.                     fontWeight = FontWeight.Bold,
75.                     modifier                                =
Modifier.padding(start = 16.dp, end = 8.dp)
76.                 )
77.             },
78.             colors = TextFieldDefaults.colors(
79.                 unfocusedContainerColor                    =
Color(0xFFFF3F3F6),
80.                 focusedContainerColor                      =
Color(0xFFFF3F3F6),
81.                 unfocusedIndicatorColor                   =
Color.Transparent,
82.                 focusedIndicatorColor                     =
Color.Transparent
83.             )
84.         )
85.
86.         ExposedDropDownMenuBox(

```

```

87.         expanded = dropdownExpanded,
88.         onExpandedChange = { dropdownExpanded
= !dropdownExpanded },
89.         modifier = Modifier.padding(bottom =
32.dp)
90.     ) {
91.         TextField(
92.             value = pilihanTerpilih.first,
93.             onValueChange = {},
94.             readOnly = true,
95.             leadingIcon = {
96.                 Text(
97.                     text = "%",
98.                     fontSize = 18.sp,
99.                     fontWeight
=
FontWeight.Bold,
100.                    modifier
=
Modifier.padding(start = 16.dp, end = 8.dp)
101.                )
102.            },
103.            trailingIcon
=
{
ExposedDropdownMenuDefaults.TrailingIcon(expanded
= dropdownExpanded) },
104.            modifier = Modifier
105.                .menuAnchor()
106.                .fillMaxWidth()
107.                .height(60.dp),
108.            colors
=
TextFieldDefaults.colors(
109.                unfocusedContainerColor
=
Color(0xFFF3F3F6),
110.                focusedContainerColor
=
Color(0xFFF3F3F6),
111.                unfocusedIndicatorColor
=
Color.Transparent,

```

```

112.         focusedIndicatorColor =
Color.Transparent
113.     )
114. )
115.     ExposedDropDownMenu (
116.         expanded = dropdownExpanded,
117.         onDismissRequest = {
dropdownExpanded = false }
118.     ) {
119.         daftarPilihan.forEach { pilihan -
>
120.             DropdownMenuItem(
121.                 text = {
Text(pilihan.first) },
122.                 onClick = {
123.                     pilihanTerpilih =
pilihan
124.                     dropdownExpanded =
false
125.                 }
126.             )
127.         }
128.     }
129. }
130.
131.     Row(
132.         modifier = Modifier
133.             .fillMaxWidth()
134.             .padding(bottom = 32.dp),
135.         verticalAlignment =
Alignment.CenterVertically
136.     ) {
137.         Text(
138.             text = "Round up tip?",

```



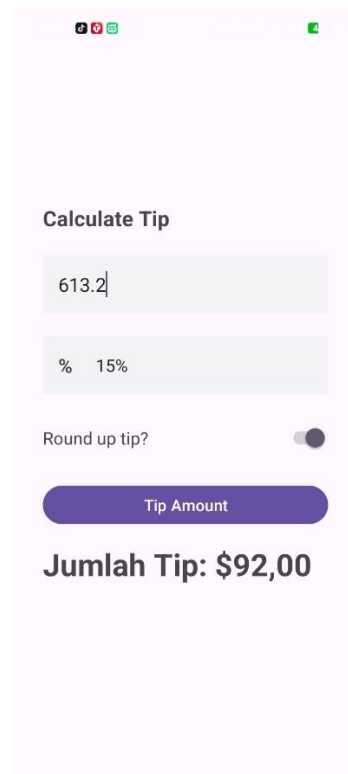
```

139.         fontSize = 16.sp,
140.         modifier = Modifier.weight(1f)
141.     )
142.     Switch(
143.         checked = switchBulat,
144.         onCheckedChange = { switchBulat =
it }
145.     )
146. }
147.
148. Button(
149.     onClick = {
150.         val tagihan =
inputTagihan.toDoubleOrNull() ?: 0.0
151.
152.         val persentaseTip =
pilihanTerpilih.second
153.
154.         var jumlahTip = tagihan *
persentaseTip
155.
156.         if (switchBulat) {
157.             jumlahTip = ceil(jumlahTip)
158.         }
159.
160.         teksHasil = String.format("Tip
Amount: $%.2f", jumlahTip)
161.     },
162.     modifier = Modifier.fillMaxWidth()
163. ) {
164.     Text("Tip Amount")
165. }
166.
167. Text(

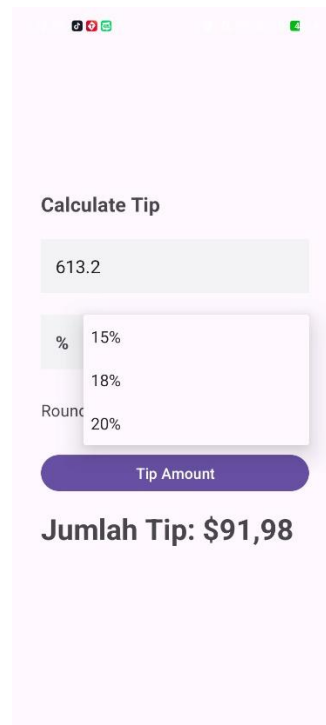
```

```
168.         text = teksHasil,  
169.         fontSize = 32.sp,  
170.         fontWeight = FontWeight.Bold,  
171.         modifier = Modifier.padding(top =  
16.dp)  
172.     )  
173. }  
174. }
```

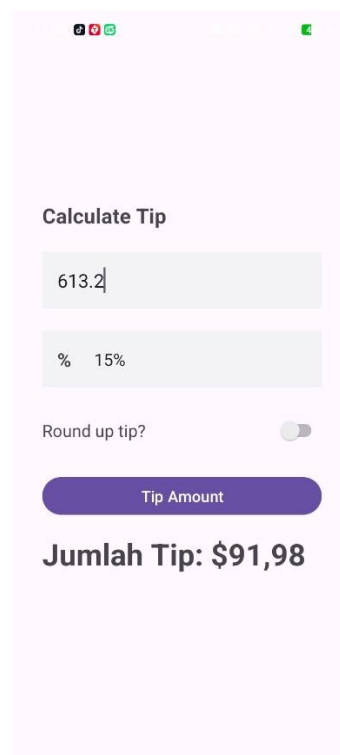
B. OUTPUT CODE



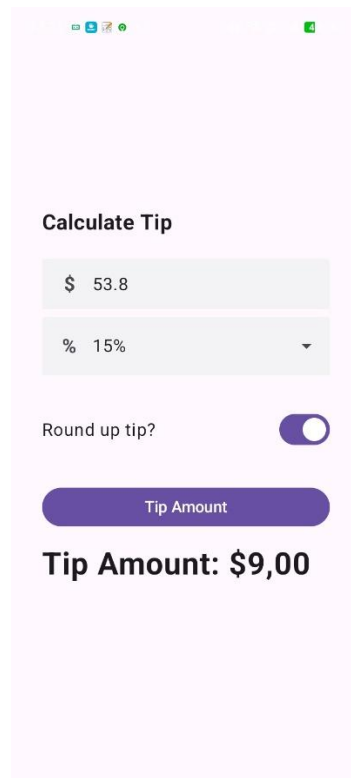
Gambar 10. Screenshot Hasil Jawaban Soal 1 XML



Gambar 11. Screenshot Hasil Jawaban Soal 1 XML

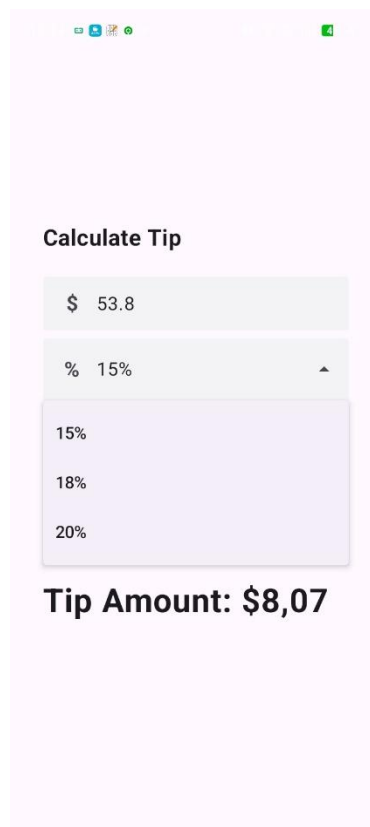


Gambar 12. Screenshot Hasil Jawaban Soal 1 XML



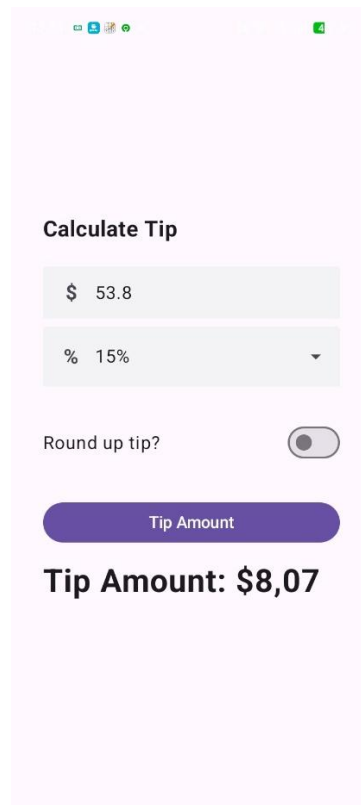
A screenshot of a mobile application titled "Calculate Tip". The interface has a light pink background. At the top, there's a status bar with icons for signal, Wi-Fi, and battery. Below the title, there are two input fields: the first shows "\$ 53.8" and the second shows "% 15%". Below these is a toggle switch labeled "Round up tip?" which is currently turned on. A purple button labeled "Tip Amount" is positioned below the toggle. The result is displayed in large, bold black text as "Tip Amount: \$9,00".

Gambar 13. Screenshot Hasil Jawaban Soal 1 Compose



A screenshot of the same "Calculate Tip" application. The input fields for "\$ 53.8" and "% 15%" remain. The "Round up tip?" toggle is still on. The purple "Tip Amount" button is present. A dropdown menu is open below the percentage field, showing three options: "15%", "18%", and "20%". The result is displayed in large, bold black text as "Tip Amount: \$8,07".

Gambar 14. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 15. Screenshot Hasil Jawaban Soal 1 Compose

C. PEMBAHASAN

MainActivity.ky XML

“SetContentView” untuk menampilkan tata letak statis dari file `activity_main.xml` ke layar. Selanjutnya, fungsi “`findViewById`” digunakan untuk mencari dan menghubungkan komponen desain seperti kotak input, menu tarik-turun, dan sakelar agar dapat dimanipulasi melalui kode Kotlin. Setelah komponen terhubung, program mendeklarasikan variabel array untuk menyimpan teks persentase tip beserta nilai desimalnya. Data teks tersebut kemudian dipetakan ke dalam komponen menu tarik-turun (Spinner) dengan bantuan “`ArrayAdapter`”. Bagian inti dari logika ini berada di dalam *event listener* pada tombol hitung (`btnHitung.setOnClickListener`), yang akan mengeksekusi perhitungan hanya ketika tombol ditekan. Di dalam blok tersebut, program mengambil teks input dan mengonversinya menjadi angka desimal menggunakan “`toDoubleOrNull`”, mengambil nilai persentase berdasarkan posisi yang dipilih pengguna, lalu menghitung total tip. Jika opsi pembulatan pada sakelar diaktifkan, fungsi “`ceil`” akan membulatkan hasil tersebut ke bilangan bulat di atasnya sebelum akhirnya diformat menjadi teks mata uang dan ditampilkan ke layar.

Activity_main.xml

activity_main.xml berfungsi murni sebagai cetak biru visual aplikasi yang ditulis menggunakan bahasa penanda XML. Tampilan aplikasi ini menggunakan “LinearLayout” dengan orientasi vertikal sebagai wadah utama untuk memastikan semua elemen antarmuka disusun rapi secara bertumpuk dari atas ke bawah. Di dalam wadah tersebut, terdapat berbagai komponen dasar seperti “TextView” yang bertugas menampilkan teks statis untuk judul dan label, serta “EditText” dengan tipe input angka desimal agar pengguna hanya bisa mengetikkan nominal tagihan. Untuk memberikan opsi persentase, kode ini memanfaatkan komponen “Spinner” yang menghasilkan menu dropdown. Selain itu, ditambahkan pula komponen “Switch” yang bertindak sebagai sakelar interaktif (On/Off) untuk fitur pembulatan nilai tip. Susunan antarmuka ini kemudian ditutup dengan sebuah “Button” yang berfungsi sebagai pemicu agar file Kotlin memulai proses komputasi ketika tombol tersebut ditekan.

MainActivity.kt Compose

“setContent” yang menandakan bahwa tampilan akan digambar sepenuhnya oleh mesin Compose. Konsep paling krusial dalam kode ini adalah penggunaan State melalui fungsi “remember { mutableStateOf(...) }”. Variabel state ini sangat reaktif; jika nilainya berubah (misalnya saat pengguna mengetik angka atau hasil dihitung), bagian layar yang terkait dengan variabel tersebut akan otomatis digambar ulang. Tata letaknya sendiri dibangun secara intuitif menggunakan komponen Column untuk menyusun elemen secara vertikal dan “Row” untuk susunan horizontal. Perubahan komponen juga sangat terlihat; kotak input kini menggunakan “TextField”, sedangkan menu dropdown dibangun menggunakan kombinasi “ExposedDropdownMenuBox” dan perulangan baris data. Seluruh interaksi diakhiri pada fungsi “onClick” di dalam “Button”, di mana program langsung mengeksekusi perhitungan matematis dari nilai state yang ada, membulatkannya jika sakelar aktif, dan langsung memperbarui state hasil sehingga teks jumlah tip di layar otomatis berubah.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. PEMBAHASAN

Perbedaan mendasar antara implementasi XML dan Jetpack Compose terletak pada paradigma pemrograman dan pengelolaan tampilannya. Pendekatan XML menggunakan

paradigma imperatif di mana desain antarmuka (UI) dipisahkan secara fisik di dalam file berformat .xml, sementara logika program ditulis di file Kotlin atau Java, sehingga pengembang harus secara manual menuliskan perintah untuk memperbarui elemen UI setiap kali ada perubahan data (misalnya memanggil `textView.text = "Baru"`). Sebaliknya, Jetpack Compose menggunakan paradigma deklaratif yang menyatukan pembuatan desain antarmuka dan logika ke dalam satu bahasa, yaitu Kotlin, melalui fungsi beranotasi `@Composable`. Alih-alih memerintahkan "bagaimana" UI harus diubah, Compose memungkinkan pengembang mendeklarasikan "apa" yang harus ditampilkan berdasarkan status (state) data saat ini, dan UI akan secara otomatis menggambar ulang (recompose) dirinya sendiri ketika state tersebut mengalami perubahan.

Implementasi menggunakan XML memiliki kelebihan utama berupa ekosistem yang sudah sangat matang dan stabil, mengingat metode ini telah menjadi standar pengembangan Android selama belasan tahun, sehingga dokumentasi, referensi perbaikan bug, serta dukungan pustaka (library) pihak ketiga sangat melimpah. Selain itu, XML memberikan pemisahan peran yang tegas antara desain visual dan logika bisnis (separation of concerns), yang didukung oleh alat Layout Editor bawaan Android Studio untuk melakukan pratinjau dan desain drag and drop dengan sangat ringan. Namun, kelemahan utama XML adalah banyaknya boilerplate atau kode panjang yang berulang, khususnya saat membangun daftar bergulir yang rumit (seperti keharusan membuat Adapter dan ViewHolder pada RecyclerView). Selain itu, karena pembaruan UI dilakukan secara manual, metode ini sangat rentan terhadap bug sinkronisasi, di mana data di latar belakang sudah berubah namun tampilan di layar gagal diperbarui karena pengembang terlewat menambahkan baris kode pembaruannya.

Di sisi lain, kelebihan utama Jetpack Compose adalah penulisan kode yang jauh lebih ringkas, intuitif, dan efisien sehingga secara drastis memangkas kode boilerplate yang sebelumnya wajib ada di XML. Penggunaan satu bahasa (Kotlin) untuk membangun antarmuka dan logika membuat pengembangan menjadi lebih terpadu, ditambah dengan kemampuannya yang sangat reaktif sehingga antarmuka selalu terjamin selaras dengan kondisi data (state) terbaru tanpa perlu pembaruan manual. Meski menawarkan banyak kemudahan, kekurangan dari Jetpack Compose adalah kurva pembelajarannya yang cukup curam, terutama bagi pengembang yang sudah terbiasa memanipulasi UI secara imperatif, karena mereka harus merombak pola pikir menjadi reaktif. Selain itu, fitur pratinjau (Preview) Compose di Android Studio sering kali membutuhkan lebih banyak alokasi

memori (RAM) serta waktu build yang lebih lama dibandingkan Layout Editor XML, dan masih ada beberapa pustaka lama yang belum mendukung Compose secara penuh tanpa trik penyesuaian (interoperability).

ADDITIONAL

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**

a) Null safety

Dalam lingkungan Java, `NullPointerException` (NPE) merupakan anomali yang sering memicu kegagalan sistem pada saat runtime. Kotlin mereduksi risiko ini secara fundamental melalui implementasi Null Safety pada tingkat sistem tipe (type system). Kompiler Kotlin membedakan secara ketat antara referensi yang diizinkan untuk menampung nilai kosong (nullable references) dan yang tidak diizinkan (non-nullable references). Mekanisme ini memaksa pengembang untuk menangani potensi nilai null pada fase kompilasi (compile-time), sehingga meningkatkan stabilitas dan keandalan perangkat lunak secara signifikan sebelum aplikasi dijalankan.

b) Chaining

Method Chaining merupakan paradigma sintaksis yang memfasilitasi pemanggilan serangkaian fungsi secara sekuensial dalam satu instruksi tunggal. Pada Kotlin, fitur ini dioptimalkan melalui implementasi scope functions (seperti “let”, “run”, “with”, “apply”, dan “also”). Hal ini memungkinkan setiap pemanggilan metode untuk memproses sebuah objek dan mengembalikannya, yang kemudian langsung menjadi subjek bagi eksekusi metode selanjutnya. Pendekatan ini secara efektif meningkatkan tingkat keterbacaan kode (code readability) dan mereduksi kompleksitas siklomatik dengan meminimalisasi deklarasi variabel sementara yang berlebihan.

c) Extension Functions

Fungsi ekstensi merupakan mekanisme yang mengizinkan pengembang untuk mengaugmentasi fungsi perilaku baru ke dalam suatu kelas yang telah ada tanpa perlu mengimplementasikan pewarisan hierarkis (inheritance) atau mengadopsi pola desain

dekorator (decorator design pattern). Fitur ini memungkinkan modifikasi pada kelas-kelas tertutup (final classes), seperti komponen antarmuka pengguna (View) atau kelas dasar (String). Secara leksikal, fungsi ekstensi dipanggil seolah-olah merupakan metode internal dari kelas tersebut, yang pada gilirannya menghasilkan struktur kode yang lebih terenkapsulasi dan modular.

d) Data class

Dalam paradigma berorientasi objek konvensional, pendefinisian entitas data murni (Plain Old Java Object / POJO) mewajibkan penulisan kode repetitif (boilerplate code) yang substansial, meliputi deklarasi fungsi aksesori (getter dan setter), “equals()”, “hashCode()”, dan “toString()”. Kotlin mengatasi inefisiensi ini melalui sintaksis “data class”. Kompiler secara otomatis mensintesis seluruh fungsi standar yang diperlukan untuk representasi dan manipulasi data pada level sistem, sehingga menghasilkan arsitektur kode yang lebih ringkas serta meminimalisasi potensi human error dalam pendefinisian entitas data.

Sumber =

* **Jameron, Dmitry & Isakova, Svetlana (2017).** Kotlin in Action. ([Books/Kotlin_in_Action.Dmitry_Jemerov_Svetlana_Isakova_.pdf at master · IlyaShelkovenko/Books](#))

* **Subramaniam, Venkat (2019).** Programming Kotlin: Create Elegant, Expressive, and Performant JVM and Android Applications. ([Download PDF - Programming Kotlin: Create Elegant, Expressive, And Performant Jvm And Android Applications \[PDF\] \[sf7deeissko0\]](#))

2. berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

Kita akan membuat fungsi tambahan untuk kelas bawaan Kotlin, yaitu Int (Integer/Angka Bawah). Kita akan menambahkan fungsi baru bernama formatRupiah() agar setiap angka bisa langsung mengubah dirinya menjadi format teks mata uang tanpa perlu membuat kelas bantuan (utility class) yang rumit.

“fun Int.formatRupiah()” ini memberi tahu Kotlin, "Tambahkan fungsi formatRupiah ke dalam kelas Int". Padahal, kelas Int adalah kelas bawaan inti dari Kotlin yang source codenya tidak bisa kita ubah. Di dalam extension function, “this” mewakili objek yang memanggilnya. Jika “150000.formatRupiah()” dipanggil, maka “this” bernilai 150000.

Tabel 7. Source Code Testcase

1.	fun Int.formatRupiah(): String {
2.	return "Rp \$this,00"
3.	}
4.	
5.	fun main() {
6.	val hargaBaju = 150000
7.	val hargaSepatu = 350000
8.	
9.	val teksBaju = hargaBaju.formatRupiah()
10.	val teksSepatu = hargaSepatu.formatRupiah()
11.	
12.	println("Harga Baju : \$teksBaju")
13.	println("Harga Sepatu : \$teksSepatu")
14.	println("Harga Topi : " +
	45000.formatRupiah())
15.	}

```
"C:\Program Files\Java\jdk-25\bin\java.
Harga Baju : Rp 150000,00
Harga Sepatu : Rp 350000,00
Harga Topi : Rp 45000,00

Process finished with exit code 0
```

Gambar 16. Output TestCase Extension Functions

MODUL 3 : Build a Scrollable List

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a) Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b) Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library 'material' (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah "Item Highlight" yang menggunakan Carousel atau LazyRow dengan ketentuan: Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

A. SOURCE CODE

MainActivity.Kt Compose

Tabel 8. Source Code MainActivity Compose

1.	package com.example.clickanime
2.	
3.	import android.content.Context
4.	import android.os.Bundle
5.	import androidx.activity.ComponentActivity
6.	import androidx.activity.compose.setContent
7.	import androidx.activity.enableEdgeToEdge
8.	import androidx.compose.foundation.layout.fillMaxSize
9.	import androidx.compose.material3.Surface
10.	import androidx.compose.runtime.*
11.	import androidx.compose.ui.Modifier
12.	import androidx.navigation.compose.rememberNavController
13.	import com.example.clickanime.navigation.NavGraph
14.	import com.example.clickanime.ui.theme.AnimeAppTheme
15.	import java.util.Locale
16.	
17.	class MainActivity : ComponentActivity() {
18.	
19.	override fun attachBaseContext(newBase: Context) {
20.	val prefs = newBase.getSharedPreferences("settings", Context.MODE_PRIVATE)
21.	val lang = prefs.getString("language", Locale.getDefault().language) ?: "en"
22.	val locale = Locale(lang)
23.	Locale.setDefault(locale)

```

24.         val config =
newBase.resources.configuration
25.         config.setLocale(locale)
26.
super.attachBaseContext(newBase.createConfigura
tionContext(config))
27.     }
28.
29.     override fun onCreate(savedInstanceState:
Bundle?) {
30.         super.onCreate(savedInstanceState)
31.         enableEdgeToEdge()
32.         setContent {
33.             AnimeAppTheme {
34.                 Surface(modifier =
Modifier.fillMaxSize()) {
35.                     val navController =
rememberNavController()
36.                     NavGraph(
37.                         navController =
navController,
38.                         onLocaleChange = {
39.                             langCode ->
applyLocale(langCode)
40.                         }
41.                     )
42.                 }
43.             }
44.         }
45.     }
46.
47.     private fun applyLocale(langCode: String) {

```

48.	getSharedPreferences("settings",
	Context.MODE_PRIVATE)
49.	.edit()
50.	.putString("language", langCode)
51.	.apply()
52.	recreate()
53.	}
54.	}

Tabel 9. Source Code AnimeViewModel Compose

1.	package com.example.clickanime.viewmodel
2.	
3.	import androidx.lifecycle.ViewModel
4.	import com.example.clickanime.data.AnimeData
5.	import
	com.example.clickanime.model.AnimeCharacter
6.	import kotlinx.coroutines.flow.MutableStateFlow
7.	import kotlinx.coroutines.flow.StateFlow
8.	import kotlinx.coroutines.flow.asStateFlow
9.	
10.	class AnimeViewModel : ViewModel() {
11.	
12.	private val _characters =
	MutableStateFlow<List<AnimeCharacter>>(AnimeData.characters)
13.	val characters:
	StateFlow<List<AnimeCharacter>> =
	_characters.asStateFlow()
14.	

15.	private val _carouselIndex =
	MutableStateFlow(0)
16.	val carouselIndex: StateFlow<Int> =
	_carouselIndex.asStateFlow()
17.	
18.	fun updateCarouselIndex(index: Int) {
19.	_carouselIndex.value = index
20.	}
21.	}

Tabel 10. Source Code AnimeData Compose

1.	package com.example.clickanime.data
2.	
3.	import com.example.clickanime.R
4.	import
	com.example.clickanime.model.AnimeCharacter
5.	
6.	object AnimeData {
7.	val characters = listOf(
8.	AnimeCharacter(
9.	id = 1,
10.	nameResId = R.string.char1_name,
11.	animeResId = R.string.char1_anime,
12.	genreResId = R.string.char1_genre,
13.	descriptionResId =
	R.string.char1_description,

14.	abilityResId	=
	R.string.char1_ability,	
15.	imageUrl	=
	"https://cdn.myanimelist.net/images/characters/ 2/284121.jpg",	
16.	myAnimeListUrl	=
	"https://myanimelist.net/character/17/Naruto_Uz umaki",	
17.	rating = 9.2f,	
18.	episodesOrChapters = 720,	
19.	year = 2002,	
20.	typeResId = R.string.type_anime	
21.	),	
22.	AnimeCharacter(	
23.	id = 2,	
24.	nameResId = R.string.char2_name,	
25.	animeResId = R.string.char2_anime,	
26.	genreResId = R.string.char2_genre,	
27.	descriptionResId	=
	R.string.char2_description,	
28.	abilityResId	=
	R.string.char2_ability,	
29.	imageUrl	=
	"https://cdn.myanimelist.net/images/characters/ 9/310307.jpg",	

30.	myAnimeListUrl	=
	"https://myanimelist.net/character/40/Monkey_D_Luffy",	
31.	rating = 9.4f,	
32.	episodesOrChapters = 1100,	
33.	year = 1999,	
34.	typeResId = R.string.type_anime	
35.	),	
36.	AnimeCharacter(	
37.	id = 3,	
38.	nameResId = R.string.char3_name,	
39.	animeResId = R.string.char3_anime,	
40.	genreResId = R.string.char3_genre,	
41.	descriptionResId	=
	R.string.char3_description,	
42.	abilityResId	=
	R.string.char3_ability,	
43.	imageUrl	=
	"https://myanimelist.net/images/characters/8/349229.jpg",	
44.	myAnimeListUrl	=
	"https://myanimelist.net/character/5/Ichigo_Kurosaki",	
45.	rating = 8.9f,	
46.	episodesOrChapters = 366,	
47.	year = 2004,	

48.	typeResId = R.string.type_anime
49.	),
50.	AnimeCharacter(
51.	id = 4,
52.	nameResId = R.string.char4_name,
53.	animeResId = R.string.char4_anime,
54.	genreResId = R.string.char4_genre,
55.	descriptionResId =
	R.string.char4_description,
56.	abilityResId =
	R.string.char4_ability,
57.	imageUrl =
	"https://myanimelist.net/images/characters/9/206427.jpg",
58.	myAnimeListUrl =
	"https://myanimelist.net/character/40882/Eren_Yeager",
59.	rating = 9.1f,
60.	episodesOrChapters = 87,
61.	year = 2013,
62.	typeResId = R.string.type_anime
63.	),
64.	AnimeCharacter(
65.	id = 5,
66.	nameResId = R.string.char5_name,
67.	animeResId = R.string.char5_anime,

68.	genreResId = R.string.char5_genre,	
69.	descriptionResId	=
	R.string.char5_description,	
70.	abilityResId	=
	R.string.char5_ability,	
71.	imageUrl	=
	"https://myanimelist.net/images/characters/15/72546.jpg",	
72.	myAnimeListUrl	=
	"https://myanimelist.net/character/246/Goku",	
73.	rating = 9.0f,	
74.	episodesOrChapters = 291,	
75.	year = 1989,	
76.	typeResId = R.string.type_anime	
77.	),	
78.	AnimeCharacter(	
79.	id = 6,	
80.	nameResId = R.string.char6_name,	
81.	animeResId = R.string.char6_anime,	
82.	genreResId = R.string.char6_genre,	
83.	descriptionResId	=
	R.string.char6_description,	
84.	abilityResId	=
	R.string.char6_ability,	
85.	imageUrl	=
	"https://myanimelist.net/images/characters/9/72533.jpg",	

86.	myAnimeListUrl	=
	"https://myanimelist.net/character/11/Edward_Elric",	
87.	rating = 9.3f,	
88.	episodesOrChapters = 64,	
89.	year = 2009,	
90.	typeResId = R.string.type_anime	
91.	)	
92.	)	
93.	}	

Tabel 11. Source Code AnimeCharacter Compose

1.	package com.example.clickanime.model
2.	
3.	data class AnimeCharacter(
4.	val id: Int,
5.	val nameResId: Int,
6.	val animeResId: Int,
7.	val genreResId: Int,
8.	val descriptionResId: Int,
9.	val abilityResId: Int,
10.	val imageUrl: String,
11.	val myAnimeListUrl: String,
12.	val rating: Float,
13.	val episodesOrChapters: Int,
14.	val year: Int,

15.	<code>val typeResId: Int</code>
16.	<code>)</code>

Tabel 12. SourceCode NavGraph Compose

1.	<code>package com.example.clickanime.navigation</code>
2.	
3.	<code>import androidx.compose.runtime.Composable</code>
4.	<code>import androidx.navigation.NavHostController</code>
5.	<code>import androidx.navigation.NavType</code>
6.	<code>import androidx.navigation.compose.NavHost</code>
7.	<code>import androidx.navigation.compose.composable</code>
8.	<code>import androidx.navigation.navArgument</code>
9.	<code>import com.example.clickanime.data.AnimeData</code>
10.	<code>import</code> <code>com.example.clickanime.ui.detail.DetailScreen</code>
11.	<code>import</code> <code>com.example.clickanime.ui.home.HomeScreen</code>
12.	<code>import</code> <code>com.example.clickanime.ui.language.LanguageScreen</code>
13.	
14.	<code>@Composable</code>
15.	<code>fun NavGraph(</code>
16.	<code> navController: NavHostController,</code>
17.	<code> onLocaleChange: (String) -> Unit</code>
18.	<code>) {</code>

```

19.     NavHost (
20.         navController = navController,
21.         startDestination = Screen.Home.route
22.     ) {
23.         composable(Screen.Home.route) {
24.             HomeScreen (
25.                 characters =
AnimeData.characters,
26.                 onDetailClick = { characterId -
>
27.
navController.navigate(Screen.Detail.createRoute(characterId))
28.                 },
29.                 onLanguageClick = {
30.
navController.navigate(Screen.Language.route)
31.                 }
32.             )
33.         }
34.         composable(
35.             route = Screen.Detail.route,
36.             arguments =
listOf(navArgument("characterId") { type =
NavType.IntType })
37.         ) { backStackEntry ->

```

38.	<pre> val characterId = backStackEntry.arguments?.getInt("characterId") ?: return@composable </pre>
39.	<pre> val character = AnimeData.characters.find { it.id == characterId } </pre>
40.	<pre> character?.let { </pre>
41.	<pre> DetailScreen(</pre>
42.	<pre> character = it, </pre>
43.	<pre> onBackPressed = { navController.popBackStack() } </pre>
44.	<pre>) </pre>
45.	<pre> } </pre>
46.	<pre> } </pre>
47.	<pre> composable(Screen.Language.route) { </pre>
48.	<pre> LanguageScreen(</pre>
49.	<pre> onLanguageSelected = { langCode -> </pre>
50.	<pre> onLocaleChange(langCode) </pre>
51.	<pre> navController.popBackStack() </pre>
52.	<pre> }, </pre>
53.	<pre> onBackPressed = { navController.popBackStack() } </pre>
54.	<pre>) </pre>
55.	<pre> } </pre>

56.	}
57.	}

Tabel 13. Source Code Screen Compose

1.	package com.example.clickanime.navigation
2.	
3.	sealed class Screen(val route: String) {
4.	object Home : Screen("home")
5.	object Detail : Screen("detail/{characterId}") {
6.	fun createRoute(characterId: Int) =
	"detail/\$characterId"
7.	}
8.	object Language : Screen("language")
9.	}

MainActivity.Kt XML

Tabel 14. Source Code MainActivity XML

1.	package com.example.animepopular
2.	
3.	import android.content.Context
4.	import android.os.Bundle
5.	import androidx.appcompat.app.AppCompatActivity
6.	import
	androidx.navigation.fragment.NavHostFragment

```

7. import
   androidx.navigation.ui.setupActionBarWithNavController
8. import
   com.example.animepopular.databinding.ActivityMain
   Binding
9. import java.util.Locale
10.
11. class MainActivity : AppCompatActivity() {
12.
13.     private lateinit var binding:
   ActivityMainBinding
14.
15.     override fun attachBaseContext(newBase:
   Context) {
16.         val prefs =
   newBase.getSharedPreferences("settings",
   Context.MODE_PRIVATE)
17.         val lang = prefs.getString("language",
   "en") ?: "en"
18.         val locale = Locale(lang)
19.         Locale.setDefault(locale)
20.         val config =
   newBase.resources.configuration
21.         config.setLocale(locale)
22.         val context =
   newBase.createConfigurationContext(config)

```

23.	super.attachBaseContext(context)	
24.	}	
25.		
26.	override fun onCreate(savedInstanceState: Bundle?) {	
27.	super.onCreate(savedInstanceState)	
28.	binding	=
	ActivityMainBinding.inflate(layoutInflater)	
29.	setContentView(binding.root)	
30.		
31.	val navHostFragment	=
	supportFragmentManager	
32.	.findFragmentById(R.id.nav_host_fragment)	as
	NavHostFragment	
33.	val navController	=
	navHostFragment.navController	
34.		
35.	setSupportActionBar(binding.toolbar)	
36.	setupActionBarWithNavController(navController)	
37.		
38.	navController.addOnDestinationChangedListener {	
	_, destination, _ ->	
39.	supportActionBar?.title	=
	destination.label	

40.	}
41.	}
42.	
43.	override fun onSupportNavigateUp(): Boolean {
44.	val navHostFragment =
	supportFragmentManager
45.	.findFragmentById(R.id.nav_host_fragment) as
	NavHostFragment
46.	return
	navHostFragment.navController.navigateUp()
	super.onSupportNavigateUp()
47.	}
48.	}

Tabel 15. Source Code AnimeAdapter XML

1.	package com.example.animepopular.adapter
2.	
3.	import android.content.Context
4.	import android.view.LayoutInflater
5.	import android.view.ViewGroup
6.	import androidx.recyclerview.widget.DiffUtil
7.	import androidx.recyclerview.widget.ListAdapter
8.	import androidx.recyclerview.widget.RecyclerView
9.	import com.bumptech.glide.Glide

```

10. import
    com.example.animepopular.databinding.ItemAnimeBin
    ding
11. import com.example.animepopular.model.AnimeItem
12.
13. class AnimeAdapter(
14.     private val onImdbClick: (AnimeItem) -> Unit,
15.     private val onDetailClick: (AnimeItem) -> Unit
16. ) : ListAdapter<AnimeItem,
    AnimeAdapter.AnimeViewHolder>(DiffCallback()) {
17.
18.     override fun onCreateViewHolder(parent:
    ViewGroup, viewType: Int): AnimeViewHolder {
19.         val binding = ItemAnimeBinding.inflate(
20.             LayoutInflater.from(parent.context),
    parent, false
21.         )
22.         return AnimeViewHolder(binding,
    parent.context)
23.     }
24.
25.     override fun onBindViewHolder(holder:
    AnimeViewHolder, position: Int) {
26.         holder.bind(getItem(position))
27.     }
28.

```

```

29.         inner class AnimeViewHolder(
30.             private val binding: ItemAnimeBinding,
31.             private val context: Context
32.         ) : RecyclerView.ViewHolder(binding.root) {
33.
34.             fun bind(anime: AnimeItem) {
35.                 val prefs = context.getSharedPreferences("settings",
context.MODE_PRIVATE)
36.                 val isId = prefs.getString("language", "en") == "id"
37.
38.                 binding.tvTitle.text = if (isId)
anime.titleId else anime.titleEn
39.                 binding.tvYear.text = anime.year.toString()
40.                 binding.tvGenre.text = if (isId)
anime.genreId else anime.genreEn
41.                 binding.tvPlot.text = if (isId)
anime.plotId else anime.plotEn
42.                 binding.tvRating.text = anime.rating
43.                 binding.tvEpisodes.text = anime.episodes
44.
45.                 Glide.with(context)
46.                     .load(anime.imageUrl)

```

47.	<code>.placeholder(android.R.drawable.ic_menu_gallery)</code>
48.	<code>.error(android.R.drawable.ic_menu_gallery)</code>
49.	<code>.into(binding.imgPoster)</code>
50.	
51.	<code>binding.btnImdb.setOnClickListener {</code> <code>onImdbClick(anime) }</code>
52.	<code>binding.btnDetail.setOnClickListener</code> <code>{ onDetailClick(anime) }</code>
53.	<code>}</code>
54.	<code>}</code>
55.	
56.	<code>class DiffCallback :</code> <code>DiffUtil.ItemCallback<AnimeItem>() {</code>
57.	<code>override fun areItemsTheSame(oldItem:</code> <code>AnimeItem, newItem: AnimeItem) = oldItem.id ==</code> <code>newItem.id</code>
58.	<code>override fun areContentsTheSame(oldItem:</code> <code>AnimeItem, newItem: AnimeItem) = oldItem ==</code> <code>newItem</code>
59.	<code>}</code>
60.	<code>}</code>

Tabel 16. Source Code AnimeRepository XML

1.	<code>package com.example.animepopular.data</code>
2.	
3.	<code>import com.example.animepopular.model.AnimeItem</code>

4.	
5.	object AnimeRepository {
6.	
7.	fun getAnimeList(): List<AnimeItem> = listOf(
8.	AnimeItem(
9.	id = 1,
10.	titleEn = "Attack on Titan",
11.	titleId = "Serangan Titan",
12.	year = 2013,
13.	genreEn = "Action / Dark Fantasy",
14.	genreId = "Aksi / Fantasi Gelap",
15.	plotEn = "In a world where humanity lives within enormous walled cities to protect themselves from Titans, gigantic humanoid creatures, young Eren Yeager vows to exterminate all Titans after his mother is consumed by one.",
16.	plotId = "Di dunia di mana umat manusia hidup dalam kota berdinding raksasa untuk melindungi diri dari Titan, makhluk humanoid raksasa, Eren Yeager yang muda bersumpah untuk memusnahkan semua Titan setelah ibunya dimakan oleh salah satunya.",
17.	studioEn = "Wit Studio / MAPPA",
18.	studioId = "Wit Studio / MAPPA",
19.	episodes = "87 Episodes",
20.	rating = "★ 9.0 / 10",

21.	imageUrl =
	"https://cdn.myanimelist.net/images/anime/10/47347.jpg",
22.	imdbUrl =
	"https://www.imdb.com/title/tt2560140/"
23.	),
24.	AnimeItem(
25.	id = 2,
26.	titleEn = "Hunter x Hunter",
27.	titleId = "Hunter x Hunter",
28.	year = 2011,
29.	genreEn = "Adventure / Fantasy",
30.	genreId = "Petualangan / Fantasi",
31.	plotEn = "Gon Freecss aspires to become a Hunter, a licensed professional who specializes in tracking down fantastical treasures, rare animals, or even other people, to find his missing father who was also a legendary Hunter.",
32.	plotId = "Gon Freecss bercita-cita menjadi Hunter, seorang profesional berlisensi yang mengkhususkan diri dalam melacak harta ajaib, hewan langka, bahkan orang lain, untuk menemukan ayahnya yang hilang yang juga seorang Hunter legendaris.",
33.	studioEn = "Madhouse",
34.	studioId = "Madhouse",
35.	episodes = "148 Episodes",

36.	rating = "★ 9.0 / 10",
37.	imageUrl = "https://cdn.myanimelist.net/images/anime/11/33657.jpg",
38.	imdbUrl = "https://www.imdb.com/title/tt2098220/"
39.	),
40.	AnimeItem(id = 3, titleEn = "Gurren Lagann", titleId = "Gurren Lagann", year = 2007, genreEn = "Mecha / Action", genreId = "Mecha / Aksi", plotEn = "Simon and Kamina were born and raised in a deep underground village. One day they find a small mecha which helps them break through the surface and discover the vast world above, where they must fight against the Spiral King.", plotId = "Simon dan Kamina lahir dan dibesarkan di desa bawah tanah. Suatu hari mereka menemukan mecha kecil yang membantu mereka menembus permukaan dan menemukan dunia luas di atas, di mana mereka harus melawan Raja Spiral.", studioEn = "Gainax", studioId = "Gainax",
49.	
50.	

51.	episodes = "27 Episodes",
52.	rating = "★ 8.7 / 10",
53.	imageUrl = "https://myanimelist.net/images/anime/4/51231.jpg",
54.	imdbUrl = "https://www.imdb.com/title/tt0948103/"
55.	),
56.	AnimeItem(id = 4, titleEn = "Code Geass", titleId = "Code Geass", year = 2008, genreEn = "Mecha / Drama", genreId = "Mecha / Drama", plotEn = "A disgraced prince uses a power called Geass to lead a rebellion against the Holy Britannian Empire that has conquered Japan, while hiding his identity as the masked revolutionary Zero.", plotId = "Seorang pangeran yang dipermalukan menggunakan kekuatan bernama Geass untuk memimpin pemberontakan melawan Kekaisaran Britannia Suci yang telah menaklukkan Jepang, sambil menyembunyikan identitasnya sebagai revolusioner bertopeng Zero.", studioEn = "Sunrise",
65.	

66.	studioId = "Sunrise",
67.	episodes = "50 Episodes",
68.	rating = "★ 8.7 / 10",
69.	imageUrl = "https://myanimelist.net/images/anime/1032/1350881.jpg",
70.	imdbUrl = "https://www.imdb.com/title/tt0994314/"
71.	),
72.	AnimeItem(id = 5, titleEn = "Demon Slayer", titleId = "Pembasmi Iblis", year = 2019, genreEn = "Action / Supernatural", genreId = "Aksi / Supranatural", plotEn = "Tanjiro Kamado becomes a Demon Slayer of the Demon Slayer Corps after his family was slaughtered and his younger sister Nezuko turned into a demon, to avenge their family and cure his sister.", plotId = "Tanjiro Kamado menjadi Pembasmi Iblis setelah keluarganya dibantai dan adiknya Nezuko berubah menjadi iblis, untuk membalas dendam keluarga dan menyembuhkan adiknya.", studioEn = "ufotable",
81.	

82.	studioId = "ufotable",																		
83.	episodes = "44+ Episodes",																		
84.	rating = "★ 8.7 / 10",																		
85.	imageUrl = "https://myanimelist.net/images/anime/1286/998891.jpg",																		
86.	imdbUrl = "https://www.imdb.com/title/tt9335498/"																		
87.	),																		
88.	AnimeItem(<tr> <td>89.</td><td>id = 6,</td></tr> <tr> <td>90.</td><td>titleEn = "Trigun",</td></tr> <tr> <td>91.</td><td>titleId = "Trigun",</td></tr> <tr> <td>92.</td><td>year = 1998,</td></tr> <tr> <td>93.</td><td>genreEn = "Sci-Fi / Western",</td></tr> <tr> <td>94.</td><td>genreId = "Fiksi Ilmiah / Barat",</td></tr> <tr> <td>95.</td><td>plotEn = "Vash the Stampede is a legendary gunman with a bounty of 60 billion double dollars on his head. He roams the desert planet Gunsmoke, followed by two insurance agents, though he is actually a pacifist.",</td></tr> <tr> <td>96.</td><td>plotId = "Vash the Stampede adalah penembak legendaris dengan hadiah 60 miliar double dollar di kepalanya. Ia berkeliaran di planet gurun Gunsmoke, diikuti dua agen asuransi, meskipun sebenarnya ia seorang pasifis.",</td></tr> <tr> <td>97.</td><td>studioEn = "Madhouse",</td></tr>	89.	id = 6,	90.	titleEn = "Trigun",	91.	titleId = "Trigun",	92.	year = 1998,	93.	genreEn = "Sci-Fi / Western",	94.	genreId = "Fiksi Ilmiah / Barat",	95.	plotEn = "Vash the Stampede is a legendary gunman with a bounty of 60 billion double dollars on his head. He roams the desert planet Gunsmoke, followed by two insurance agents, though he is actually a pacifist.",	96.	plotId = "Vash the Stampede adalah penembak legendaris dengan hadiah 60 miliar double dollar di kepalanya. Ia berkeliaran di planet gurun Gunsmoke, diikuti dua agen asuransi, meskipun sebenarnya ia seorang pasifis.",	97.	studioEn = "Madhouse",
89.	id = 6,																		
90.	titleEn = "Trigun",																		
91.	titleId = "Trigun",																		
92.	year = 1998,																		
93.	genreEn = "Sci-Fi / Western",																		
94.	genreId = "Fiksi Ilmiah / Barat",																		
95.	plotEn = "Vash the Stampede is a legendary gunman with a bounty of 60 billion double dollars on his head. He roams the desert planet Gunsmoke, followed by two insurance agents, though he is actually a pacifist.",																		
96.	plotId = "Vash the Stampede adalah penembak legendaris dengan hadiah 60 miliar double dollar di kepalanya. Ia berkeliaran di planet gurun Gunsmoke, diikuti dua agen asuransi, meskipun sebenarnya ia seorang pasifis.",																		
97.	studioEn = "Madhouse",																		

98.	studioId = "Madhouse",
99.	episodes = "26 Episodes",
100.	rating = "★ 8.2 / 10",
101.	imageUrl = "https://myanimelist.net/images/anime/7/203101.jpg",
102.	imdbUrl = "https://www.imdb.com/title/tt0251439/"
103.	),
104.	AnimeItem(id = 7, titleEn = "Spy x Family", titleId = "Spy x Family", year = 2022, genreEn = "Comedy / Action", genreId = "Komedi / Aksi", plotEn = "A spy on a secret mission must build a fake family to complete it. He recruits a telepathic girl as his daughter and an assassin as his wife, neither knowing each other's true identities.", plotId = "Seorang mata-mata dalam misi rahasia harus membangun keluarga palsu untuk menyelesaikannya. Ia merekrut gadis telepati sebagai putrinya dan seorang pembunuh bayaran sebagai istrinya, tanpa mengetahui identitas asli satu sama lain.",

113.	studioEn = "Wit Studio / CloverWorks",
114.	studioId = "Wit Studio / CloverWorks",
115.	episodes = "37 Episodes",
116.	rating = "★ 8.5 / 10",
117.	imageUrl = "https://myanimelist.net/images/anime/1111/1212621.jpg",
118.	imdbUrl = "https://www.imdb.com/title/tt13706018/"
119.	)
120.	)
121.	}

Tabel 17. Source Code AnimeItem XML

1.	package com.example.animepopular.model
2.	
3.	import android.os.Parcelable
4.	import kotlinx.parcelize.Parcelize
5.	
6.	@Parcelize
7.	data class AnimeItem(
8.	val id: Int,
9.	val titleEn: String,
10.	val titleId: String,
11.	val year: Int,

12.	val genreEn: String,
13.	val genreId: String,
14.	val plotEn: String,
15.	val plotId: String,
16.	val studioEn: String,
17.	val studioId: String,
18.	val episodes: String,
19.	val rating: String,
20.	val imageUrl: String,
21.	val imdbUrl: String
22.	) : Parcelable

Tabel 18. Source Code HomeFragment

1.	package com.example.animepopular.ui.home
2.	
3.	import android.content.Intent
4.	import android.net.Uri
5.	import android.os.Bundle
6.	import android.view.LayoutInflater
7.	import android.view.Menu
8.	import android.view.MenuInflater
9.	import android.view.MenuItem
10.	import android.view.View
11.	import android.view.ViewGroup
12.	import androidx.core.view.MenuProvider


```

13. import androidx.fragment.app.Fragment
14. import androidx.fragment.app.viewModels
15. import androidx.lifecycle.Lifecycle
16. import
    androidx.navigation.fragment.findNavController
17. import
    androidx.recyclerview.widget.GridLayoutManager
18. import
    androidx.recyclerview.widget.LinearLayoutManager
19. import com.example.animepopular.R
20. import
    com.example.animepopular.adapter.AnimeAdapter
21. import
    com.example.animepopular.adapter.CarouselAdapter
22. import
    com.example.animepopular.databinding.FragmentHomeBinding
23. import com.example.animepopular.model.AnimeItem
24. import
    com.example.animepopular.viewmodel.AnimeViewModel
25.
26. class HomeFragment : Fragment() {
27.
28.     private var _binding: FragmentHomeBinding? =
        null
29.     private val binding get() = _binding!!

```

```

30.     private val viewModel: AnimeViewModel by
      viewModel()
31.
32.     private lateinit var animeAdapter:
      AnimeAdapter
33.     private lateinit var carouselAdapter:
      CarouselAdapter
34.
35.     override fun onCreateView(
36.         inflater: LayoutInflater, container:
      ViewGroup?,
37.         savedInstanceState: Bundle?
38.     ): View {
39.         _binding =
      FragmentHomeBinding.inflate(inflater, container,
      false)
40.         return binding.root
41.     }
42.
43.     override fun onViewCreated(view: View,
      savedInstanceState: Bundle?) {
44.         super.onViewCreated(view,
      savedInstanceState)
45.
46.         setupMenu()
47.         setupCarousel()

```

48.	setupRecyclerView()
49.	observeData()
50.	}
51.	
52.	private fun setupMenu() {
53.	requireActivity().addMenuProvider(object : MenuProvider {
54.	override fun onCreateMenu(menu: Menu, menuInflater: MenuInflater) {
55.	menuInflater.inflate(R.menu.home_menu, menu)
56.	}
57.	override fun onMenuItemSelected(menuItem: MenuItem): Boolean {
58.	return when (menuItem.itemId) {
59.	R.id.action_language -> {
60.	findNavController().navigate(R.id.action_homeFra gment_to_languageFragment)
61.	true
62.	}
63.	else -> false
64.	}
65.	}
66.	}, viewLifecycleOwner, Lifecycle.State.RESUMED)

```

67.     }
68.
69.     private fun setupCarousel() {
70.         carouselAdapter = CarouselAdapter(
71.             onImdbClick = { anime ->
72.                 openUrl(anime.imdbUrl) },
73.             onDetailClick = { anime ->
74.                 navigateToDetail(anime) }
75.         )
76.         binding.viewPagerCarousel.adapter =
77.             carouselAdapter
78.
79.         // Attach dots indicator
80.         binding.dotsIndicator.attachTo(binding.viewPager
81.             Carousel)
82.     }
83.
84.     private fun setupRecyclerView() {
85.         animeAdapter = AnimeAdapter(
86.             onImdbClick = { anime ->
87.                 openUrl(anime.imdbUrl) },
88.             onDetailClick = { anime ->
89.                 navigateToDetail(anime) }
90.         )

```

86.	<pre> val layoutManager = if (resources.configuration.orientation == </pre>
87.	<pre> android.content.res.Configuration.ORIENTATION_LA NDSCAPE) { </pre>
88.	<pre> GridLayoutManager(requireContext(), 2) </pre>
89.	<pre> } else { </pre>
90.	<pre> LinearLayoutManager(requireContext()) </pre>
91.	<pre> } </pre>
92.	<pre> binding.rvAnime.layoutManager = layoutManager </pre>
93.	<pre> binding.rvAnime.adapter = animeAdapter </pre>
94.	<pre> } </pre>
95.	
96.	<pre> private fun observeData() { </pre>
97.	<pre> viewModel.animeList.observe(viewLifecycleOwner) { list -> </pre>
98.	<pre> animeAdapter.submitList(list) </pre>
99.	<pre> carouselAdapter.submitList(list) </pre>
100.	<pre> } </pre>
101.	<pre> } </pre>
102.	
103.	<pre> private fun openUrl(url: String) { </pre>

104.	val intent = Intent(Intent.ACTION_VIEW, Uri.parse(url))
105.	startActivity(intent)
106.	}
107.	
108.	private fun navigateToDetail(anime: AnimeItem) {
109.	val action = HomeFragmentDirections.actionHomeFragmentToDetailFragment(anime)
110.	findNavController().navigate(action)
111.	}
112.	
113.	override fun onDestroyView() {
114.	super.onDestroyView()
115.	_binding = null
116.	}
117.	}

Tabel 19. Source Code DetailFragment XML

1.	package com.example.animepopular.ui.detail
2.	
3.	import android.content.Context
4.	import android.content.Intent
5.	import android.net.Uri
6.	import android.os.Bundle

```

7. import android.view.LayoutInflater
8. import android.view.View
9. import android.view.ViewGroup
10. import androidx.fragment.app.Fragment
11. import com.bumptech.glide.Glide
12. import
    com.example.animepopular.databinding.FragmentDetailBinding
13. import com.example.animepopular.model.AnimeItem
14.
15. class DetailFragment : Fragment() {
16.
17.     private var _binding: FragmentDetailBinding?
    = null
18.     private val binding get() = _binding!!
19.
20.     override fun onCreateView(
21.         inflater: LayoutInflater, container:
    ViewGroup?,
22.         savedInstanceState: Bundle?
23.     ): View {
24.         _binding =
    FragmentDetailBinding.inflate(inflater,
    container, false)
25.         return binding.root
26.     }

```

```

27.
28.     override fun onCreateView(view: View,
savedInstanceState: Bundle?) {
29.         super.onCreateView(view,
savedInstanceState)
30.
31.         val anime =
DetailFragmentArgs.fromBundle(requireArguments()
).anime
32.         val isIndonesian = getCurrentLanguage()
== "id"
33.         bindData(anime, isIndonesian)
34.     }
35.
36.     private fun getCurrentLanguage(): String {
37.         val prefs =
requireContext().getSharedPreferences("settings"
, Context.MODE_PRIVATE)
38.         return prefs.getString("language", "en")
?: "en"
39.     }
40.
41.     private fun bindData(anime: AnimeItem,
isIndonesian: Boolean) {
42.         Glide.with(this)
43.             .load(anime.imageUrl)

```


44.	<code>.placeholder(android.R.drawable.ic_menu_gallery)</code>
45.	<code>.error(android.R.drawable.ic_menu_gallery)</code>
46.	<code>.centerCrop()</code>
47.	<code>.into(binding.imgDetailPoster)</code>
48.	
49.	<code>binding.tvDetailTitle.text = if</code> <code>(isIndonesian) anime.titleId else anime.titleEn</code>
50.	<code>binding.tvDetailYear.text =</code> <code>anime.year.toString()</code>
51.	<code>binding.tvDetailGenre.text = if</code> <code>(isIndonesian) anime.genreId else anime.genreEn</code>
52.	<code>binding.tvDetailStudio.text = if</code> <code>(isIndonesian) anime.studioId else anime.studioEn</code>
53.	<code>binding.tvDetailEpisodes.text =</code> <code>anime.episodes</code>
54.	<code>binding.tvDetailRating.text =</code> <code>anime.rating</code>
55.	<code>binding.tvDetailPlot.text = if</code> <code>(isIndonesian) anime.plotId else anime.plotEn</code>
56.	
57.	<code>binding.btnDetailImdb.setOnClickListener</code> <code>{</code>
58.	<code>val intent =</code> <code>Intent(Intent.ACTION_VIEW,</code> <code>Uri.parse(anime.imdbUrl))</code>

59.	startActivity(intent)
60.	}
61.	}
62.	
63.	override fun onDestroyView() {
64.	super.onDestroyView()
65.	_binding = null
66.	}
67.	}

Tabel 20. Source Code AnimeViewModel XML

1.	package com.example.animepopular.viewmodel
2.	
3.	import androidx.lifecycle.LiveData
4.	import androidx.lifecycle.MutableLiveData
5.	import androidx.lifecycle.ViewModel
6.	import
	com.example.animepopular.data.AnimeRepository
7.	import com.example.animepopular.model.AnimeItem
8.	
9.	class AnimeViewModel : ViewModel() {
10.	
11.	private val _animeList =
	MutableLiveData<List<AnimeItem>>()
12.	val animeList: LiveData<List<AnimeItem>> =
	_animeList

13.	
14.	init {
15.	_animeList.value =
	AnimeRepository.getAnimeList()
16.	}
17.	
18.	fun getAnimeList(): List<AnimeItem> =
	AnimeRepository.getAnimeList()
19.	}

Activity_main.xml

Tabel 21. Source Code Activity_main.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<androidx.coordinatorlayout.widget.CoordinatorL
3.	ayout
4.	xmlns:android="http://schemas.android.com/apk/r
5.	es/android"
6.	xmlns:app="http://schemas.android.com/apk/res-
7.	auto"
8.	android:layout_width="match_parent"
9.	android:layout_height="match_parent">
10.	<com.google.android.material.appbar.AppBarLayout
11.	t
	android:id="@+id/appBarLayout"
	android:layout_width="match_parent"
	android:layout_height="wrap_content"

12.	<code>android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"></code>
13.	
14.	<code><androidx.appcompat.widget.Toolbar</code>
15.	<code> android:id="@+id/toolbar"</code>
16.	<code> android:layout_width="match_parent"</code>
17.	<code> android:layout_height="?attr/actionBarSize"</code>
18.	<code> android:background="?attr/colorPrimary"</code>
19.	<code> app:popupTheme="@style/ThemeOverlay.MaterialComponents.Light"</code>
20.	<code> app:title="@string/app_name" /></code>
21.	
22.	<code></com.google.android.material.appbar.AppBarLayout></code>
23.	
24.	<code><androidx.fragment.app.FragmentContainerView</code>
25.	<code> android:id="@+id/nav_host_fragment"</code>
26.	<code> android:name="androidx.navigation.fragment.NavHostFragment"</code>
27.	<code> android:layout_width="match_parent"</code>
28.	<code> android:layout_height="match_parent"</code>
29.	<code> app:layout_behavior="@string/appbar_scrolling_view_behavior"</code>
30.	<code> app:defaultNavHost="true"</code>
31.	<code> app:navGraph="@navigation/nav_graph" /></code>
32.	

33.	<code></androidx.coordinatorlayout.widget.CoordinatorLayout></code>
-----	---

Tabel 22. Source Code item_anime.xml

1.	<code><?xml version="1.0" encoding="utf-8"?></code>
2.	<code><com.google.android.material.card.MaterialCardView</code>
3.	<code>xmlns:android="http://schemas.android.com/apk/res/android"</code>
4.	<code>xmlns:app="http://schemas.android.com/apk/res-auto"</code>
5.	<code>android:layout_width="match_parent"</code>
6.	<code>android:layout_height="wrap_content"</code>
7.	<code>android:layout_marginBottom="12dp"</code>
8.	<code>app:cardBackgroundColor="@color/card_background"</code>
9.	<code>app:cardCornerRadius="16dp"</code>
10.	<code>app:cardElevation="4dp"></code>
11.	
12.	<code><LinearLayout</code>
13.	<code>android:layout_width="match_parent"</code>
14.	<code>android:layout_height="wrap_content"</code>
15.	<code>android:orientation="horizontal"</code>
16.	<code>android:padding="12dp"></code>
17.	
18.	
19.	<code><com.google.android.material.imageview.ShapeableImageView</code>
20.	<code>android:id="@+id/imgPoster"</code>
21.	<code>android:layout_width="100dp"</code>
22.	<code>android:layout_height="140dp"</code>

23.	android:scaleType="centerCrop"
24.	app:shapeAppearanceOverlay="@style/RoundedCornerImage" />
25.	
26.	
27.	<LinearLayout
28.	android:layout_width="0dp"
29.	android:layout_height="wrap_content"
30.	android:layout_weight="1"
31.	android:orientation="vertical"
32.	android:paddingStart="12dp">
33.	
34.	
35.	<LinearLayout
36.	android:layout_width="match_parent"
37.	android:layout_height="wrap_content"
38.	android:orientation="horizontal"
39.	android:gravity="center_vertical">
40.	
41.	<TextView
42.	android:id="@+id/tvTitle"
43.	android:layout_width="0dp"
44.	android:layout_height="wrap_content"
45.	android:layout_weight="1"
46.	android:textColor="@color/text_primary"

47.	android:textSize="16sp"
48.	android:textStyle="bold"
49.	android:maxLines="2"
50.	android:ellipsize="end" />
51.	
52.	<TextView
53.	android:id="@+id/tvYear"
54.	android:layout_width="wrap_content"
55.	android:layout_height="wrap_content"
56.	android:textColor="@color/text_secondary"
57.	android:textSize="12sp"
58.	android:paddingStart="4dp"
	/>
59.	</LinearLayout>
60.	
61.	
62.	<TextView
63.	android:id="@+id/tvGenre"
64.	android:layout_width="wrap_content"
65.	android:layout_height="wrap_content"
66.	android:layout_marginTop="2dp"
67.	android:background="@drawable/badge_background"
68.	android:paddingHorizontal="8dp"
69.	android:paddingVertical="2dp"
70.	android:textColor="@color/accent_color"
71.	android:textSize="11sp" />

```

72.
73.
74.         <LinearLayout
75.
76.             android:layout_width="match_parent"
77.
78.             android:layout_height="wrap_content"
79.
80.             android:orientation="horizontal"
81.
82.                 android:layout_marginTop="4dp">
83.
84.                 <TextView
85.
86.                     android:id="@+id/tvRating"
87.
88.                     android:layout_width="0dp"
89.
90.                     android:layout_height="wrap_content"
91.
92.                     android:layout_weight="1"
93.
94.                     android:textColor="@color/rating_color"
95.
96.                     android:textSize="12sp"
97.
98.                     android:textStyle="bold" />
99.
100.                 <TextView
101.
102.                     android:id="@+id/tvEpisodes"
103.
104.                     android:layout_width="wrap_content"
105.
106.                     android:layout_height="wrap_content"
107.
108.                     android:textColor="@color/text_secondary"
109.
110.                     android:textSize="11sp" />
111.             </LinearLayout>

```


96.	
97.	
98.	<TextView
99.	android:id="@+id/tvPlot"
100.	android:layout_width="match_parent"
101.	android:layout_height="wrap_content"
102.	android:layout_marginTop="4dp"
103.	android:maxLines="3"
104.	android:ellipsize="end"
105.	android:textColor="@color/text_secondary"
106.	android:textSize="12sp" />
107.	
108.	
109.	<LinearLayout
110.	android:layout_width="match_parent"
111.	android:layout_height="wrap_content"
112.	android:orientation="horizontal"
113.	android:layout_marginTop="8dp"
114.	android:gravity="start">
115.	
116.	<com.google.android.material.button.MaterialBut ton
117.	android:id="@+id/btnImdb"
118.	android:layout_width="wrap_content"
119.	android:layout_height="36dp"

120.	android:text="@string/btn_imdb"
121.	android:textSize="11sp"
122.	app:cornerRadius="18dp"
123.	style="@style/Widget.MaterialComponents.Button.OutlinedButton"
124.	app:strokeColor="@color/accent_color"
125.	android:textColor="@color/accent_color"
126.	android:layout_marginEnd="8dp" />
127.	
128.	<com.google.android.material.button.MaterialButton
129.	android:id="@+id/btnDetail"
130.	android:layout_width="wrap_content"
131.	android:layout_height="36dp"
132.	android:text="@string/btn_detail"
133.	android:textSize="11sp"
134.	app:cornerRadius="18dp"
135.	style="@style/Widget.MaterialComponents.Button"
136.	app:backgroundTint="@color/accent_color" />
137.	
138.	</LinearLayout>
139.	</LinearLayout>
140.	</LinearLayout>
141.	

142.	</com.google.android.material.card.MaterialCardView>
------	--

Tabel 23. Source Code fragment_home.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<androidx.core.widget.NestedScrollView
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	android:layout_width="match_parent"
6.	android:layout_height="match_parent"
7.	android:background="@color/background_dark"
8.	android:fillViewport="true">
9.	
10.	<LinearLayout
11.	android:layout_width="match_parent"
12.	android:layout_height="wrap_content"
13.	android:orientation="vertical"
14.	android:padding="12dp">
15.	
16.	
17.	<TextView
18.	android:layout_width="match_parent"
19.	android:layout_height="wrap_content"
20.	android:text="@string/label_highlights"
21.	android:textColor="@color/text_primary"
22.	android:textSize="18sp"
23.	android:textStyle="bold"

24.	android:layout_marginBottom="8dp"
	/>
25.	
26.	
27.	<androidx.viewpager2.widget.ViewPager2
28.	android:id="@+id/viewPagerCarousel"
29.	android:layout_width="match_parent"
30.	android:layout_height="220dp"
31.	android:layout_marginBottom="4dp"
	/>
32.	
33.	
34.	<com.tbuonomo.viewpagerdotsindicator.DotsIndicator
35.	android:id="@+id/dotsIndicator"
36.	android:layout_width="wrap_content"
37.	android:layout_height="wrap_content"
38.	android:layout_gravity="center_horizontal"
39.	android:layout_marginBottom="16dp"
40.	app:dotsColor="@color/accent_color"
41.	app:dotsSize="8dp"
42.	app:dotsSpacing="6dp"
43.	app:dotsWidthFactor="2.5" />
44.	
45.	
46.	<TextView
47.	android:layout_width="match_parent"
48.	android:layout_height="wrap_content"

49.	<code>android:text="@string/label_popular"</code>
50.	<code>android:textColor="@color/text_primary"</code>
51.	<code>android:textSize="18sp"</code>
52.	<code>android:textStyle="bold"</code>
53.	<code>android:layout_marginBottom="8dp"</code>
	<code></></code>
54.	
55.	
56.	<code><androidx.recyclerview.widget.RecyclerView</code>
57.	<code>android:id="@+id/rvAnime"</code>
58.	<code>android:layout_width="match_parent"</code>
59.	<code>android:layout_height="wrap_content"</code>
60.	<code>android:nestedScrollingEnabled="false"</code>
61.	<code>android:clipToPadding="false" /></code>
62.	
63.	<code></LinearLayout></code>
64.	
65.	<code></androidx.core.widget.NestedScrollView></code>

Tabel 24. Source Code fragment_detail.xml

1.	<code><?xml version="1.0" encoding="utf-8"?></code>
2.	<code><androidx.core.widget.NestedScrollView</code>
3.	<code>xmlns:android="http://schemas.android.com/apk/res/android"</code>
4.	<code>xmlns:app="http://schemas.android.com/apk/res-auto"</code>
5.	<code>android:layout_width="match_parent"</code>
6.	<code>android:layout_height="match_parent"</code>

7.	android:background="@color/background_dark">
8.	
9.	<LinearLayout
10.	android:layout_width="match_parent"
11.	android:layout_height="wrap_content"
12.	android:orientation="vertical">
13.	
14.	<ImageView
15.	android:id="@+id/imgDetailPoster"
16.	android:layout_width="match_parent"
17.	android:layout_height="280dp"
18.	android:scaleType="centerCrop"
19.	android:src="@drawable/ic_placeholder" />
20.	
21.	<View
22.	android:layout_width="match_parent"
23.	android:layout_height="60dp"
24.	android:layout_marginTop="-60dp"
25.	android:background="@drawable/gradient_bottom_fade" />
26.	
27.	<com.google.android.material.card.MaterialCardView
28.	android:layout_width="match_parent"
29.	android:layout_height="wrap_content"
30.	android:layout_marginHorizontal="16dp"
31.	android:layout_marginTop="-24dp"

32.	android:layout_marginBottom="16dp"
33.	app:cardBackgroundColor="@color/card_background"
34.	app:cardCornerRadius="20dp"
35.	app:cardElevation="8dp">
36.	
37.	<LinearLayout
38.	android:layout_width="match_parent"
39.	android:layout_height="wrap_content"
40.	android:orientation="vertical"
41.	android:padding="20dp">
42.	
43.	<LinearLayout
44.	android:layout_width="match_parent"
45.	android:layout_height="wrap_content"
46.	android:orientation="horizontal"
47.	android:gravity="center_vertical">
48.	
49.	<TextView
50.	android:id="@+id/tvDetailTitle"
51.	android:layout_width="0dp"
52.	android:layout_height="wrap_content"
53.	android:layout_weight="1"

```
54.         android:textColor="@color/text_primary"
55.                 android:textSize="22sp"
56.
57.         android:textStyle="bold" />
58.
59.         <TextView
60.
61.             android:id="@+id/tvDetailYear"
62.
63.             android:layout_width="wrap_content"
64.             android:layout_height="wrap_content"
65.
66.             android:background="@drawable/badge_background"
67.
68.             android:paddingHorizontal="10dp"
69.             android:paddingVertical="4dp"
70.
71.             android:textColor="@color/accent_color"
72.                 android:textSize="14sp"
73.
74.             android:textStyle="bold" />
75.         </LinearLayout>
76.
77.         <TextView
78.
79.             android:id="@+id/tvDetailGenre"
80.
81.             android:layout_width="wrap_content"
82.             android:layout_height="wrap_content"
83.
84.             android:layout_marginTop="8dp"
```


75.	android:background="@drawable/badge_background"
76.	android:paddingHorizontal="10dp"
77.	android:paddingVertical="3dp"
78.	android:textColor="@color/accent_color"
79.	android:textSize="13sp" />
80.	
81.	<View
82.	android:layout_width="match_parent"
83.	android:layout_height="1dp"
84.	android:layout_marginVertical="12dp"
85.	android:background="#3BFFFFFF" />
86.	
87.	<LinearLayout
88.	android:layout_width="match_parent"
89.	android:layout_height="wrap_content"
90.	android:orientation="horizontal"
91.	android:gravity="center_vertical">
92.	
93.	<TextView
94.	android:layout_width="wrap_content"
95.	android:layout_height="wrap_content"

96.	android:text="@string/label_rating"
97.	android:textColor="@color/text_secondary"
98.	android:textSize="13sp"
99.	android:layout_marginEnd="8dp" />
100.	
101.	<TextView
102.	android:id="@+id/tvDetailRating"
103.	android:layout_width="wrap_content"
104.	android:layout_height="wrap_content"
105.	android:textColor="@color/rating_color"
106.	android:textSize="16sp"
107.	android:textStyle="bold" />
108.	</LinearLayout>
109.	
110.	<LinearLayout
111.	android:layout_width="match_parent"
112.	android:layout_height="wrap_content"
113.	android:orientation="horizontal"
114.	android:gravity="center_vertical"
115.	android:layout_marginTop="8dp">
116.	
117.	<TextView

```

118.     android:layout_width="wrap_content"
119.
120.     android:layout_height="wrap_content"
121.
122.     android:text="@string/label_episodes"
123.
124.     android:textColor="@color/text_secondary"
125.         android:textSize="13sp"
126.
127.     android:layout_marginEnd="8dp" />
128.
129.         <TextView
130.
131.             android:id="@+id/tvDetailEpisodes"
132.
133.             android:layout_width="wrap_content"
134.
135.             android:layout_height="wrap_content"
136.
137.             android:textColor="@color/text_primary"
138.                 android:textSize="13sp"
139.         />
140.
141.     </LinearLayout>
142.
143.     <LinearLayout
144.
145.         android:layout_width="match_parent"
146.
147.         android:layout_height="wrap_content"
148.
149.         android:orientation="horizontal"
150.
151.         android:gravity="center_vertical"
152.
153.         android:layout_marginTop="8dp">

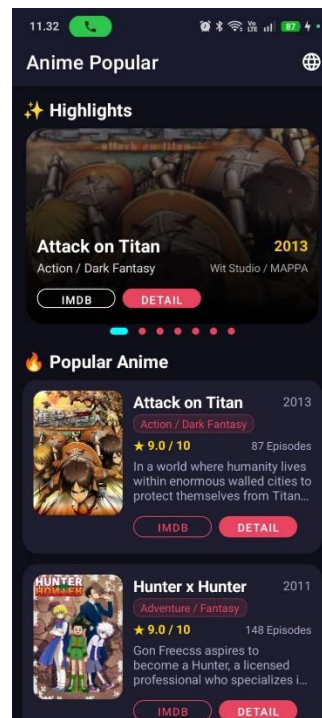
```

```
139.
140.         <TextView
141.             android:layout_width="wrap_content"
142.             android:layout_height="wrap_content"
143.             android:text="@string/label_studio"
144.             android:textColor="@color/text_secondary"
145.                 android:textSize="13sp"
146.             android:layout_marginEnd="8dp" />
147.
148.         <TextView
149.             android:id="@+id/tvDetailStudio"
150.             android:layout_width="wrap_content"
151.             android:layout_height="wrap_content"
152.             android:textColor="@color/text_primary"
153.                 android:textSize="13sp"
154.             />
155.         </LinearLayout>
156.
157.         <View
158.             android:layout_width="match_parent"
159.                 android:layout_height="1dp"
160.             android:layout_marginVertical="12dp"
161.             android:background="#3BFFFFFF" />
```

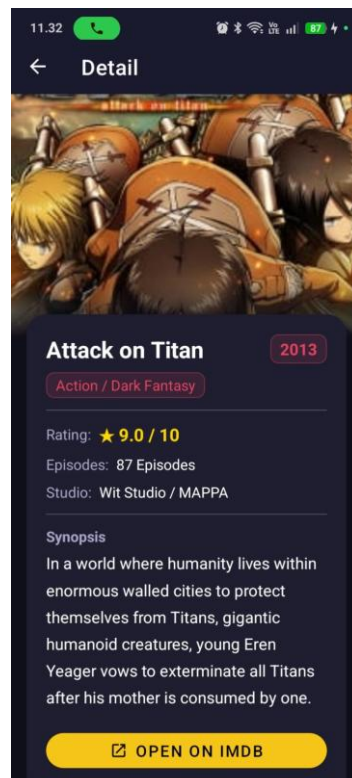
161.	
162.	<TextView
163.	android:layout_width="wrap_content"
164.	android:layout_height="wrap_content"
165.	android:text="@string/label_plot"
166.	android:textColor="@color/text_secondary"
167.	android:textSize="13sp"
168.	android:textStyle="bold"
169.	android:layout_marginBottom="6dp" />
170.	
171.	<TextView
172.	android:id="@+id/tvDetailPlot"
173.	android:layout_width="match_parent"
174.	android:layout_height="wrap_content"
175.	android:textColor="@color/text_primary"
176.	android:textSize="14sp"
177.	android:lineSpacingMultiplier="1.4" />
178.	
179.	<com.google.android.material.button.MaterialBut ton
180.	android:id="@+id/btnDetailImdb"
181.	android:layout_width="match_parent"

```
182.     android:layout_height="48dp"
183.
184.     android:text="@string/btn_open_imdb"
185.             android:textSize="14sp"
186.             app:cornerRadius="24dp"
187.
188.     app:backgroundTint="@color/imdb_yellow"
189.
190.     android:textColor="@android:color/black"
191.
192.     app:icon="@drawable/ic_open_external"
193.             app:iconGravity="textStart"
194.
195.     app:iconTint="@android:color/black" />
196.
197.         </LinearLayout>
198.     </com.google.android.material.card.MaterialCard
        View>
199.
200.     </LinearLayout>
201. </androidx.core.widget.NestedScrollView>
```

B. OUTPUT CODE



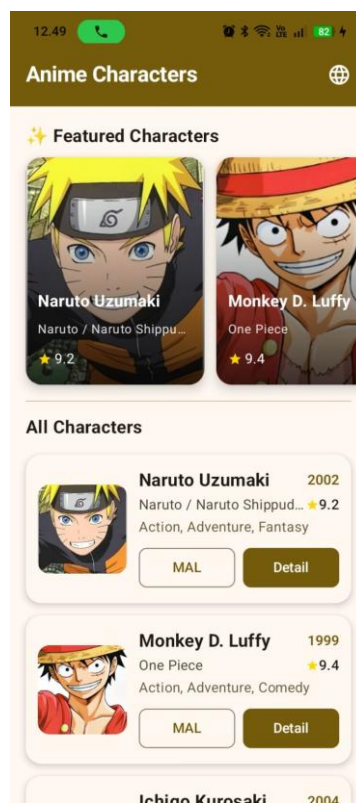
Gambar 17. Screenshot Hasil Jawaban Soal 1 XML



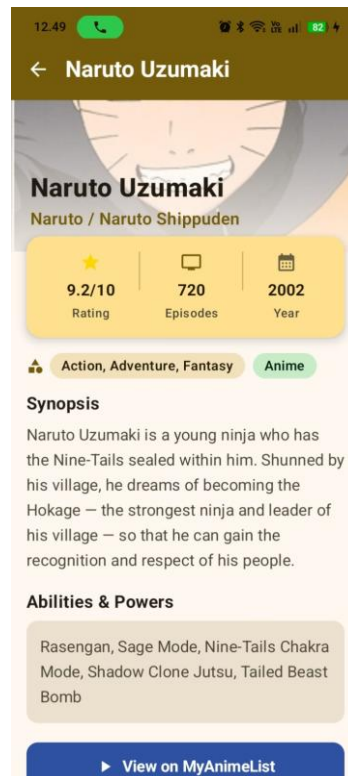
Gambar 18. Screenshot Hasil Jawaban Soal 1 XML



Gambar 19. Screenshot Hasil Jawaban Soal 1 XML



Gambar 20. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 21. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 22. Screenshot Hasil Jawaban Soal 1 Compose

C. PEMBAHASAN

MainActivity.kt Compose

File MainActivity.kt pada antarmuka Compose menandakan bahwa tampilan akan digambar sepenuhnya oleh mesin Compose melalui blok fungsi “setContent”. Di dalam file ini, program mengambil data preferensi bahasa pengguna yang tersimpan (Shared Preferences) dan menerapkan konfigurasi lokal secara terpusat untuk mendukung pergantian fitur multi-bahasa sebelum menyuntikkan tata letak awal “Surface” dan menginisiasi navigasi rute lewat “NavGraph” dengan bantuan “rememberNavController”.

AnimeViewModel.kt Compose

File AnimeViewModel.kt menerapkan pola arsitektur pengelolaan state (status data) reaktif agar informasi list tidak hilang seketika saat orientasi layar gawai diputar atau berubah. Dengan memanfaatkan fungsi “MutableStateFlow” dan “StateFlow” dari pustaka coroutines Kotlin, kelas ini bertanggung jawab sebagai wadah pengamat untuk menyimpan status indeks rotasi karakter (Carousel) serta memegang referensi daftar array pangkalan data anime yang nantinya dipantau dan digambar secara otomatis oleh komponen antarmuka.

AnimeData.kt Compose

File AnimeData.kt difungsikan oleh program sebagai modul repositori penyedia objek statis atau data rekaan murni (dummy) yang memuat informasi terkait entitas daftar anime secara spesifik. Di dalamnya, rincian daftar anime disajikan dalam pola cetak wujud "AnimeCharacter" di mana masing-masing nilainya ditautkan secara langsung menggunakan ID Resource String (seperti R.string.char1_name) untuk menyederhanakan mekanisme transisi pemutakhiran translasi multibahasa (Indonesia/Inggris), yang disandingkan dengan parameter penautan URL gambar ilustrasi.

AnimeCharacter.kt Compose

File AnimeCharacter.kt merupakan data class spesifik bawaan Kotlin yang dikhususkan sebagai cetak biru model pendefinisian atribut bagi satu objek karakter anime. Elemen-elemen penampung dalam kelas ini dirancang kuat untuk mengatur tipe variabel bawaan dari informasi inti, seperti atribut "id" yang berbentuk Integer, ID referensi teks bahasa untuk nama, rating dengan tipe pecahan (Float), dan juga nilai genre agar mudah diproses ulang di tiap penataan baris menu secara persisten.

NavGraph.kt Compose

File NavGraph.kt dioperasikan selaku pusat sistem kontrol lalu lintas perpindahan rute dan pergantian antar-layar memanfaatkan infrastruktur perpustakaan kerangka navigasi bawaan “NavHost”. Logika struktur di file ini menyusun rute tujuan destinasi layar secara runtut, meliputi pemetaan pembuatan rute ke ruang detail berbasis operan indeks argumen angka ID agar tepat mengekstrak objek tujuan ketika ditekan, serta mengakomodir rute transisi eksternal langsung menuju ruang pengaturan ganti bahasa aplikasi.

Screen.kt Compose

File `Screen.kt` menggunakan kelebihan konstruksi properti sealed class pada standar bahasa agar bisa mengunci dan merangkum ragam alamat string penamaan rute menjadi bagian statis yang aman dan tidak dapat diubah-ubah, di antaranya seperti “Home”, “Detail”, hingga “Language”. Tujuannya agar saat arsitektur layar membutuhkan pemanggilan di pengontrol `NavGraph`, program terhindar dari potensi kesalahan penghentian sistem yang konyol karena kekeliruan pengetikan (typo), terkhusus pada fungsionalitas rute mendetail yang melibatkan pembentukan string variabel lewat tambahan argumen.

MainActivity.kt XML

Dalam koridor pendekatan desain klasikal berbasis XML, `MainActivity.kt` berperan teguh menyokong struktur kerangka dasar Single-Activity yang membungkus semua tata letak transisi. Teknik “`findViewById`” diusangkan lalu digantikan dengan fungsional “`ViewBinding`” untuk mempertautkan tata letak dengan mudah, berlanjut menuju pembentukan sistem “`NavHostFragment`” guna menghubungkan manuver pergerakan fragmen yang berpadu dengan aksi kustom bilah pita (Toolbar) teratas serta merakit ulang pemuatan konfigurasi preferensi ragam penuturan bahasa bawaan secara periodik.

AnimeAdapter.kt XML

File `AnimeAdapter.kt` berfungsi penuh mencetak dan menggandakan skema barisan item `RecyclerView` tanpa batas dengan mengeksekusi kelas perpanjangan dari standar “`ListAdapter`” yang diperkuat kemampuan `DiffCallback` agar perbandingan dan pembaruan elemen baris tampak natural nan mulus tanpa perlu merender dari ulang. Memasuki fase “`ViewHolder`”, ia mentransformasi logika pembacaan variabel objek lalu menerjemahkannya untuk menyuntikkan data ke teks grafis sembari mendelegasikan pustaka tangguh (Glide) dalam merender URL jaring internet menjadi poster serta merekatkan event klik di tombol eksternal maupun internal navigasi.

AnimeRepository.kt XML

File `AnimeRepository.kt` adalah file terisolasi khusus wadah penyimpanan gudang data statis (dummy data) yang bertugas mereturn nilai kembalian kolektif untuk dikonsumsi. Tanpa memberatkan muatan tampilan visual, sistem pembentukan objek disusun mengandalkan wujud kelas data secara eksplisit dengan menyimpan masing-masing atributnya menjadi sepasang salinan, contohnya pada narasi sinopsis (“`plotEn`” dan “`plotId`”) yang disematkan secara spesifik agar fragmen detail UI layer dengan mudah memilihnya berdasar logika pengaturan bahasa yang dicari.

AnimeItem.kt XML

File `AnimeItem.kt` hanya ditugaskan sebagai pondasi abstraksi dan wujud dari kerangka arsitektur struktur pangkalan objek dengan jenis data class. Modifikasi paling mendasar di file kerangka model ini adalah pembungkusan lewat alat dekorator khusus “`@Parcelize`” lalu mengadopsi pewarisan “`Parcelable`”, di mana tindakan fungsional ini krusial di Android agar mesin sanggup menggulung, memadatkan bita, hingga mengemas semua serpihan informasi anime utuh secara efisien via argumen `SafeArgs` tanpa kerusakan korupsi struktur saat menyeberang fragment layar.

HomeFragment.kt XML

File `HomeFragment.kt` menempati pos krusial berwujud presentasi wajah utama menu aplikasi bagi para pengakses yang dijalin dari hasil ikatan `ViewBinding`. Lewat intervensi pada fungsi bawaan sistem perputaran gawai, kelas mendeteksi orientasi gawai aktual untuk otomatis menerapkan tampilan deret linear memanjang (`LinearLayoutManager`) pada potret, atau susunan bertumpuk dua kolom via pembagian `GridLayout` pada layar mendatar, dan ditutup lewat pembentukan tautan pemanggil fitur Carousel hingga pengaturan delegasi klik implisit (`intent URL`) yang interaktif.

DetailFragment.kt XML

Pada file `DetailFragment.kt`, kendali presentasi layar berpusat di ruang penerimaan argumentasi di mana objek “`Parcelable`” disedot sepenuhnya melalui bantuan keping perantara. Mekanisme internal mencari tahu rujukan riwayat penyetelan `Shared Preferences` yang pernah diinisiasi untuk mengembalikan teks informasi yang tepat, mengatur pelabelan properti teks, dan menjahitnya kembali ke dalam elemen tampilan antarmuka beserta implementasi peluncur gambar instan ke wadah kanvas yang tersedia.

AnimeViewModel.kt XML

File `AnimeViewModel.kt` untuk perancangan variasi arsitektur XML memegang mandat menyuplai deretan variabel referensi pasif yang dikoleksi eksklusif dari pusat gudang statis (`Repository`). Menyatu di dalam kerangka kerja `ViewModel`, daftar tersebut diselimuti pembungkus sistem asinkronisasi yang menjembatani lapisan fragmen dan data layer agar program terbebas dari kesalahan cacat fatal karena kehilangan alur memori dan mencegah terhapusnya daftar item saat pengguna gawai menggulir atau mengubah proporsi fisik ukuran secara dinamis.

activity_main.xml

File `activity_main.xml` berfungsi selayaknya denah dasar rumah atau kerangka terluar untuk menampung lalu lalang komponen transisi fragment di kerangka sistem `Single-Activity`. Disusun hanya dengan kode penanda (XML) yang statis, antarmuka utamanya mengandalkan satu tag “`NavHostFragment`” yang melingkupi sebagian besar luas ruang layar dengan sisipan dekorasi bilah kepala di bagian paling pucuk (`ActionBar`) yang memastikan integrasi reaktif antara menu antarmuka dan panel layar di dalamnya.

item_anime.xml

File `item_anime.xml` diplot merujuk pada tata letak kardus bingkai dari komponen satuan terkecil per baris tampilan list aplikasi list tanpa gulir yang berulang-ulang. Agar terlihat kekinian ala rancangan sistem `Material Design`, komponen “`CardView`” membungkus struktur guna menghasilkan rekayasa pangkasan tepi lengkung (`rounded corners`), seraya menyusun kotak muatan bagi rupa poster, paragraf pendeskripsi identitas, hingganja peletakan sepasang susunan instrumen tombol yang mendistribusikan tanggapan klik.

fragment_home.xml

File `fragment_home.xml` merupakan kanvas sketsa statik perwujudan kombinasi instrumen ruang tatap muka interaktif pertama yang berpusat pada halaman Beranda utama. Desain menyematkan satu deret tag komponen fungsi Carousel mengusung “`ViewPager2`”

bersanding penanda titik (dots indicator) di titik atas untuk disisipkan, sedangkan bidang bawah menampung ruang gulir “RecyclerView” memanjang secara proporsional supaya bisa memaparkan barisan komponen detail anime berjumlah banyak yang padu bersinkronasi dari tumpukan elemennya.

fragment_detail.xml

File `fragment_detail.xml` dibangun dengan menugasi area pinggiran antarmuka layar dengan komponen lentur pelindung guliran (NestedScrollView) demi memastikan rentetan rincian teks sinopsis naratif mendalam mampu dibaca hingga beres merespons usapan layar gawai pengguna. Sorotan pilar tampilan dibentangkan lewat kanvas visual poster utuh yang membentang disusul penyusunan bertumpuk untuk setiap properti statis keterangan label detail mulai judul hingga ulasan.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. PEMBAHASAN

Meskipun kehadiran LazyColumn pada Jetpack Compose menawarkan penulisan kode yang jauh lebih ringkas, RecyclerView berbasis XML tetap dipertahankan karena perannya yang sangat vital dalam memelihara aplikasi berskala besar (legacy). Banyak perusahaan dengan basis kode Android lama enggan melakukan migrasi total ke Compose karena proses refactoring menyeluruh akan memakan waktu yang lama, membutuhkan biaya besar, dan berisiko mengganggu stabilitas sistem yang sudah berjalan dengan baik. Oleh karena itu, penguasaan terhadap komponen lama ini tetap menjadi tuntutan industri yang tidak bisa dihindari oleh pengembang.

Di sisi lain, RecyclerView menawarkan tingkat kustomisasi yang lebih mendalam sekaligus menjadi sarana pembelajaran fondasi sistem Android yang esensial. Secara teknis, komponen ini memberikan kontrol penuh kepada pengembang untuk merancang animasi transisi yang rumit, tata letak asimetris, hingga jarak antar-item yang spesifik melalui manipulasi ItemAnimator, ItemDecoration, dan LayoutManager. Lebih jauh lagi, keterlibatan pola Adapter dan ViewHolder di dalamnya secara langsung melatih pengembang untuk menguasai mekanisme daur ulang memori perangkat secara efisien, yang merupakan keterampilan fundamental dalam mengoptimalkan performa antarmuka (UI) aplikasi seluler.

SOAL 3

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut pendapat kalian, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

A. PEMBAHASAN

Menurut saya, penerapan Clean Architecture itu sangat penting. Apalagi untuk proyek aplikasi berskala besar. Dengan Clean Architecture, seperti sedang menyusun kode-kode tersebut ke dalam rak atau lapisannya masing-masing. Ada lapisan yang khusus mengurus data saja (Data Layer), logika aplikasinya (Domain Layer), dan tampilan antarmukanya (Presentation Layer). Karena setiap tugas dipisah-pisah seperti ini, nantinya kalau ada error, kita bisa langsung mencari letak kerusakannya di folder yang spesifik tanpa harus membongkar seluruh kode dari awal. Terlebih lagi, ini membuat proyek aplikasi kita bisa dikerjakan oleh beberapa orang sekaligus (teamwork) tanpa takut kode saling bertabrakan (bentrok), sehingga aplikasi jadi jauh lebih rapi, terstruktur, dan gampang dites (testable).

MODUL 4 : VIEWMODEL AND DEBUGGING

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out SOURCE CODE

A. SOURCE CODE

Tabel 25. Source Code MainActivity Compose

1.	<code>package com.example.clickanime</code>
2.	
3.	<code>import android.content.Context</code>
4.	<code>import android.os.Bundle</code>
5.	<code>import androidx.activity.ComponentActivity</code>
6.	<code>import androidx.activity.compose.setContent</code>
7.	<code>import androidx.activity.enableEdgeToEdge</code>
8.	<code>import androidx.compose.foundation.layout.fillMaxSize</code>
9.	<code>import androidx.compose.material3.Surface</code>
10.	<code>import androidx.compose.runtime.remember</code>
11.	<code>import androidx.compose.ui.Modifier</code>
12.	<code>import androidx.lifecycle.viewmodel.compose.viewModel</code>

13.	import androidx.navigation.compose.rememberNavController
14.	import com.example.clickanime.navigation.NavGraph
15.	import com.example.clickanime.ui.theme.AnimeAppTheme
16.	import com.example.clickanime.viewmodel.AnimeListViewModel
17.	import com.example.clickanime.viewmodel.AnimeListViewModelFactory
18.	import java.util.Locale
19.	
20.	class MainActivity : ComponentActivity() {
21.	
22.	override fun attachBaseContext(newBase: Context) {
23.	val prefs = newBase.getSharedPreferences("settings", Context.MODE_PRIVATE)
24.	val lang = prefs.getString("language", Locale.getDefault().language) ?: "en"
25.	val locale = Locale(lang)
26.	Locale.setDefault(locale)
27.	val config = newBase.resources.configuration
28.	config.setLocale(locale)
29.	super.attachBaseContext(newBase.createConfigurationContext(config))
30.	}
31.	
32.	override fun onCreate(savedInstanceState: Bundle?) {
33.	super.onCreate(savedInstanceState)
34.	enableEdgeToEdge()
35.	setContent {
36.	AnimeAppTheme {
37.	Surface(modifier = Modifier.fillMaxSize()) {
38.	val navController = rememberNavController()
39.	val factory = remember { AnimeListViewModelFactory("AnimeList")} }

40.	val animeListViewModel: AnimeListViewModel =
	viewModel(factory = factory)
41.	NavGraph(
42.	navController = navController,
43.	viewModel = animeListViewModel,
44.	onLocaleChange = { langCode -> applyLocale(langCode)
	}
45.	)
46.	}
47.	}
48.	}
49.	}
50.	
51.	private fun applyLocale(langCode: String) {
52.	getSharedPreferences("settings", Context.MODE_PRIVATE)
53.	.edit()
54.	.putString("language", langCode)
55.	.apply()
56.	recreate()
57.	}
58.	}

Tabel 26. AnimeApp.kt Compose

1.	package com.example.clickanime
2.	
3.	import android.app.Application
4.	import timber.log.Timber
5.	
6.	class AnimeApp : Application() {
7.	override fun onCreate() {
8.	super.onCreate()
9.	if (BuildConfig.DEBUG) {

10.	Timber.plant(Timber.DebugTree())
11.	}
12.	}
13.	}

Tabel 27. AnimeListViewModel.kt Compose

1.	package com.example.clickanime.viewmodel
2.	
3.	import androidx.lifecycle.ViewModel
4.	import androidx.lifecycle.ViewModelProvider
5.	import com.example.clickanime.data.AnimeData
6.	import com.example.clickanime.model.AnimeCharacter
7.	import kotlinx.coroutines.flow.MutableStateFlow
8.	import kotlinx.coroutines.flow.StateFlow
9.	import kotlinx.coroutines.flow.asStateFlow
10.	import timber.log.Timber
11.	
12.	class AnimeListViewModel(private val logTag: String) : ViewModel() {
13.	
14.	private val _characters = MutableStateFlow<List<AnimeCharacter>>(emptyList())
15.	val characters: StateFlow<List<AnimeCharacter>> = _characters.asStateFlow()
16.	
17.	
18.	private val _selectedCharacterId = MutableStateFlow<Int?>(null)
19.	val selectedCharacterId: StateFlow<Int?> = _selectedCharacterId.asStateFlow()
20.	
21.	
22.	private val _openMalUrl = MutableStateFlow<String?>(null)

```

23.     val openMalUrl: StateFlow<String?> = _openMalUrl.asStateFlow()
24.
25.     init {
26.         val list = AnimeData.characters
27.         _characters.value = list
28.         Timber.tag(logTag).i("Loaded %d characters into list", list.size)
29.     }
30.
31.     fun onCharacterClicked(id: Int) {
32.         _selectedCharacterId.value = id
33.         val character = _characters.value.find { it.id == id }
34.         character?.let {
35.             Timber.tag(logTag).i(
36.                 "Character selected: id=%d nameResId=%d animeResId=%d",
37.                 it.id,
38.                 it.nameResId,
39.                 it.animeResId
40.             )
41.         } ?: Timber.tag(logTag).w("Character clicked but not found:
id=%d", id)
42.     }
43.
44.     fun clearSelectedCharacter() {
45.         _selectedCharacterId.value = null
46.     }
47.
48.     fun onOpenMalClicked(url: String) {
49.         _openMalUrl.value = url
50.         Timber.tag(logTag).i("Open MAL pressed for url=%s", url)
51.     }

```

52.	
53.	fun clearOpenMalEvent() {
54.	_openMalUrl.value = null
55.	}
56.	}
57.	
58.	class AnimeListViewModelFactory(private val logTag: String) : ViewModelProvider.Factory {
59.	override fun <T : ViewModel> create(modelClass: Class<T>): T {
60.	if (modelClass.isAssignableFrom(AnimeListViewModel::class.java)) {
61.	@Suppress("UNCHECKED_CAST")
62.	return AnimeListViewModel(logTag) as T
63.	}
64.	throw IllegalArgumentException("Unknown ViewModel class")
65.	}
66.	}

Tabel 28. NavGraph.kt Compose

1.	package com.example.clickanime.navigation
2.	
3.	import androidx.compose.runtime.Composable
4.	import androidx.navigation.NavHostController
5.	import androidx.navigation.NavType
6.	import androidx.navigation.compose.NavHost
7.	import androidx.navigation.compose.composable
8.	import androidx.navigation.navArgument
9.	import com.example.clickanime.ui.detail.DetailScreen
10.	import com.example.clickanime.ui.home.HomeScreen
11.	import com.example.clickanime.ui.language.LanguageScreen
12.	import com.example.clickanime.ui.favorites.FavoritesScreen

13.	import com.example.clickanime.ui.search.SearchScreen	
14.	import com.example.clickanime.ui.settings.SettingsScreen	
15.	import com.example.clickanime.viewmodel.AnimeListViewModel	
16.		
17.	@Composable	
18.	fun NavGraph(	
19.	navController: NavHostController,	
20.	viewModel: AnimeListViewModel,	
21.	onLocaleChange: (String) -> Unit	
22.	) {	
23.	NavHost(	
24.	navController = navController,	
25.	startDestination = Screen.Home.route	
26.	) {	
27.	composable(Screen.Home.route) {	
28.	HomeScreen(	
29.	viewModel = viewModel,	
30.	onDetailNavigate = { characterId ->	
31.	viewModel.onCharacterClicked(characterId)	
32.	navController.navigate(Screen.Detail.createRoute(characterId))	
33.	},	
34.	onLanguageClick = {	
35.	navController.navigate(Screen.Language.route) },	
36.	onFavoritesClick = {	
37.	navController.navigate(Screen.Favorites.route) },	
38.	onSearchClick = {	
39.	navController.navigate(Screen.Search.route) },	
	onSettingsClick = {	
	navController.navigate(Screen.Settings.route) }	
	)	
	}	

40.	
41.	composable(
42.	route = Screen.Detail.route,
43.	arguments = listOf(navArgument("characterId") { type = NavType.IntType })
44.	) { backStackEntry ->
45.	val characterId = backStackEntry.arguments?.getInt("characterId") ?: return@composable
46.	val character = viewModel.characters.value.find { it.id == characterId }
47.	character?.let {
48.	DetailScreen(
49.	character = it,
50.	onBack = { navController.popBackStack() },
51.	onOpenMal = { url ->
52.	viewModel.onOpenMalClicked(url)
53.	}
54.	)
55.	}
56.	}
57.	
58.	composable(Screen.Language.route) {
59.	LanguageScreen(
60.	onLanguageSelected = { langCode ->
61.	onLocaleChange(langCode)
62.	navController.popBackStack()
63.	},
64.	onBack = { navController.popBackStack() }
65.	)
66.	}
67.	

68.	composable(Screen.Favorites.route) {
69.	FavoritesScreen(onBack = { navController.popBackStack() })
70.	}
71.	
72.	composable(Screen.Search.route) {
73.	SearchScreen(onBack = { navController.popBackStack() }, viewModel = viewModel)
74.	}
75.	
76.	composable(Screen.Settings.route) {
77.	SettingsScreen(onBack = { navController.popBackStack() })
78.	}
79.	
80.	
81.	}
82.	}

Tabel 29. SearchScreen.kt Compose

1.	package com.example.clickanime.ui.search
2.	
3.	import androidx.compose.foundation.layout.*
4.	import androidx.compose.material.icons.Icons
5.	import androidx.compose.material.icons.filled.ArrowBack
6.	import androidx.compose.material3.*
7.	import androidx.compose.runtime.*
8.	import androidx.compose.ui.Modifier
9.	import androidx.compose.ui.unit.dp
10.	import com.example.clickanime.viewmodel.AnimeListViewModel
11.	
12.	@OptIn(ExperimentalMaterial3Api::class)

13.	@Composable	
14.	fun SearchScreen(	
15.	onBack: () -> Unit,	
16.	viewModel: AnimeListViewModel	
17.	) {	
18.	val characters by viewModel.characters.collectAsState()	
19.	var query by remember { mutableStateOf("") }	
20.	val results = remember(query, characters) {	
21.	if (query.isBlank()) characters else characters.filter {	
22.	it.year.toString().contains(query)	
	it.rating.toString().contains(query)	
23.	}	
24.	}	
25.		
26.	Scaffold(	
27.	topBar = {	
28.	TopAppBar(	
29.	title = { Text("Search") },	
30.	navigationIcon = {	
31.	IconButton(onClick = onBack) {	
32.	Icon(imageVector = Icons.Default.ArrowBack,	
	contentDescription = "Back")	
33.	}	
34.	}	
35.	}	
36.	}	
37.	) { paddingValues ->	
38.	Column(modifier =	=
	Modifier.fillMaxSize().padding(paddingValues).padding(16.dp)) {	
39.	OutlinedTextField(value = query, onValueChange = { query = it	
	}, label = { Text("Search by year or rating (demo)") }, modifier =	
	Modifier.fillMaxWidth())	

40.	Spacer(modifier = Modifier.height(12.dp))
41.	Text(text = "Results: \${results.size}", style = MaterialTheme.typography.bodyMedium)
42.	Spacer(modifier = Modifier.height(8.dp))
43.	results.forEach { item ->
44.	Text(text = "\${item.id} • \${item.year} • \${item.rating}", modifier = Modifier.padding(vertical = 6.dp))
45.	Divider()
46.	}
47.	}
48.	}
49.	}

Tabel 30. MainActivity.kt XML

1.	package com.example.animepopular
2.	
3.	import android.content.Context
4.	import android.os.Bundle
5.	import android.view.View
6.	import androidx.appcompat.app.AppCompatActivity
7.	import androidx.navigation.fragment.NavHostFragment
8.	import androidx.navigation.ui.AppBarConfiguration
9.	import androidx.navigation.ui.setupActionBarWithNavController
10.	import androidx.navigation.ui.setupWithNavController
11.	import com.example.animepopular.databinding.ActivityMainBinding
12.	import java.util.Locale
13.	
14.	class MainActivity : AppCompatActivity() {
15.	
16.	private lateinit var binding: ActivityMainBinding
17.	

18.	override fun attachBaseContext(newBase: Context) {	
19.	val prefs = newBase.getSharedPreferences("settings", Context.MODE_PRIVATE)	
20.	val lang = prefs.getString("language", "en") ?: "en"	
21.	val locale = Locale(lang)	
22.	Locale.setDefault(locale)	
23.	val config = newBase.resources.configuration	
24.	config.setLocale(locale)	
25.	val context = newBase.createConfigurationContext(config)	
26.	super.attachBaseContext(context)	
27.	}	
28.		
29.	override fun onCreate(savedInstanceState: Bundle?) {	
30.	super.onCreate(savedInstanceState)	
31.	binding = ActivityMainBinding.inflate(layoutInflater)	
32.	setContentView(binding.root)	
33.		
34.	val navHostFragment = supportFragmentManager	
35.	.findFragmentById(R.id.nav_host_fragment)	as
	NavHostFragment	
36.	val navController = navHostFragment.navController	
37.		
38.	setSupportActionBar(binding.toolbar)	
39.		
40.	val topLevelDestinations = setOf(	
41.	R.id.homeFragment,	
42.	R.id.searchFragment,	
43.	R.id.favoritesFragment,	
44.	R.id.topRatedFragment,	
45.	R.id.profileFragment	
46.	)	

47.	
48.	val appBarConfig = AppBarConfiguration(topLevelDestinations)
49.	setupActionBarWithNavController(navController, appBarConfig)
50.	
51.	binding.bottomNavigationView.setupWithNavController(navController)
52.	
53.	navController.addOnDestinationChangeListener { _, destination, _
	->
54.	if (destination.id in topLevelDestinations) {
55.	binding.bottomNavigationView.visibility = View.VISIBLE
56.	} else {
57.	binding.bottomNavigationView.visibility = View.GONE
58.	}
59.	supportActionBar?.title = destination.label
60.	}
61.	}
62.	
63.	override fun onSupportNavigateUp(): Boolean {
64.	val navHostFragment = supportFragmentManager
65.	.findFragmentById(R.id.nav_host_fragment) as NavHostFragment
66.	return navHostFragment.navController.navigateUp() super.onSupportNavigateUp()
67.	}
68.	}

Tabel 31. MyApplication.kt XML

1.	package com.example.animepopular
2.	
3.	import android.app.Application
4.	import timber.log.Timber

5.	
6.	class MyApplication : Application() {
7.	override fun onCreate() {
8.	super.onCreate()
9.	
10.	if (BuildConfig.DEBUG) {
11.	Timber.plant(Timber.DebugTree())
12.	Timber.d("Application: Timber logging initialized")
13.	}
14.	}
15.	}

Tabel 32, AnimeViewModel.kt XML

1.	package com.example.animepopular.viewmodel
2.	
3.	import androidx.lifecycle.ViewModel
4.	import com.example.animepopular.data.AnimeRepository
5.	import com.example.animepopular.model.AnimeItem
6.	import kotlinx.coroutines.flow.MutableStateFlow
7.	import kotlinx.coroutines.flow.StateFlow
8.	import timber.log.Timber
9.	
10.	class AnimeViewModel(private val filterParam: String = "all") : ViewModel() {
11.	
12.	private val _animeList = MutableStateFlow<List<AnimeItem>>>(emptyList())
13.	val animeList: StateFlow<List<AnimeItem>> = _animeList
14.	
15.	private val _selectedAnime = MutableStateFlow<AnimeItem?>(null)
16.	val selectedAnime: StateFlow<AnimeItem?> = _selectedAnime

17.	
18.	private val _detailButtonClicked = MutableStateFlow(false)
19.	val detailButtonClicked: StateFlow<Boolean> = _detailButtonClicked
20.	
21.	private val _explicitIntentClicked = MutableStateFlow(false)
22.	val explicitIntentClicked: StateFlow<Boolean> = _explicitIntentClicked
23.	
24.	init {
25.	loadAnimeList()
26.	}
27.	
28.	private fun loadAnimeList() {
29.	val list = AnimeRepository.getAnimeList()
30.	_animeList.value = list
31.	Timber.d("Anime list loaded with \${list.size} items")
32.	}
33.	
34.	fun onDetailButtonClicked(anime: AnimeItem) {
35.	Timber.d("Detail button clicked for anime: \${anime.titleEn} (ID: \${anime.id})")
36.	_selectedAnime.value = anime
37.	_detailButtonClicked.value = true
38.	Timber.d("Selected anime data - Title: \${anime.titleEn}, Year: \${anime.year}, Rating: \${anime.rating}")
39.	}
40.	
41.	fun onExplicitIntentClicked(anime: AnimeItem) {
42.	Timber.d("Explicit Intent button clicked for anime: \${anime.titleEn} (ID: \${anime.id})")
43.	_selectedAnime.value = anime

44.	<code>_explicitIntentClicked.value = true</code>
45.	<code>}</code>
46.	
47.	<code>fun resetDetailButtonState() {</code>
48.	<code> _detailButtonClicked.value = false</code>
49.	<code>}</code>
50.	
51.	<code>fun resetExplicitIntentState() {</code>
52.	<code> _explicitIntentClicked.value = false</code>
53.	<code>}</code>
54.	
55.	<code>fun addAnimeToList(anime: AnimeItem) {</code>
56.	<code> Timber.d("Adding anime to list: \${anime.titleEn} (ID: \${anime.id})")</code>
57.	<code> val currentList = _animeList.value.toMutableList()</code>
58.	<code> if (!currentList.any { it.id == anime.id }) {</code>
59.	<code> currentList.add(anime)</code>
60.	<code> _animeList.value = currentList</code>
61.	<code> Timber.d("Anime added successfully. Total items: \${currentList.size}")</code>
62.	<code> } else {</code>
63.	<code> Timber.w("Anime already exists in list: \${anime.titleEn}")</code>
64.	<code> }</code>
65.	<code>}</code>
66.	
67.	<code>fun getFilterParam(): String {</code>
68.	<code> Timber.d("Filter parameter: \$filterParam")</code>
69.	<code> return filterParam</code>
70.	<code>}</code>
71.	<code>}</code>

Tabel 33. AnimeViewModelFactory.kt XML

1.	package com.example.animepopular.viewmodel
2.	
3.	import androidx.lifecycle.ViewModel
4.	import androidx.lifecycle.ViewModelProvider
5.	
6.	class AnimeViewModelFactory(private val filterParam: String = "all") : ViewModelProvider.Factory {
7.	
8.	override fun <T : ViewModel> create(modelClass: Class<T>): T {
9.	if (modelClass.isAssignableFrom(AnimeViewModel::class.java)) {
10.	@Suppress("UNCHECKED_CAST")
11.	return AnimeViewModel(filterParam) as T
12.	}
13.	throw IllegalArgumentException("Unknown ViewModel class")
14.	}
15.	}

Tabel 34. AnimeAdapter.kt XML

1.	package com.example.animepopular.adapter
2.	
3.	import android.content.Context
4.	import android.view.LayoutInflater
5.	import android.view.ViewGroup
6.	import androidx.core.content.ContextCompat
7.	import androidx.recyclerview.widget.DiffUtil
8.	import androidx.recyclerview.widget.ListAdapter
9.	import androidx.recyclerview.widget.RecyclerView
10.	import com.bumptech.glide.Glide
11.	import com.example.animepopular.R

12.	import com.example.animepopular.data.FavoritesRepository
13.	import com.example.animepopular.databinding.ItemAnimeBinding
14.	import com.example.animepopular.model.AnimeItem
15.	
16.	class AnimeAdapter(
17.	private val onImdbClick: (AnimeItem) -> Unit,
18.	private val onDetailClick: (AnimeItem) -> Unit,
19.	private val onFavoriteToggled: ((AnimeItem, isFavorite: Boolean) -> Unit)? = null
20.	) : ListAdapter<AnimeItem, AnimeAdapter.AnimeViewHolder>(DiffCallback()) {
21.	
22.	override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AnimeViewHolder {
23.	val binding = ItemAnimeBinding.inflate(
24.	LayoutInflater.from(parent.context), parent, false
25.	)
26.	return AnimeViewHolder(binding, parent.context)
27.	}
28.	
29.	override fun onBindViewHolder(holder: AnimeViewHolder, position: Int) {
30.	holder.bind(getItem(position))
31.	}
32.	
33.	inner class AnimeViewHolder(
34.	private val binding: ItemAnimeBinding,
35.	private val context: Context
36.	) : RecyclerView.ViewHolder(binding.root) {
37.	
38.	fun bind(anime: AnimeItem) {

39.	val prefs = context.getSharedPreferences("settings", Context.MODE_PRIVATE)
40.	val isId = prefs.getString("language", "en") == "id"
41.	
42.	binding.tvTitle.text = if (isId) anime.titleId else anime.titleEn
43.	binding.tvYear.text = anime.year.toString()
44.	binding.tvGenre.text = if (isId) anime.genreId else anime.genreEn
45.	binding.tvPlot.text = if (isId) anime.plotId else anime.plotEn
46.	binding.tvRating.text = anime.rating
47.	binding.tvEpisodes.text = anime.episodes
48.	
49.	Glide.with(context)
50.	.load(anime.imageUrl)
51.	.placeholder(android.R.drawable.ic_menu_gallery)
52.	.error(android.R.drawable.ic_menu_gallery)
53.	.into(binding.imgPoster)
54.	
55.	binding.btnImdb.setOnClickListener { onImdbClick(anime) }
56.	binding.btnDetail.setOnClickListener { onDetailClick(anime) }
57.	
58.	updateFavoriteIcon(anime)
59.	
60.	binding.btnFavorite.setOnClickListener {
61.	FavoritesRepository.toggleFavorite(anime.id)
62.	updateFavoriteIcon(anime)
63.	onFavoriteToggled?.invoke(anime, FavoritesRepository.isFavorite(anime.id))
64.	}
65.	}
66.	

67.	private fun updateFavoriteIcon(anime: AnimeItem) {
68.	val isFav = FavoritesRepository.isFavorite(anime.id)
69.	if (isFav) {
70.	binding.btnFavorite.setIconResource(R.drawable.ic_favorite)
71.	binding.btnFavorite.iconTint =
72.	ContextCompat.getColorStateList(context,
	R.color.rating_color)
73.	binding.btnFavorite.strokeColor =
74.	ContextCompat.getColorStateList(context,
	R.color.rating_color)
75.	} else {
76.	binding.btnFavorite.setIconResource(R.drawable.ic_favorite_border)
77.	binding.btnFavorite.iconTint =
78.	ContextCompat.getColorStateList(context,
	R.color.text_secondary)
79.	binding.btnFavorite.strokeColor =
80.	ContextCompat.getColorStateList(context,
	R.color.text_secondary)
81.	}
82.	}
83.	}
84.	
85.	class DiffCallback : DiffUtil.ItemCallback<AnimeItem>() {
86.	override fun areItemsTheSame(oldItem: AnimeItem, newItem: AnimeItem) =
87.	oldItem.id == newItem.id
88.	override fun areContentsTheSame(oldItem: AnimeItem, newItem: AnimeItem) =
89.	oldItem == newItem
90.	}
91.	}

Tabel 35. AnimeItem.kt XML

1.	package com.example.animepopular.model
2.	
3.	import android.os.Parcelable
4.	import kotlinx.parcelize.Parcelize
5.	
6.	@Parcelize
7.	data class AnimeItem(
8.	val id: Int,
9.	val titleEn: String,
10.	val titleId: String,
11.	val year: Int,
12.	val genreEn: String,
13.	val genreId: String,
14.	val plotEn: String,
15.	val plotId: String,
16.	val studioEn: String,
17.	val studioId: String,
18.	val episodes: String,
19.	val rating: String,
20.	val imageUrl: String,
21.	val imdbUrl: String
22.	) : Parcelable

Tabel 36. DetailFragment.kt XML

1.	package com.example.animepopular.ui.detail
2.	
3.	import android.content.Context
4.	import android.content.Intent
5.	import android.net.Uri

6.	import android.os.Bundle
7.	import android.view.LayoutInflater
8.	import android.view.View
9.	import android.view.ViewGroup
10.	import android.widget.Toast
11.	import androidx.core.content.ContextCompat
12.	import androidx.fragment.app.Fragment
13.	import androidx.navigation.fragment.navArgs
14.	import androidx.recyclerview.widget.LinearLayoutManager
15.	import com.bumptech.glide.Glide
16.	import com.example.animepopular.R
17.	import com.example.animepopular.adapter.ReviewAdapter
18.	import com.example.animepopular.data.FavoritesRepository
19.	import com.example.animepopular.databinding.FragmentDetailBinding
20.	import com.example.animepopular.model.AnimeItem
21.	import com.example.animepopular.model.Review
22.	import java.text.SimpleDateFormat
23.	import java.util.Date
24.	import java.util.Locale
25.	
26.	class DetailFragment : Fragment() {
27.	
28.	private var _binding: FragmentDetailBinding? = null
29.	private val binding get() = _binding!!
30.	
31.	private val args: DetailFragmentArgs by navArgs()
32.	
33.	private lateinit var reviewAdapter: ReviewAdapter
34.	
35.	override fun onCreateView(

36.	inflater: LayoutInflater, container: ViewGroup?,
37.	savedInstanceState: Bundle?
38.	): View {
39.	_binding = FragmentDetailBinding.inflate(inflater, container, false)
40.	return binding.root
41.	}
42.	
43.	override fun onCreateView(view: View, savedInstanceState: Bundle?) {
44.	super.onCreateView(view, savedInstanceState)
45.	
46.	val anime = args.anime
47.	val isIndonesian = getCurrentLanguage() == "id"
48.	
49.	bindData(anime, isIndonesian)
50.	setupWatchedButton(anime)
51.	setupReviewSection(anime)
52.	}
53.	
54.	private fun bindData(anime: AnimeItem, isIndonesian: Boolean) {
55.	Glide.with(this)
56.	.load(anime.imageUrl)
57.	.placeholder(android.R.drawable.ic_menu_gallery)
58.	.error(android.R.drawable.ic_menu_gallery)
59.	.centerCrop()
60.	.into(binding.imgDetailPoster)
61.	
62.	binding.tvDetailTitle.text = if (isIndonesian) anime.titleId else anime.titleEn
63.	binding.tvDetailYear.text = anime.year.toString()

64.	binding.tvDetailGenre.text = if (isIndonesian) anime.genreId else anime.genreEn
65.	binding.tvDetailStudio.text = if (isIndonesian) anime.studioId else anime.studioEn
66.	binding.tvDetailEpisodes.text = anime.episodes
67.	binding.tvDetailRating.text = anime.rating
68.	binding.tvDetailPlot.text = if (isIndonesian) anime.plotId else anime.plotEn
69.	
70.	binding.btnDetailImdb.setOnClickListener {
71.	startActivity(Intent(Intent.ACTION_VIEW, Uri.parse(anime.imdbUrl)))
72.	}
73.	}
74.	
75.	private fun setupWatchedButton(anime: AnimeItem) {
76.	updateWatchedButtonState(anime.id)
77.	
78.	binding.btnWatched.setOnClickListener {
79.	FavoritesRepository.toggleWatched(anime.id)
80.	updateWatchedButtonState(anime.id)
81.	}
82.	}
83.	
84.	private fun updateWatchedButtonState(animeId: Int) {
85.	val isWatched = FavoritesRepository.isWatched(animeId)
86.	if (isWatched) {
87.	binding.btnWatched.setIconResource(R.drawable.ic_watched)
88.	binding.btnWatched.iconTint =
89.	ContextCompat.getColorStateList(requireContext(), R.color.accent_color)
90.	binding.btnWatched.strokeColor =

91.	ContextCompat.getColorStateList(requireContext(), R.color.accent_color)	
92.	} else {	
93.	binding.btnWatched.setIconResource(R.drawable.ic_watched_border)	
94.	binding.btnWatched.iconTint =	
95.	ContextCompat.getColorStateList(requireContext(), R.color.text_secondary)	
96.	binding.btnWatched.strokeColor =	
97.	ContextCompat.getColorStateList(requireContext(), R.color.text_secondary)	
98.	}	
99.	}	
100.		
101.	private fun setupReviewSection(anime: AnimeItem) {	
102.	reviewAdapter = ReviewAdapter()	
103.	binding.rvReviews.layoutManager LinearLayoutManager(requireContext())	=
104.	binding.rvReviews.adapter = reviewAdapter	
105.	binding.rvReviews.isNestedScrollingEnabled = false	
106.		
107.	refreshReviews(anime.id)	
108.		
109.	binding.btnSendReview.setOnClickListener {	
110.	submitReview(anime.id)	
111.	}	
112.	}	
113.		
114.	private fun submitReview(animeId: Int) {	
115.	val username = binding.etReviewUsername.text.toString().trim()	
116.	val reviewText = binding.etReviewText.text.toString().trim()	
117.		

118.	if (username.isEmpty()) {
119.	binding.etReviewUsername.error = "X"
120.	binding.etReviewUsername.requestFocus()
121.	return
122.	}
123.	if (reviewText.isEmpty()) {
124.	binding.etReviewText.error = "X"
125.	binding.etReviewText.requestFocus()
126.	return
127.	}
128.	
129.	val timestamp = SimpleDateFormat("dd MMM yyyy, HH:mm",
	Locale.getDefault())
130.	.format(Date())
131.	val review = Review(
132.	username = username,
133.	text = reviewText,
134.	timestamp = timestamp
135.	)
136.	
137.	FavoritesRepository.addReview(animeId, review)
138.	refreshReviews(animeId)
139.	
140.	binding.etReviewUsername.text?.clear()
141.	binding.etReviewText.text?.clear()
142.	binding.etReviewUsername.clearFocus()
143.	binding.etReviewText.clearFocus()
144.	}
145.	
146.	private fun refreshReviews(animeId: Int) {

147.	val reviews = FavoritesRepository.getReviews(animeId)
148.	reviewAdapter.submitList(reviews)
149.	
150.	binding.tvReviewCount.text = when (reviews.size) {
151.	1 -> "1 review"
152.	else -> "\${reviews.size} review"
153.	}
154.	}
155.	
156.	private fun getCurrentLanguage(): String {
157.	val prefs = requireContext().getSharedPreferences("settings",
	Context.MODE_PRIVATE)
158.	return prefs.getString("language", "en") ?: "en"
159.	}
160.	
161.	override fun onDestroyView() {
162.	super.onDestroyView()
163.	_binding = null
164.	}
165.	}

Tabel 37. activity_main.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<androidx.coordinatorlayout.widget.CoordinatorLayout
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	android:layout_width="match_parent"
6.	android:layout_height="match_parent"
7.	android:fitsSystemWindows="true">
8.	

9.	<com.google.android.material.appbar.AppBarLayout
10.	android:id="@+id/appBarLayout"
11.	android:layout_width="match_parent"
12.	android:layout_height="wrap_content"
13.	android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"
14.	android:fitsSystemWindows="true">
15.	
16.	<androidx.appcompat.widget.Toolbar
17.	android:id="@+id/toolbar"
18.	android:layout_width="match_parent"
19.	android:layout_height="?attr/actionBarSize"
20.	android:background="?attr/colorPrimary"
21.	app:popupTheme="@style/ThemeOverlay.MaterialComponents.Light"
22.	app:title="@string/app_name"/>
23.	
24.	</com.google.android.material.appbar.AppBarLayout>
25.	
26.	<androidx.fragment.app.FragmentContainerView
27.	android:id="@+id/nav_host_fragment"
28.	android:name="androidx.navigation.fragment.NavHostFragment"
29.	android:layout_width="match_parent"
30.	android:layout_height="match_parent"
31.	app:layout_behavior="@string/appbar_scrolling_view_behavior"
32.	app:defaultNavHost="true"
33.	app:navGraph="@navigation/nav_graph"/>
34.	
35.	<com.google.android.material.bottomnavigation.BottomNavigationView
36.	android:id="@+id/bottomNavView"
37.	android:layout_width="match_parent"

38.	<code>android:layout_height="wrap_content"</code>
39.	<code>android:layout_gravity="bottom"</code>
40.	<code>android:background="@color/colorPrimary"</code>
41.	<code>android:fitsSystemWindows="true"</code>
42.	<code>app:itemIconTint="@color/bottom_nav_selector"</code>
43.	<code>app:itemTextColor="@color/bottom_nav_selector"</code>
44.	<code>app:labelVisibilityMode="labeled"</code>
45.	<code>app:menu="@menu/bottom_nav_menu"/></code>
46.	
47.	<code></androidx.coordinatorlayout.widget.CoordinatorLayout></code>

Tabel 38.fragment_detail.xml

1.	<code><?xml version="1.0" encoding="utf-8"?></code>
2.	<code><androidx.core.widget.NestedScrollView</code>
3.	<code>xmlns:android="http://schemas.android.com/apk/res/android"</code>
4.	<code>xmlns:app="http://schemas.android.com/apk/res-auto"</code>
5.	<code>android:layout_width="match_parent"</code>
6.	<code>android:layout_height="match_parent"</code>
7.	<code>android:background="@color/background_dark"</code>
8.	<code>android:clipToPadding="false"></code>
9.	
10.	<code><LinearLayout</code>
11.	<code>android:layout_width="match_parent"</code>
12.	<code>android:layout_height="wrap_content"</code>
13.	<code>android:orientation="vertical"></code>
14.	
15.	<code><ImageView</code>
16.	<code>android:id="@+id/imgDetailPoster"</code>
17.	<code>android:layout_width="match_parent"</code>

18.	android:layout_height="0dp"
19.	android:layout_weight="1"
20.	android:minHeight="160dp"
21.	android:maxHeight="260dp"
22.	android:scaleType="centerCrop"
23.	android:adjustViewBounds="false"
24.	android:src="@drawable/ic_placeholder"/>
25.	
26.	<View
27.	android:layout_width="match_parent"
28.	android:layout_height="56dp"
29.	android:layout_marginTop="-56dp"
30.	android:background="@drawable/gradient_bottom_fade"/>
31.	
32.	<com.google.android.material.card.MaterialCardView
33.	android:layout_width="match_parent"
34.	android:layout_height="wrap_content"
35.	android:layout_marginHorizontal="12dp"
36.	android:layout_marginTop="-20dp"
37.	android:layout_marginBottom="12dp"
38.	app:cardBackgroundColor="@color/card_background"
39.	app:cardCornerRadius="20dp"
40.	app:cardElevation="6dp">
41.	
42.	<LinearLayout
43.	android:layout_width="match_parent"
44.	android:layout_height="wrap_content"
45.	android:orientation="vertical"
46.	android:padding="16dp">
47.	

48.	<LinearLayout
49.	android:layout_width="match_parent"
50.	android:layout_height="wrap_content"
51.	android:orientation="horizontal"
52.	android:gravity="center_vertical">
53.	
54.	<TextView
55.	android:id="@+id/tvDetailTitle"
56.	android:layout_width="0dp"
57.	android:layout_height="wrap_content"
58.	android:layout_weight="1"
59.	android:textColor="@color/text_primary"
60.	android:textSize="20sp"
61.	android:textStyle="bold"
62.	android:layout_marginEnd="8dp"/>
63.	
64.	<TextView
65.	android:id="@+id/tvDetailYear"
66.	android:layout_width="wrap_content"
67.	android:layout_height="wrap_content"
68.	android:background="@drawable/badge_background"
69.	android:paddingHorizontal="10dp"
70.	android:paddingVertical="4dp"
71.	android:textColor="@color/accent_color"
72.	android:textSize="13sp"
73.	android:textStyle="bold"/>
74.	</LinearLayout>
75.	
76.	<TextView
77.	android:id="@+id/tvDetailGenre"

78.	android:layout_width="wrap_content"
79.	android:layout_height="wrap_content"
80.	android:layout_marginTop="6dp"
81.	android:background="@drawable/badge_background"
82.	android:paddingHorizontal="10dp"
83.	android:paddingVertical="3dp"
84.	android:textColor="@color/accent_color"
85.	android:textSize="12sp"/>
86.	
87.	<View
88.	android:layout_width="match_parent"
89.	android:layout_height="1dp"
90.	android:layout_marginVertical="10dp"
91.	android:background="#3BFFFFFF"/>
92.	
93.	<LinearLayout
94.	android:layout_width="match_parent"
95.	android:layout_height="wrap_content"
96.	android:orientation="horizontal"
97.	android:gravity="center_vertical">
98.	
99.	<TextView
100.	android:layout_width="wrap_content"
101.	android:layout_height="wrap_content"
102.	android:text="@string/label_rating"
103.	android:textColor="@color/text_secondary"
104.	android:textSize="12sp"
105.	android:layout_marginEnd="6dp"/>
106.	
107.	<TextView

108.	android:id="@+id/tvDetailRating"
109.	android:layout_width="0dp"
110.	android:layout_height="wrap_content"
111.	android:layout_weight="1"
112.	android:textColor="@color/rating_color"
113.	android:textSize="15sp"
114.	android:textStyle="bold"/>
115.	
116.	<TextView
117.	android:layout_width="wrap_content"
118.	android:layout_height="wrap_content"
119.	android:text="@string/label_episodes"
120.	android:textColor="@color/text_secondary"
121.	android:textSize="12sp"
122.	android:layout_marginEnd="6dp"/>
123.	
124.	<TextView
125.	android:id="@+id/tvDetailEpisodes"
126.	android:layout_width="wrap_content"
127.	android:layout_height="wrap_content"
128.	android:textColor="@color/text_primary"
129.	android:textSize="12sp"/>
130.	</LinearLayout>
131.	
132.	<LinearLayout
133.	android:layout_width="match_parent"
134.	android:layout_height="wrap_content"
135.	android:orientation="horizontal"
136.	android:gravity="center_vertical"
137.	android:layout_marginTop="6dp">

138.	
139.	<TextView
140.	android:layout_width="wrap_content"
141.	android:layout_height="wrap_content"
142.	android:text="@string/label_studio"
143.	android:textColor="@color/text_secondary"
144.	android:textSize="12sp"
145.	android:layout_marginEnd="6dp"/>
146.	
147.	<TextView
148.	android:id="@+id/tvDetailStudio"
149.	android:layout_width="match_parent"
150.	android:layout_height="wrap_content"
151.	android:textColor="@color/text_primary"
152.	android:textSize="12sp"
153.	android:maxLines="1"
154.	android:ellipsize="end"/>
155.	</LinearLayout>
156.	
157.	<View
158.	android:layout_width="match_parent"
159.	android:layout_height="1dp"
160.	android:layout_marginVertical="10dp"
161.	android:background="#3BFFFFFF"/>
162.	
163.	<TextView
164.	android:layout_width="wrap_content"
165.	android:layout_height="wrap_content"
166.	android:text="@string/label_plot"
167.	android:textColor="@color/text_secondary"

168.	android:textSize="12sp"
169.	android:textStyle="bold"
170.	android:layout_marginBottom="5dp"/>
171.	
172.	<TextView
173.	android:id="@+id/tvDetailPlot"
174.	android:layout_width="match_parent"
175.	android:layout_height="wrap_content"
176.	android:textColor="@color/text_primary"
177.	android:textSize="13sp"
178.	android:lineSpacingMultiplier="1.5"/>
179.	
180.	<LinearLayout
181.	android:layout_width="match_parent"
182.	android:layout_height="wrap_content"
183.	android:orientation="horizontal"
184.	android:layout_marginTop="16dp"
185.	android:gravity="center_vertical">
186.	
187.	<com.google.android.material.button.MaterialButton
188.	android:id="@+id/btnDetailImdb"
189.	android:layout_width="0dp"
190.	android:layout_height="46dp"
191.	android:layout_weight="1"
192.	android:layout_marginEnd="8dp"
193.	android:text="@string/btn_open_imdb"
194.	android:textSize="12sp"
195.	android:paddingHorizontal="8dp"
196.	app:cornerRadius="23dp"
197.	app:backgroundTint="@color/imdb_yellow"

198.	android:textColor="@android:color/black"
199.	app:icon="@drawable/ic_open_external"
200.	app:iconGravity="textStart"
201.	app:iconTint="@android:color/black"/>
202.	
203.	<com.google.android.material.button.MaterialButton
204.	android:id="@+id/btnWatched"
205.	android:layout_width="46dp"
206.	android:layout_height="46dp"
207.	android:padding="0dp"
208.	android:minWidth="0dp"
209.	style="@style/Widget.MaterialComponents.Button.OutlinedButton"
210.	app:cornerRadius="23dp"
211.	app:strokeColor="@color/text_secondary"
212.	app:icon="@drawable/ic_watched_border"
213.	app:iconSize="20dp"
214.	app:iconTint="@color/text_secondary"
215.	app:iconGravity="textStart"
216.	app:iconPadding="0dp"/>
217.	
218.	</LinearLayout>
219.	</LinearLayout>
220.	</com.google.android.material.card.MaterialCardView>
221.	
222.	<com.google.android.material.card.MaterialCardView
223.	android:layout_width="match_parent"
224.	android:layout_height="wrap_content"
225.	android:layout_marginHorizontal="12dp"
226.	android:layout_marginBottom="20dp"

227.	app:cardBackgroundColor="@color/card_background"
228.	app:cardCornerRadius="20dp"
229.	app:cardElevation="6dp">
230.	
231.	<LinearLayout
232.	android:layout_width="match_parent"
233.	android:layout_height="wrap_content"
234.	android:orientation="vertical"
235.	android:padding="16dp">
236.	
237.	<TextView
238.	android:layout_width="wrap_content"
239.	android:layout_height="wrap_content"
240.	android:text="@string/reviewuser"
241.	android:textColor="@color/text_primary"
242.	android:textSize="15sp"
243.	android:textStyle="bold"
244.	android:layout_marginBottom="8dp"/>
245.	
246.	<TextView
247.	android:id="@+id/tvReviewCount"
248.	android:layout_width="wrap_content"
249.	android:layout_height="wrap_content"
250.	android:text="@string/noreview"
251.	android:textColor="@color/text_secondary"
252.	android:textSize="12sp"
253.	android:layout_marginBottom="8dp"/>
254.	
255.	<androidx.recyclerview.widget.RecyclerView
256.	android:id="@+id/rvReviews"

257.	android:layout_width="match_parent"
258.	android:layout_height="wrap_content"
259.	android:nestedScrollingEnabled="false"
260.	android:layout_marginBottom="12dp"/>
261.	
262.	<View
263.	android:layout_width="match_parent"
264.	android:layout_height="1dp"
265.	android:background="#3BFFFFFF"
266.	android:layout_marginBottom="12dp"/>
267.	
268.	<TextView
269.	android:layout_width="wrap_content"
270.	android:layout_height="wrap_content"
271.	android:text="@string/typereview"
272.	android:textColor="@color/text_primary"
273.	android:textSize="13sp"
274.	android:textStyle="bold"
275.	android:layout_marginBottom="10dp"/>
276.	
277.	<com.google.android.material.textfield.TextInputLayout
278.	android:layout_width="match_parent"
279.	android:layout_height="wrap_content"
280.	android:hint="User"
281.	style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
282.	android:layout_marginBottom="10dp"
283.	app:boxStrokeColor="@color/accent_color"
284.	app:hintTextColor="@color/text_secondary">
285.	

286.	<com.google.android.material.textfield.TextInputEditText
287.	android:id="@+id/etReviewUsername"
288.	android:layout_width="match_parent"
289.	android:layout_height="wrap_content"
290.	android:textColor="@color/text_primary"
291.	android:inputType="textPersonName"
292.	android:imeOptions="actionNext"/>
293.	</com.google.android.material.textfield.TextInputLayout>
294.	
295.	<!-- Review Text -->
296.	<com.google.android.material.textfield.TextInputLayout
297.	android:layout_width="match_parent"
298.	android:layout_height="wrap_content"
299.	android:hint="Review"
300.	style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
301.	android:layout_marginBottom="12dp"
302.	app:boxStrokeColor="@color/accent_color"
303.	app:hintTextColor="@color/text_secondary"
304.	app:counterEnabled="true"
305.	app:counterMaxLength="300">
306.	
307.	<com.google.android.material.textfield.TextInputEditText
308.	android:id="@+id/etReviewText"
309.	android:layout_width="match_parent"
310.	android:layout_height="wrap_content"
311.	android:textColor="@color/text_primary"
312.	android:inputType="textMultiLine"
313.	android:minLines="3"
314.	android:maxLines="6"

315.	android:gravity="top start"
316.	android:maxLength="300"/>
317.	</com.google.android.material.textfield.TextInputLayout>
318.	
319.	<!-- Send -->
320.	<com.google.android.material.button.MaterialButton
321.	android:id="@+id/btnSendReview"
322.	android:layout_width="match_parent"
323.	android:layout_height="46dp"
324.	android:text="@string/sendreview"
325.	android:textSize="13sp"
326.	app:cornerRadius="23dp"
327.	app:backgroundTint="@color/accent_color"/>
328.	
329.	</LinearLayout>
330.	</com.google.android.material.card.MaterialCardView>
331.	
332.	</LinearLayout>
333.	
334.	</androidx.core.widget.NestedScrollView>

Tabel 39. item_anime.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<com.google.android.material.card.MaterialCardView
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	android:layout_width="match_parent"
6.	android:layout_height="wrap_content"
7.	android:layout_marginHorizontal="8dp"

8.	<code>android:layout_marginBottom="12dp"</code>
9.	<code>app:cardBackgroundColor="@color/card_background"</code>
10.	<code>app:cardCornerRadius="16dp"</code>
11.	<code>app:cardElevation="4dp"></code>
12.	
13.	<code><LinearLayout</code>
14.	<code> android:layout_width="match_parent"</code>
15.	<code> android:layout_height="wrap_content"</code>
16.	<code> android:orientation="horizontal"</code>
17.	<code> android:padding="12dp"></code>
18.	
19.	<code> <com.google.android.material.imageview.ShapeableImageView</code>
20.	<code> android:id="@+id/imgPoster"</code>
21.	<code> android:layout_width="88dp"</code>
22.	<code> android:layout_height="124dp"</code>
23.	<code> android:scaleType="centerCrop"</code>
24.	<code> android:adjustViewBounds="true"</code>
25.	<code>app:shapeAppearanceOverlay="@style/RoundedCornerImage"/></code>
26.	
27.	
28.	<code><LinearLayout</code>
29.	<code> android:layout_width="0dp"</code>
30.	<code> android:layout_height="wrap_content"</code>
31.	<code> android:layout_weight="1"</code>
32.	<code> android:orientation="vertical"</code>
33.	<code> android:paddingStart="10dp"></code>
34.	
35.	<code><TextView</code>
36.	<code> android:id="@+id/tvTitle"</code>

37.	<code>android:layout_width="match_parent"</code>
38.	<code>android:layout_height="wrap_content"</code>
39.	<code>android:textColor="@color/text_primary"</code>
40.	<code>android:textSize="15sp"</code>
41.	<code>android:textStyle="bold"</code>
42.	<code>android:maxLines="2"</code>
43.	<code>android:ellipsize="end"/></code>
44.	
45.	<code><LinearLayout</code>
46.	<code> android:layout_width="match_parent"</code>
47.	<code> android:layout_height="wrap_content"</code>
48.	<code> android:orientation="horizontal"</code>
49.	<code> android:gravity="center_vertical"</code>
50.	<code> android:layout_marginTop="3dp"></code>
51.	
52.	<code><TextView</code>
53.	<code> android:id="@+id/tvYear"</code>
54.	<code> android:layout_width="wrap_content"</code>
55.	<code> android:layout_height="wrap_content"</code>
56.	<code> android:textColor="@color/text_secondary"</code>
57.	<code> android:textSize="11sp"</code>
58.	<code> android:layout_marginEnd="6dp"/></code>
59.	
60.	<code><TextView</code>
61.	<code> android:id="@+id/tvGenre"</code>
62.	<code> android:layout_width="wrap_content"</code>
63.	<code> android:layout_height="wrap_content"</code>
64.	<code> android:layout_weight="1"</code>
65.	<code> android:background="@drawable/badge_background"</code>
66.	<code> android:paddingHorizontal="6dp"</code>

67.	android:paddingVertical="2dp"
68.	android:textColor="@color/accent_color"
69.	android:textSize="10sp"
70.	android:maxLines="1" />
71.	</LinearLayout>
72.	
73.	<LinearLayout
74.	android:layout_width="match_parent"
75.	android:layout_height="wrap_content"
76.	android:orientation="horizontal"
77.	android:layout_marginTop="3dp">
78.	
79.	<TextView
80.	android:id="@+id/tvRating"
81.	android:layout_width="0dp"
82.	android:layout_height="wrap_content"
83.	android:layout_weight="1"
84.	android:textColor="@color/rating_color"
85.	android:textSize="11sp"
86.	android:textStyle="bold"/>
87.	
88.	<TextView
89.	android:id="@+id/tvEpisodes"
90.	android:layout_width="wrap_content"
91.	android:layout_height="wrap_content"
92.	android:textColor="@color/text_secondary"
93.	android:textSize="10sp"/>
94.	</LinearLayout>
95.	
96.	<TextView

97.	android:id="@+id/tvPlot"
98.	android:layout_width="match_parent"
99.	android:layout_height="wrap_content"
100.	android:layout_marginTop="3dp"
101.	android:maxLines="2"
102.	android:ellipsize="end"
103.	android:textColor="@color/text_secondary"
104.	android:textSize="11sp"
105.	android:lineSpacingMultiplier="1.3"/>
106.	
107.	<LinearLayout
108.	android:layout_width="match_parent"
109.	android:layout_height="wrap_content"
110.	android:orientation="horizontal"
111.	android:layout_marginTop="8dp"
112.	android:gravity="start">
113.	
114.	<com.google.android.material.button.MaterialButton
115.	android:id="@+id/btnImdb"
116.	android:layout_width="wrap_content"
117.	android:layout_height="36dp"
118.	android:text="@string/btn_imdb"
119.	android:textSize="10sp"
120.	android:minWidth="0dp"
121.	android:paddingHorizontal="8dp"
122.	app:cornerRadius="16dp"
123.	style="@style/Widget.MaterialComponents.Button.OutlinedButton"
124.	app:strokeColor="@color/accent_color"
125.	android:textColor="@color/accent_color"

126.	android:layout_marginEnd="6dp"/>
127.	
128.	<com.google.android.material.button.MaterialButton
129.	android:id="@+id/btnDetail"
130.	android:layout_width="wrap_content"
131.	android:layout_height="36dp"
132.	android:text="@string/btn_detail"
133.	android:textSize="10sp"
134.	android:minWidth="0dp"
135.	android:paddingHorizontal="8dp"
136.	app:cornerRadius="16dp"
137.	style="@style/Widget.MaterialComponents.Button"
138.	app:backgroundTint="@color/accent_color"
139.	android:layout_marginEnd="6dp"/>
140.	
141.	<com.google.android.material.button.MaterialButton
142.	android:id="@+id/btnFavorite"
143.	style="@style/Widget.MaterialComponents.Button.OutlinedButton"
144.	android:layout_width="36dp"
145.	android:layout_height="36dp"
146.	android:layout_marginEnd="6dp"
147.	android:layout_marginBottom="2dp"
148.	android:layout_marginTop="3dp"
149.	android:minWidth="0dp"
150.	app:cornerRadius="16dp"
151.	app:icon="@drawable/ic_favorite_border"
152.	app:iconGravity="textStart"
153.	app:iconPadding="0dp"
154.	app:iconSize="18dp"

155.	app:iconTint="@color/rating_color"
156.	app:strokeColor="@color/rating_color" />
157.	
158.	</LinearLayout>
159.	</LinearLayout>
160.	</LinearLayout>
161.	
162.	</com.google.android.material.card.MaterialCardView>

Tabel 40.nav_graph.xml

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<navigation
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	xmlns:app="http://schemas.android.com/apk/res-auto"
5.	xmlns:tools="http://schemas.android.com/tools"
6.	android:id="@+id/nav_graph"
7.	app:startDestination="@id/homeFragment">
8.	
9.	<fragment
10.	android:id="@+id/homeFragment"
11.	android:name="com.example.animepopular.ui.home.HomeFragment"
12.	android:label="@string/app_name"
13.	tools:layout="@layout/fragment_home">
14.	<action android:id="@+id/action_homeFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
15.	<action android:id="@+id/action_homeFragment_to_languageFragment" app:destination="@id/languageFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left"

	<pre> app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/> </pre>
16.	<pre> <action android:id="@+id/action_homeFragment_to_genreFragment" app:destination="@id/genreFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/> </pre>
17.	<pre> <action android:id="@+id/action_homeFragment_to_newsFragment" app:destination="@id/newsFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/> </pre>
18.	<pre> <action android:id="@+id/action_homeFragment_to_scheduleFragment" app:destination="@id/scheduleFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/> </pre>
19.	<pre> </fragment> </pre>
20.	
21.	<pre> <fragment </pre>
22.	<pre> android:id="@+id/detailFragment" </pre>
23.	<pre> android:name="com.example.animepopular.ui.detail.DetailFragment" </pre>
24.	<pre> android:label="@string/label_detail" </pre>
25.	<pre> tools:layout="@layout/fragment_detail"> </pre>
26.	<pre> <argument android:name="anime" app:argType="com.example.animepopular.model.AnimeItem"/> </pre>
27.	<pre> </fragment> </pre>
28.	
29.	<pre> <fragment </pre>
30.	<pre> android:id="@+id/languageFragment" </pre>
31.	<pre> android:name="com.example.animepopular.ui.language.LanguageFragment" </pre>
32.	<pre> android:label="@string/label_language" </pre>

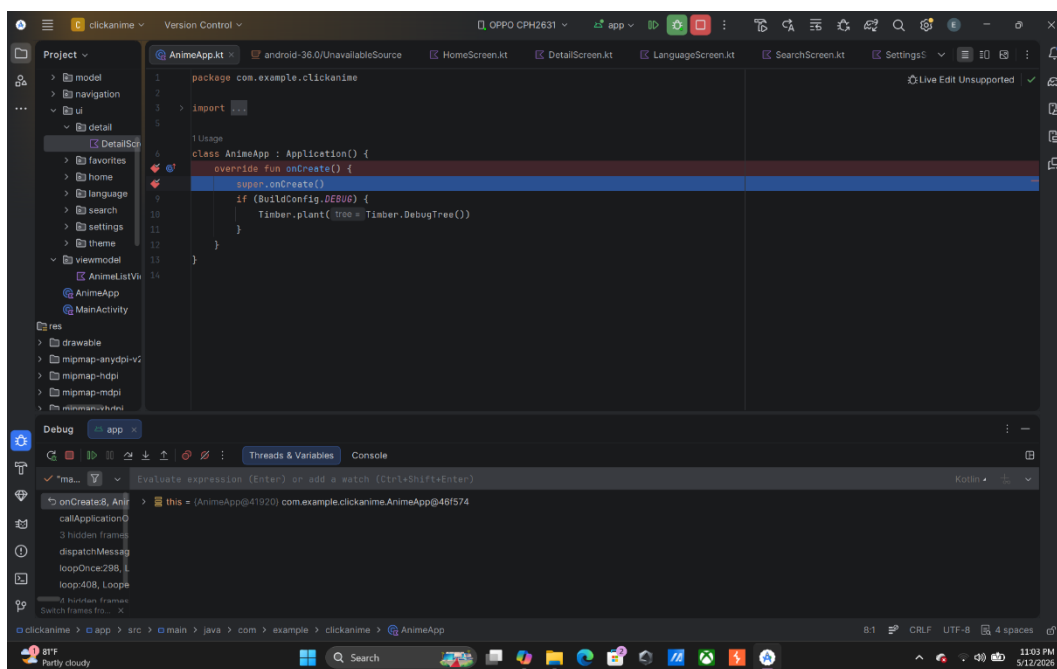
33.	tools:layout="@layout/fragment_language"/>
34.	
35.	<fragment
36.	android:id="@+id/searchFragment"
37.	android:name="com.example.animepopular.ui.feature.SearchFragment"
38.	android:label="@string/searchanime"
39.	tools:layout="@layout/fragment_search">
40.	<action android:id="@+id/action_searchFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
41.	</fragment>
42.	
43.	<fragment
44.	android:id="@+id/favoritesFragment"
45.	android:name="com.example.animepopular.ui.feature.FavoritesFragment"
46.	android:label="@string/favorite"
47.	tools:layout="@layout/fragment_favorites">
48.	<action android:id="@+id/action_favoritesFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
49.	</fragment>
50.	
51.	<fragment
52.	android:id="@+id/topRatedFragment"
53.	android:name="com.example.animepopular.ui.feature.TopRatedFragment"

54.	android:label="@string/toprated"
55.	tools:layout="@layout/fragment_top_rated">
56.	<action android:id="@+id/action_topRatedFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
57.	</fragment>
58.	
59.	<fragment
60.	android:id="@+id/genreFragment"
61.	android:name="com.example.animepopular.ui.feature.GenreFragment"
62.	android:label="@string/Genre"
63.	tools:layout="@layout/fragment_genre">
64.	<argument android:name="genre" app:argType="string" android:defaultValue="Action"/>
65.	<action android:id="@+id/action_genreFragment_to_detailFragment" app:destination="@id/detailFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
66.	</fragment>
67.	
68.	<fragment
69.	android:id="@+id/scheduleFragment"
70.	android:name="com.example.animepopular.ui.feature.ScheduleFragment"
71.	android:label="@string/airing"
72.	tools:layout="@layout/fragment_schedule"/>
73.	
74.	<fragment

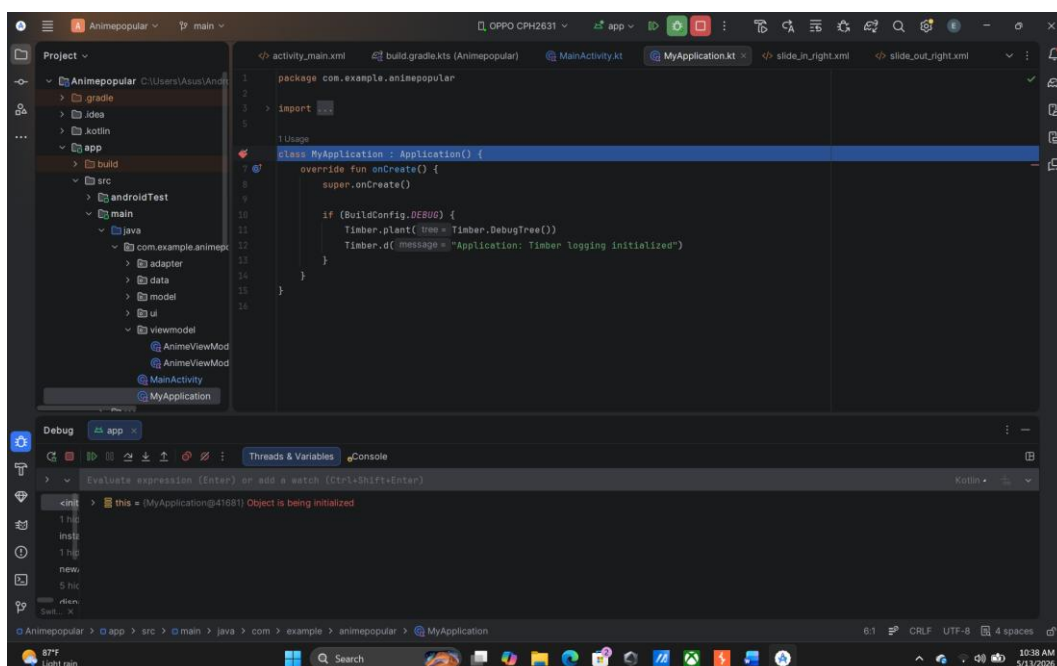
75.	android:id="@+id/newsFragment"
76.	android:name="com.example.animepopular.ui.feature.NewsFragment"
77.	android:label="@string/News"
78.	tools:layout="@layout/fragment_news"/>
79.	
80.	<fragment
81.	android:id="@+id/profileFragment"
82.	android:name="com.example.animepopular.ui.about.ProfileFragment"
83.	android:label="@string/profile"
84.	tools:layout="@layout/fragment_profile">
85.	<action android:id="@+id/action_profileFragment_to_settingsFragment" app:destination="@id/settingsFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
86.	<action android:id="@+id/action_profileFragment_to_aboutFragment" app:destination="@id/aboutFragment" app:enterAnim="@anim/slide_in_right" app:exitAnim="@anim/slide_out_left" app:popEnterAnim="@anim/slide_in_left" app:popExitAnim="@anim/slide_out_right"/>
87.	</fragment>
88.	
89.	<fragment
90.	android:id="@+id/settingsFragment"
91.	android:name="com.example.animepopular.ui.about.SettingsFragment"
92.	android:label="@string/settings"
93.	tools:layout="@layout/fragment_settings"/>
94.	
95.	<fragment
96.	android:id="@+id/aboutFragment"
97.	android:name="com.example.animepopular.ui.about.AboutFragment"

98.	<code>android:label="@string/about"</code>
99.	<code>tools:layout="@layout/fragment_about"/></code>
100.	
101.	<code></navigation></code>

B. OUTPUT CODE



Gambar 23. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 24. Screenshot Hasil Jawaban Soal 1 XML

C. PEMBAHASAN

MainActivity.kt Compose

MainActivity menginisialisasi state pengaturan bahasa (Locale) melalui fungsi `attachBaseContext`, dan membangun UI aplikasi melalui fungsi `setContent` yang dibungkus oleh tema `AnimeAppTheme`. Aplikasi membuat dan menginisialisasi `AnimeListViewModel` menggunakan `AnimeListViewModelFactory`, lalu meneruskan kontrol navigasi ke `NavGraph`. Aliran ini bersifat satu arah, di mana fungsi pada root meneruskan state navigasi agar aplikasi dapat berpindah antar layar secara reaktif.

AnimeApp.kt Compose

AnimeApp menginisialisasi konfigurasi awal di tingkat global (kelas turunan `Application`) saat aplikasi dijalankan pada metode `onCreate`. Modul ini akan memantau *environment* sistem dan mengaktifkan instance library Timber (`Timber.DebugTree()`) khusus jika aplikasi dijalankan pada mode debug (`BuildConfig.DEBUG`) guna memastikan pengelolaan log tidak tumpang tindih pada versi rilis.

AnimeListViewModel.kt Compose

`AnimeListViewModel` mengelola state dari daftar karakter anime melalui penggunaan `StateFlow` (seperti `_characters`, `_selectedCharacterId`, dan `_openMalUrl`). Aliran data bersifat satu arah (*unidirectional*); seluruh interaksi pengguna (seperti `onCharacterClicked` atau `onOpenMalClicked`) diproses di dalam `ViewModel` dan dicatat melalui fungsi logging Timber, dan perubahan state ini kemudian dapat direspons oleh layar UI secara otomatis. Factory pada file yang sama menjamin `ViewModel` terinisiasi dengan benar sesuai siklus *lifecycle* dengan menerima parameter *string logTag*.

NavGraph.kt Compose

`NavGraph` mengelola graf navigasi aplikasi secara terpusat menggunakan antarmuka `NavHost`. Komponen ini merender layar (*Screen*) berdasarkan state *route*, misalnya `HomeScreen` untuk list menu, atau `DetailScreen` yang otomatis menarik data karakter yang relevan dari *ViewModel* berdasarkan `characterId` sebelum di-render. Seluruh aksi klik perpindahan layar ditangani lewat parameter fungsi (*lambda*).

SearchScreen.kt Compose

`SearchScreen` membangun UI untuk layar pencarian memanfaatkan tata letak Scaffold dan fungsi input `OutlinedTextField`. Layar ini memantau state data dari `ViewModel` menggunakan `collectAsState()`. Jika state teks pada kolom pencarian (*query*) diperbarui, sistem akan langsung menyeleksi dan menyaring iterasi daftar anime berdasarkan tahun atau rating secara *real-time*, sehingga UI otomatis me-refresh data tanpa pengolahan ganda di sisi *View*.

MainActivity.kt XML

MainActivity versi XML menghubungkan data dan logika UI menggunakan objek ActivityMainBinding pada fungsi onCreate, sekaligus mengatur sinkronisasi bahasa (Locale) dengan sistem. Alur interaksi tampilan navigasi (AppBar dan BottomNav) dikelola secara otomatis menggunakan NavHostFragment. UI memantau state *destination* navigasi; di mana sistem akan menyembunyikan BottomNavigationView bila pengguna keluar dari menu utama (seperti menuju halaman detail) secara otomatis mengikuti pergantian rute.

MyApplication.kt XML

MyApplication mewarisi Application sebagai entri utama sebelum aktivitas apa pun dijalankan. Ini adalah titik inisialisasi untuk komponen global, yakni memicu pohon utilitas dari Timber apabila BuildConfig.DEBUG bernilai *true*. Ini menciptakan sistem logging (*logging tree*) transparan untuk *debugging*.

AnimeAdapter.kt XML

AnimeAdapter membangun list dinamis menggunakan ListAdapter yang bergantung pada utilitas asinkron DiffUtil agar pergantian iterasi memori jauh lebih responsif dibanding standar RecyclerView. UI diikat langsung (*binding*) pada ItemAnimeBinding, kemudian event pada berbagai tombol (misal klik IMDb atau penambahan Favorites) dieksekusi dengan mendelegasikannya kembali (*callback invoke*) ke antarmuka lambda tanpa harus mengeksekusi logika data secara langsung di dalam Adapter.

AnimeItem.kt XML

AnimeItem mengonfigurasi struktur entitas data (*Data Class*) dan diintegrasikan dengan tag @Parcelable. Anotasi ini memungkinkan seluruh bundle informasi objek seperti id, judul, episode, gambar, hingga URL dikemas/di-*serialize* untuk dilempar dengan aman secara efisien ke Fragment tujuan tanpa melalui proses *parsing* yang berat.

DetailFragment.kt XML

DetailFragment mengatur representasi visual detail konten anime via FragmentDetailBinding serta mencomot serpihan argumen (navArgs()) dari kelas Navigasi pengirim. UI di Fragment beroperasi tanpa kendali *state data source*, melainkan merespons fungsi-fungsi sederhana seperti perenderan visual Glide, logika penambahan *Reviews* pengguna baru, sampai pergantian ikon/warna penanda *Watched* yang dikoordinasikan lewat sinkronisasi dengan *FavoritesRepository*.

AnimeViewModel.kt XML

AnimeViewModel menyimpan state utama untuk halaman berbasis XML, seperti data list yang di-*load* (*_animeList*) dan event penekanan detail (*_selectedAnime*, *_detailButtonClicked*) menggunakan perantara StateFlow. Fragment tidak menyimpan atau memperbarui list sendiri, melainkan memanggil metode (misalnya onDetailButtonClicked()) agar diolah oleh ViewModel dan menghasilkan output *Timber logs*. Ini mewujudkan arsitektur yang terisolasi dan *unidirectional*.

AnimeViewModelFactory.kt XML

AnimeViewModelFactory mengimplementasikan fitur cetak dasar melalui kelas abstrak ViewModelProvider.Factory. Pola ini digunakan untuk membuat spesifikasi instance khusus dari AnimeViewModel yang meminta parameter tipe String (filterParam), karena ViewModel secara bawaan di dalam kerangka Android tidak dapat dikonstruksi secara langsung menggunakan parameter.

XML resource code

Rangkaian file activity_main.xml, fragment_detail.xml, dan item_anime.xml membangun kanvas interaksi statis secara terukur. Fragment detail mendefinisikan layout dengan NestedScrollView agar elemen padat terbaca, sementara item_anime menyematkan visual card per item untuk diisi Adapter. Pengaturan rute antarkomponen diatur transparan melalui skema nav_graph.xml untuk memudahkan konfigurasi argumen perpindahan.

Debugger

Debugger adalah instrumen bawaan di IDE (seperti Android Studio) yang berfungsi untuk menghentikan, menganalisis, dan memeriksa alur berjalannya kode program secara seketika (*real-time*). Fitur ini membantu developer melacak state memori, menangkap proses yang mengarah pada *crash/bug*, serta mengobservasi *value* dari properti atau variabel tanpa perlu menebak atau mencetak banyak fungsi *Log/Print*.

Cara Menggunakan Debugger

1. Tempatkan *Breakpoint* pada salah satu baris kode di margin sebelah kiri editor kode. Saat di-klik, muncul bulatan merah.
2. Jalankan aplikasi tidak dengan ikon *Run* biasa (segitiga hijau), melainkan menggunakan mode **Debug** (ikon berbentuk serangga).
3. Saat operasi di aplikasi menyentuh baris kode yang diberi tanda *breakpoint*, status eksekusi aplikasi Anda akan jeda sementara, dan panel tab menu "*Debug*" akan terbuka di bawah. Anda bisa melihat status memori dan nilai variabel pada jendela "*Variables*" di waktu tersebut.

Step Into, Step Over, Step Out

Step Into digunakan ketika eksekusi *breakpoint* berada di sebuah baris yang memanggil metode atau fungsi. Perintah ini akan masuk secara mendalam ke dalam kode logika fungsi yang dipanggil tersebut untuk memeriksa isinya baris per baris. **Step Over** digunakan untuk mengarahkan eksekusi melangkah ke baris berikutnya di file atau konteks saat ini tanpa perlu masuk ke dalam implementasi detail fungsi di baris tersebut. **Step Out** digunakan bila masuk ke dalam fungsi yang sangat panjang (akibat *Step Into*) dan ingin langsung melangkah keluar menuju titik asal, membiarkan sisa baris kode di dalam fungsi itu diselesaikan seketika di latar belakang.

MODUL 5 : CONNECT TO THE INTERNET

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- f. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. SOURCE CODE

Tabel 41. Source Code MainActivity.kt

1.	<code>package com.example.clickanime</code>
2.	
3.	<code>import android.content.Context</code>
4.	<code>import android.os.Bundle</code>
5.	<code>import androidx.activity.ComponentActivity</code>
6.	<code>import androidx.activity.compose.setContent</code>
7.	<code>import androidx.activity.enableEdgeToEdge</code>
8.	<code>import androidx.compose.foundation.layout.fillMaxSize</code>
9.	<code>import androidx.compose.material3.Surface</code>
10.	<code>import androidx.compose.runtime.remember</code>
11.	<code>import androidx.compose.ui.Modifier</code>

```

12. import
    androidx.lifecycle.viewmodel.compose.viewModel
13. import
    androidx.navigation.compose.rememberNavController
14. import com.example.clickanime.navigation.NavGraph
15. import com.example.clickanime.ui.theme.MovieAppTheme
16. import
    com.example.clickanime.viewmodel.MovieViewModel
17. import java.util.Locale
18.
19. class MainActivity : ComponentActivity() {
20.
21.     override fun attachBaseContext(newBase: Context)
    {
22.         val prefs =
    newBase.getSharedPreferences("clickanime_settings",
    Context.MODE_PRIVATE)
23.         val lang = prefs.getString("language",
    Locale.getDefault().language) ?: "en"
24.         val locale = Locale(lang)
25.         Locale.setDefault(locale)
26.         val config = newBase.resources.configuration
27.         config.setLocale(locale)
28.
    super.attachBaseContext(newBase.createConfigurationC
    ontext(config))
29.     }
30.
31.     override fun onCreate(savedInstanceState:
    Bundle?) {
32.         super.onCreate(savedInstanceState)
33.         enableEdgeToEdge()
34.
35.         val app = application as MovieApp
36.
37.         setContent {
38.             MovieAppTheme {

```

39.	Surface(modifier	=
	Modifier.fillMaxSize()) {	
40.	val navController	=
	rememberNavController()	
41.	val movieViewModel:	
	MovieViewModel = viewModel(	
42.	factory	=
	app.container.movieViewModelFactory	
43.	)	
44.	NavGraph(	
45.	navController	=
	navController,	
46.	movieViewModel	=
	movieViewModel,	
47.	appPreferences	=
	app.container.appPreferences,	
48.	onLocaleChange = { langCode -	
	> applyLocale(langCode) }	
49.	)	
50.	}	
51.	}	
52.	}	
53.	}	
54.		
55.	private fun applyLocale(langCode: String) {	
56.	getSharedPreferences("clickanime_settings",	
	Context.MODE_PRIVATE)	
57.	.edit()	
58.	.putString("language", langCode)	
59.	.apply()	
60.	recreate()	
61.	}	
62.	}	

Tabel 42. Source Code MovieApp.kt

1.	package com.example.clickanime
2.	

3.	import android.app.Application
4.	import com.example.clickanime.di.AppContainer
5.	import timber.log.Timber
6.	
7.	class MovieApp : Application() {
8.	
9.	lateinit var container: AppContainer
10.	private set
11.	
12.	override fun onCreate() {
13.	super.onCreate()
14.	container = AppContainer(this)
15.	if (BuildConfig.DEBUG) {
16.	Timber.plant(Timber.DebugTree())
17.	}
18.	Timber.d("MovieApp initialized")
19.	}
20.	}

Tabel 43. Source Code MovieDao.kt

1.	package com.example.clickanime.data.local.dao
2.	
3.	import androidx.room.*
4.	import com.example.clickanime.data.local.entity.FavoriteMovieEntity
5.	import com.example.clickanime.data.local.entity.MovieDetailEntity
6.	import com.example.clickanime.data.local.entity.MovieEntity
7.	import kotlinx.coroutines.flow.Flow
8.	
9.	@Dao
10.	interface MovieDao {

```

11.
12.         @Insert(onConflict = OnConflictStrategy.REPLACE)
13.         suspend fun insertMovies(movies:
List<MovieEntity>)
14.
15.         @Query("SELECT * FROM movies WHERE category =
:category ORDER BY popularity DESC")
16.         fun getMoviesByCategory(category: String):
Flow<List<MovieEntity>>
17.
18.         @Query("SELECT MAX(cachedAt) FROM movies WHERE
category = :category")
19.         suspend fun getLatestCacheTime(category: String):
Long?
20.
21.         @Query("SELECT * FROM movies WHERE title LIKE '%'
|| :q || '%' OR originalTitle LIKE '%' || :q || '%'")
22.         fun searchMovies(q: String):
Flow<List<MovieEntity>>
23.
24.         @Query("DELETE FROM movies WHERE category =
:category")
25.         suspend fun deleteByCategory(category: String)
26.
27.         @Query("DELETE FROM movies")
28.         suspend fun clearAll()
29.
30.         @Query("DELETE FROM movies WHERE cachedAt <
:expiryTime")
31.         suspend fun deleteExpired(expiryTime: Long)
32.     }
33.
34. @Dao
35. interface MovieDetailDao {
36.
37.         @Insert(onConflict = OnConflictStrategy.REPLACE)
38.         suspend fun insert(detail: MovieDetailEntity)

```

```

39.
40.     @Query("SELECT * FROM movie_details WHERE id =
      :movieId LIMIT 1")
41.     suspend fun getById(movieId: Int):
      MovieDetailEntity?
42.
43.     @Query("SELECT cachedAt FROM movie_details WHERE
      id = :movieId LIMIT 1")
44.     suspend fun getCacheTime(movieId: Int): Long?
45.
46.     @Query("DELETE FROM movie_details WHERE cachedAt
      < :expiryTime")
47.     suspend fun deleteExpired(expiryTime: Long)
48.
49.     @Query("DELETE FROM movie_details")
50.     suspend fun clearAll()
51. }
52.
53. @Dao
54. interface FavoriteMovieDao {
55.
56.     @Insert(onConflict = OnConflictStrategy.REPLACE)
57.     suspend fun add(favorite: FavoriteMovieEntity)
58.
59.     @Query("DELETE FROM favorite_movies WHERE movieId
      = :movieId")
60.     suspend fun removeById(movieId: Int)
61.
62.     @Query("SELECT * FROM favorite_movies ORDER BY
      addedAt DESC")
63.     fun getAll(): Flow<List<FavoriteMovieEntity>>
64.
65.     @Query("SELECT EXISTS(SELECT 1 FROM
      favorite_movies WHERE movieId = :movieId)")
66.     fun isFavorite(movieId: Int): Flow<Boolean>
67.

```

68.	@Query("SELECT EXISTS(SELECT 1 FROM favorite_movies WHERE movieId = :movieId)")
69.	suspend fun isFavoriteOnce(movieId: Int): Boolean
70.	
71.	@Query("SELECT COUNT(*) FROM favorite_movies")
72.	fun count(): Flow<Int>
73.	}

Tabel 44. Source Code MovieEntity.kt

1.	package com.example.clickanime.data.local.entity
2.	
3.	import androidx.room.Entity
4.	import androidx.room.PrimaryKey
5.	
6.	@Entity(tableName = "movies")
7.	data class MovieEntity(
8.	@PrimaryKey val id: Int,
9.	val title: String,
10.	val originalTitle: String,
11.	val overview: String,
12.	val posterPath: String?,
13.	val backdropPath: String?,
14.	val voteAverage: Double,
15.	val voteCount: Int,
16.	val releaseDate: String,
17.	val genreIds: String,
18.	val popularity: Double,
19.	val originalLanguage: String,
20.	val category: String,
21.	val cachedAt: Long = System.currentTimeMillis()
22.	)
23.	
24.	@Entity(tableName = "movie_details")
25.	data class MovieDetailEntity(

```

26.     @PrimaryKey val id: Int,
27.     val title: String,
28.     val originalTitle: String,
29.     val overview: String,
30.     val posterPath: String?,
31.     val backdropPath: String?,
32.     val voteAverage: Double,
33.     val voteCount: Int,
34.     val releaseDate: String,
35.     val runtime: Int?,
36.     val genres: String,
37.     val tagline: String,
38.     val status: String,
39.     val popularity: Double,
40.     val budget: Long,
41.     val revenue: Long,
42.     val homepage: String,
43.     val originalLanguage: String,
44.     val cachedAt: Long = System.currentTimeMillis()
45. )
46.
47. @Entity(tableName = "favorite_movies")
48. data class FavoriteMovieEntity(
49.     @PrimaryKey val movieId: Int,
50.     val title: String,
51.     val posterPath: String?,
52.     val voteAverage: Double,
53.     val releaseDate: String,
54.     val overview: String,
55.     val addedAt: Long = System.currentTimeMillis()
56. )

```

Tabel 45. Source Code AppPreferences.kt

1.	package com.example.clickanime.data.local
2.	
3.	import android.content.Context
4.	
5.	class AppPreferences(context: Context) {
6.	
7.	private val prefs =
	context.getSharedPreferences(PREFS_NAME,
	Context.MODE_PRIVATE)
8.	
9.	var language: String
10.	get() = prefs.getString(KEY_LANGUAGE, "en") ?:
	"en"
11.	set(v) = prefs.edit().putString(KEY_LANGUAGE,
	v).apply()
12.	
13.	var isNotificationsEnabled: Boolean
14.	get() = prefs.getBoolean(KEY_NOTIFICATIONS,
	false)
15.	set(v)
	prefs.edit().putBoolean(KEY_NOTIFICATIONS, v).apply()
16.	
17.	var isGridView: Boolean
18.	get() = prefs.getBoolean(KEY_GRID_VIEW,
	false)
19.	set(v)
	prefs.edit().putBoolean(KEY_GRID_VIEW, v).apply()
20.	
21.	var lastMovieCategory: String
22.	get() = prefs.getString(KEY_LAST_CATEGORY,
	CAT_POPULAR) ?: CAT_POPULAR
23.	set(v)
	prefs.edit().putString(KEY_LAST_CATEGORY, v).apply()
24.	
25.	companion object {
26.	const val PREFS_NAME = "clickanime_settings"

27.	private const val KEY_LANGUAGE	=
	"language"	
28.	private const val KEY_NOTIFICATIONS	=
	"notifications_enabled"	
29.	private const val KEY_GRID_VIEW	=
	"grid_view"	
30.	private const val KEY_LAST_CATEGORY	=
	"last_movie_category"	
31.		
32.	const val CAT_POPULAR	= "popular"
33.	const val CAT_NOW_PLAYING	= "now_playing"
34.	const val CAT_TOP_RATED	= "top_rated"
35.	const val CAT_UPCOMING	= "upcoming"
36.	}	
37.	}	

Tabel 46. Source Code MovieDatabase.kt

1.	package com.example.clickanime.data.local
2.	
3.	import android.content.Context
4.	import androidx.room.Database
5.	import androidx.room.Room
6.	import androidx.room.RoomDatabase
7.	import com.example.clickanime.data.local.dao.FavoriteMovieD ao
8.	import com.example.clickanime.data.local.dao.MovieDao
9.	import com.example.clickanime.data.local.dao.MovieDetailDao
10.	import com.example.clickanime.data.local.entity.FavoriteMov ieEntity
11.	import com.example.clickanime.data.local.entity.MovieDetail Entity
12.	import com.example.clickanime.data.local.entity.MovieEntity
13.	

```

14. @Database(
15.     entities = [MovieEntity::class,
MovieDetailEntity::class,
FavoriteMovieEntity::class],
16.     version = 1,
17.     exportSchema = false
18. )
19. abstract class MovieDatabase : RoomDatabase() {
20.
21.     abstract fun movieDao(): MovieDao
22.     abstract fun movieDetailDao(): MovieDetailDao
23.     abstract fun favoriteMovieDao(): FavoriteMovieDao
24.
25.     companion object {
26.         @Volatile private var INSTANCE:
MovieDatabase? = null
27.
28.         fun getInstance(context: Context):
MovieDatabase =
29.             INSTANCE ?: synchronized(this) {
30.                 Room.databaseBuilder(
31.                     context.applicationContext,
32.                     MovieDatabase::class.java,
33.                     "clickanime_movie_db"
34.                 )
35.                 .fallbackToDestructiveMigration()
36.                 .build()
37.                 .also { INSTANCE = it }
38.             }
39.     }
40. }

```

Tabel 47. Source Code MovieMapper.kt

1.	package com.example.clickanime.data.mapper
----	--

2.	
3.	import com.example.clickanime.data.local.entity.FavoriteMovieEntity
4.	import com.example.clickanime.data.local.entity.MovieDetailEntity
5.	import com.example.clickanime.data.local.entity.MovieEntity
6.	import com.example.clickanime.data.remote.model.TmdbGenreDto
7.	import com.example.clickanime.data.remote.model.TmdbMovieDetailDto
8.	import com.example.clickanime.data.remote.model.TmdbMovieDto
9.	import com.example.clickanime.model.FavoriteMovie
10.	import com.example.clickanime.model.Genre
11.	import com.example.clickanime.model.Movie
12.	import com.example.clickanime.model.MovieDetail
13.	
14.	fun TmdbMovieDto.toEntity(category: String) = MovieEntity(15. id = id, title = title, originalTitle = originalTitle, 16. overview = overview, posterPath = posterPath, backdropPath = backdropPath, 17. voteAverage = voteAverage, voteCount = voteCount, releaseDate = releaseDate, 18. genreIds = genreIds.joinToString(","), popularity = popularity, 19. originalLanguage = originalLanguage, category = category, 20. cachedAt = System.currentTimeMillis() 21.) 22.
23.	fun TmdbMovieDetailDto.toEntity() = MovieDetailEntity(

```

24.     id = id, title = title, originalTitle =
originalTitle,
25.     overview = overview, posterPath = posterPath,
backdropPath = backdropPath,
26.     voteAverage = voteAverage, voteCount = voteCount,
releaseDate = releaseDate,
27.     runtime = runtime, genres =
genres.joinToString(",") { "${it.id}:${it.name}" },
28.     tagline = tagline, status = status, popularity =
popularity,
29.     budget = budget, revenue = revenue, homepage =
homepage,
30.     originalLanguage = originalLanguage, cachedAt =
System.currentTimeMillis()
31. )
32.
33. fun TmdbMovieDto.toDomain(category: String) = Movie(
34.     id = id, title = title, originalTitle =
originalTitle,
35.     overview = overview, posterPath = posterPath,
backdropPath = backdropPath,
36.     voteAverage = voteAverage, voteCount = voteCount,
releaseDate = releaseDate,
37.     genreIds = genreIds, popularity = popularity,
38.     originalLanguage = originalLanguage, category =
category
39. )
40.
41. fun TmdbMovieDetailDto.toDomain() = MovieDetail(
42.     id = id, title = title, originalTitle =
originalTitle,
43.     overview = overview, posterPath = posterPath,
backdropPath = backdropPath,
44.     voteAverage = voteAverage, voteCount = voteCount,
releaseDate = releaseDate,
45.     runtime = runtime, genres = genres.map {
Genre(it.id, it.name) },
46.     tagline = tagline, status = status, popularity =
popularity,

```

47.	budget = budget, revenue = revenue, homepage = homepage,
48.	originalLanguage = originalLanguage
49.	)
50.	
51.	fun MovieEntity.toDomain() = Movie(
52.	id = id, title = title, originalTitle = originalTitle,
53.	overview = overview, posterPath = posterPath, backdropPath = backdropPath,
54.	voteAverage = voteAverage, voteCount = voteCount, releaseDate = releaseDate,
55.	genreIds = genreIds.split(",").mapNotNull { it.trim().toIntOrNull() },
56.	popularity = popularity, originalLanguage = originalLanguage, category = category
57.	)
58.	
59.	fun MovieDetailEntity.toDomain() = MovieDetail(
60.	id = id, title = title, originalTitle = originalTitle,
61.	overview = overview, posterPath = posterPath, backdropPath = backdropPath,
62.	voteAverage = voteAverage, voteCount = voteCount, releaseDate = releaseDate,
63.	runtime = runtime, genres = parseGenres(genres),
64.	tagline = tagline, status = status, popularity = popularity,
65.	budget = budget, revenue = revenue, homepage = homepage,
66.	originalLanguage = originalLanguage
67.	)
68.	
69.	fun FavoriteMovieEntity.toDomain() = FavoriteMovie(
70.	movieId = movieId, title = title, posterPath = posterPath,
71.	voteAverage = voteAverage, releaseDate = releaseDate,
72.	overview = overview, addedAt = addedAt

73.	)
74.	
75.	fun Movie.toFavoriteEntity() = FavoriteMovieEntity(
76.	movieId = id, title = title, posterPath = posterPath,
77.	voteAverage = voteAverage, releaseDate = releaseDate,
78.	overview = overview, addedAt = System.currentTimeMillis()
79.	)
80.	
81.	fun MovieDetail.toFavoriteEntity() = FavoriteMovieEntity(
82.	movieId = id, title = title, posterPath = posterPath,
83.	voteAverage = voteAverage, releaseDate = releaseDate,
84.	overview = overview, addedAt = System.currentTimeMillis()
85.	)
86.	
87.	private fun parseGenres(raw: String): List<Genre> {
88.	if (raw.isBlank()) return emptyList()
89.	return raw.split(",").mapNotNull { part ->
90.	val s = part.trim().split(":")
91.	if (s.size == 2) Genre(s[0].toIntOrNull() ?: return@mapNotNull null, s[1]) else null
92.	}
93.	}

Tabel 48. Source Code TmdbResponse.kt

1.	package com.example.clickanime.data.remote.model
2.	
3.	import kotlinx.serialization.SerialName
4.	import kotlinx.serialization.Serializable
5.	
6.	sealed class ApiResponse<out T> {

7.	data class Success<T>(val data: T) : ApiResponse<T>()
8.	data class Error(val code: Int, val message: String) : ApiResponse<Nothing>()
9.	data class Exception(val e: Throwable) : ApiResponse<Nothing>()
10.	}
11.	@Serializable
12.	data class TmdbMovieListResponse(
13.	@SerializedName("page") val page: Int = 1,
14.	@SerializedName("results") val results: List<TmdbMovieDto> = emptyList(),
15.	@SerializedName("total_pages") val totalPages: Int = 1,
16.	@SerializedName("total_results") val totalResults: Int = 0
17.	)
18.	
19.	@Serializable
20.	data class TmdbMovieDto(
21.	@SerializedName("id") val id: Int,
22.	@SerializedName("title") val title: String = "",
23.	@SerializedName("original_title") val originalTitle: String = "",
24.	@SerializedName("overview") val overview: String = "",
25.	@SerializedName("poster_path") val posterPath: String? = null,
26.	@SerializedName("backdrop_path") val backdropPath: String? = null,
27.	@SerializedName("vote_average") val voteAverage: Double = 0.0,
28.	@SerializedName("vote_count") val voteCount: Int = 0,
29.	@SerializedName("release_date") val releaseDate: String = "",
30.	@SerializedName("genre_ids") val genreIds: List<Int> = emptyList(),

```

31.     @SerializedName("popularity")          val popularity:
Double = 0.0,
32.     @SerializedName("adult")                val adult: Boolean
= false,
33.     @SerializedName("original_language")    val
originalLanguage: String = ""
34. )
35.
36. @Serializable
37. data class TmdbMovieDetailDto(
38.     @SerializedName("id")                    val id: Int,
39.     @SerializedName("title")                 val title: String
= "",
40.     @SerializedName("original_title")        val
originalTitle: String = "",
41.     @SerializedName("overview")              val overview:
String = "",
42.     @SerializedName("poster_path")           val posterPath:
String? = null,
43.     @SerializedName("backdrop_path")         val backdropPath:
String? = null,
44.     @SerializedName("vote_average")          val voteAverage:
Double = 0.0,
45.     @SerializedName("vote_count")            val voteCount:
Int = 0,
46.     @SerializedName("release_date")          val releaseDate:
String = "",
47.     @SerializedName("runtime")               val runtime: Int?
= null,
48.     @SerializedName("genres")                val genres:
List<TmdbGenreDto> = emptyList(),
49.     @SerializedName("tagline")               val tagline:
String = "",
50.     @SerializedName("status")                val status: String
= "",
51.     @SerializedName("popularity")            val popularity:
Double = 0.0,
52.     @SerializedName("budget")                val budget: Long
= 0L,
53.     @SerializedName("revenue")               val revenue: Long
= 0L,

```

54.	<pre> @SerializedName("homepage") val homepage: String = "", </pre>
55.	<pre> @SerializedName("original_language") val originalLanguage: String = "" </pre>
56.	<pre>) </pre>
57.	
58.	<pre> @Serializable </pre>
59.	<pre> data class TmdbGenreDto(</pre>
60.	<pre> @SerializedName("id") val id: Int, </pre>
61.	<pre> @SerializedName("name") val name: String </pre>
62.	<pre>) </pre>
63.	
64.	<pre> @Serializable </pre>
65.	<pre> data class TmdbGenreListResponse(</pre>
66.	<pre> @SerializedName("genres") val genres: List<TmdbGenreDto> = emptyList() </pre>
67.	<pre>) </pre>

Tabel 49. Source Code ApiHandler.kt

1.	<pre> package com.example.clickanime.data.remote </pre>
2.	
3.	<pre> import com.example.clickanime.data.remote.model.ApiResponse </pre>
4.	<pre> import retrofit2.Response </pre>
5.	<pre> import timber.log.Timber </pre>
6.	
7.	<pre> suspend fun <T> safeApiCall(apiCall: suspend () -> Response<T>): ApiResponse<T> { </pre>
8.	<pre> return try { </pre>
9.	<pre> val response = apiCall() </pre>
10.	<pre> if (response.isSuccessful) { </pre>
11.	<pre> val body = response.body() </pre>
12.	<pre> if (body != null) ApiResponse.Success(body) </pre>
13.	<pre> else ApiResponse.Error(response.code(), "Response body is null") </pre>

14.	} else {
15.	val errorMsg = response.errorBody()?.string() ?: "Unknown error"
16.	Timber.e("API Error [\${response.code()}]: \$errorMsg")
17.	ApiResponse.Error(response.code(), errorMsg)
18.	}
19.	} catch (e: Exception) {
20.	Timber.e(e, "API Exception")
21.	ApiResponse.Exception(e)
22.	}
23.	}

Tabel 50. Source Code NetworkClient.kt

1.	package com.example.clickanime.data.remote
2.	
3.	import com.example.clickanime.BuildConfig
4.	import com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
5.	import kotlinx.serialization.json.Json
6.	import okhttp3.MediaType.Companion.toMediaType
7.	import okhttp3.OkHttpClient
8.	import okhttp3.logging.HttpLoggingInterceptor
9.	import retrofit2.Retrofit
10.	import timber.log.Timber
11.	import java.util.concurrent.TimeUnit
12.	
13.	object NetworkClient {
14.	
15.	private val json = Json {
16.	ignoreUnknownKeys = true
17.	coerceInputValues = true
18.	encodeDefaults = true


```

19.     }
20.
21.     private val authInterceptor = okhttp3.Interceptor
22.     { chain ->
23.         val req = chain.request().newBuilder()
24.             .addHeader("Authorization", "Bearer
25.             ${BuildConfig.TMDB_API_KEY}")
26.             .addHeader("accept", "application/json")
27.             .build()
28.         chain.proceed(req)
29.     }
30.
31.     private val loggingInterceptor =
32.     HttpLoggingInterceptor { msg ->
33.         Timber.tag("OkHttp").d(msg)
34.     }.apply {
35.         level = if (BuildConfig.DEBUG)
36.         HttpLoggingInterceptor.Level.BODY
37.         else HttpLoggingInterceptor.Level.NONE
38.     }
39.
40.     private val okHttpClient = OkHttpClient.Builder()
41.         .addInterceptor(authInterceptor)
42.         .addInterceptor(loggingInterceptor)
43.         .connectTimeout(30, TimeUnit.SECONDS)
44.         .readTimeout(30, TimeUnit.SECONDS)
45.         .writeTimeout(30, TimeUnit.SECONDS)
46.         .build()
47.
48.     private val retrofit = Retrofit.Builder()
49.         .baseUrl(BuildConfig.TMDB_BASE_URL)
50.         .client(okHttpClient)
51.         .addConverterFactory(json.asConverterFactory("applic
52.         ation/json".toMediaType()))
53.         .build()

```

49.	
50.	val tmdbApiService: TmdbApiService = retrofit.create(TmdbApiService::class.java)
51.	}

Tabel 51. Source Code TmdbApiService.kt

1.	package com.example.clickanime.data.remote	
2.		
3.	import com.example.clickanime.data.remote.model.TmdbGenreLi stResponse	
4.	import com.example.clickanime.data.remote.model.TmdbMovieDe tailDto	
5.	import com.example.clickanime.data.remote.model.TmdbMovieLi stResponse	
6.	import retrofit2.Response	
7.	import retrofit2.http.GET	
8.	import retrofit2.http.Path	
9.	import retrofit2.http.Query	
10.		
11.	interface TmdbApiService {	
12.		
13.	@GET("movie/now_playing")	
14.	suspend fun getNowPlayingMovies(15.	@Query("language") language: String = "en-US",
16.	@Query("page") page: Int = 1	
17.	): Response<TmdbMovieListResponse>	
18.		
19.	@GET("movie/popular")	
20.	suspend fun getPopularMovies(21.	@Query("language") language: String = "en-US",
22.	@Query("page") page: Int = 1	
23.	): Response<TmdbMovieListResponse>	

```

24.
25.     @GET("movie/topRated")
26.     suspend fun getTopRatedMovies(
27.         @Query("language") language: String = "en-
US",
28.         @Query("page")      page: Int = 1
29.     ): Response<TmdbMovieListResponse>
30.
31.     @GET("movie/upcoming")
32.     suspend fun getUpcomingMovies(
33.         @Query("language") language: String = "en-
US",
34.         @Query("page")      page: Int = 1
35.     ): Response<TmdbMovieListResponse>
36.
37.     @GET("movie/{movie_id}")
38.     suspend fun getMovieDetail(
39.         @Path("movie_id") movieId: Int,
40.         @Query("language") language: String = "en-US"
41.     ): Response<TmdbMovieDetailDto>
42.
43.     @GET("search/movie")
44.     suspend fun searchMovies(
45.         @Query("query")      query: String,
46.         @Query("language") language: String = "en-
US",
47.         @Query("page")      page: Int = 1
48.     ): Response<TmdbMovieListResponse>
49.
50.     @GET("genre/movie/list")
51.     suspend fun getMovieGenres(
52.         @Query("language") language: String = "en-US"
53.     ): Response<TmdbGenreListResponse>
54. }

```

Tabel 52. Source Code MovieRepository.kt

1.	package com.example.clickanime.data.repository
2.	
3.	import com.example.clickanime.data.local.AppPreferences
4.	import com.example.clickanime.data.local.dao.FavoriteMovieD ao
5.	import com.example.clickanime.data.local.dao.MovieDao
6.	import com.example.clickanime.data.local.dao.MovieDetailDao
7.	import com.example.clickanime.data.mapper.toDomain
8.	import com.example.clickanime.data.mapper.toEntity
9.	import com.example.clickanime.data.mapper.toFavoriteEntity
10.	import com.example.clickanime.data.remote.TmdbApiService
11.	import com.example.clickanime.data.remote.model.ApiResponse
12.	import com.example.clickanime.data.remote.safeApiCall
13.	import com.example.clickanime.model.FavoriteMovie
14.	import com.example.clickanime.model.Movie
15.	import com.example.clickanime.model.MovieDetail
16.	import kotlinx.coroutines.flow.Flow
17.	import kotlinx.coroutines.flow.flow
18.	import kotlinx.coroutines.flow.map
19.	import timber.log.Timber
20.	
21.	class MovieRepository(22. private val api: TmdbApiService, 23. private val movieDao: MovieDao, 24. private val detailDao: MovieDetailDao, 25. private val favDao: FavoriteMovieDao, 26. private val prefs: AppPreferences 27.) {

```

28.     companion object {
29.         private const val TTL_MS = 30 * 60 * 1000L
30.
31.         const val CAT_POPULAR      = "popular"
32.         const val CAT_NOW_PLAYING  = "now_playing"
33.         const val CAT_TOP_RATED    = "top Rated"
34.         const val CAT_UPCOMING     = "upcoming"
35.     }
36.
37.     fun getMovies(
38.         category: String,
39.         forceRefresh: Boolean = false
40.     ): Flow<ApiResponse<List<Movie>>> = flow {
41.
42.         val      lastCache              =
movieDao.getLatestCacheTime(category) ?: 0L
43.         val      cacheValid              =
(System.currentTimeMillis() - lastCache) < TTL_MS
44.
45.         if (!cacheValid || forceRefresh) {
46.             Timber.d("Fetching $category from API
(forceRefresh=$forceRefresh)")
47.             val result = fetchAndCache(category)
48.             if (result is ApiResponse.Error || result
is ApiResponse.Exception) {
49.                 if (lastCache == 0L) {
50.                     @Suppress("UNCHECKED_CAST")
51.                     emit(result as
ApiResponse<List<Movie>>)
52.                     return@flow
53.                 }
54.                 Timber.w("API failed for $category,
using stale cache")
55.             }
56.         } else {

```


83.	else -> response as ApiResponse<List<Movie>>
84.	}
85.	}
86.	
87.	suspend fun getMovieDetail(movieId: Int, forceRefresh: Boolean = false): ApiResponse<MovieDetail> {
88.	val lastCache = detailDao.getCacheTime(movieId) ?: 0L
89.	val cacheValid = (System.currentTimeMillis() - lastCache) < TTL_MS
90.	
91.	if (cacheValid && !forceRefresh) {
92.	val cached = detailDao.getById(movieId)
93.	if (cached != null) {
94.	Timber.d("Using cached detail for movieId=\$movieId")
95.	return ApiResponse.Success(cached.toDomain())
96.	}
97.	}
98.	
99.	Timber.d("Fetching detail from API movieId=\$movieId")
100.	return when (val r = safeApiCall { api.getMovieDetail(movieId) }) {
101.	is ApiResponse.Success -> {
102.	detailDao.insert(r.data.toEntity())
103.	ApiResponse.Success(r.data.toDomain())
104.	}
105.	else -> {
106.	// Fallback ke stale cache jika ada
107.	val stale = detailDao.getById(movieId)
108.	if (stale != null) ApiResponse.Success(stale.toDomain())

```

109.         else r as ApiResponse<MovieDetail>
110.     }
111. }
112. }
113.
114. suspend fun searchMovies(query: String):
ApiResponse<List<Movie>> {
115.     if (query.isBlank()) return
ApiResponse.Success(emptyList())
116.     return when (val r = safeApiCall {
api.searchMovies(query) }) {
117.         is ApiResponse.Success ->
ApiResponse.Success(
118.             r.data.results.map {
it.toDomain(CAT_POPULAR) }
119.         )
120.         else -> r as ApiResponse<List<Movie>>
121.     }
122. }
123.
124. fun getAllFavorites(): Flow<List<FavoriteMovie>>
=
125.     favDao.getAll().map { it.map { e ->
e.toDomain() } }
126.
127. fun isFavorite(movieId: Int): Flow<Boolean> =
favDao.isFavorite(movieId)
128.
129. fun getFavoriteCount(): Flow<Int> =
favDao.count()
130.
131. suspend fun addFavorite(movie: Movie) {
favDao.add(movie.toFavoriteEntity()) }
132. suspend fun addFavorite(detail: MovieDetail){
favDao.add(detail.toFavoriteEntity()) }
133. suspend fun removeFavorite(movieId: Int) {
favDao.removeById(movieId) }
134.

```


135.	suspend fun clearExpiredCache() {
136.	val exp = System.currentTimeMillis() - TTL_MS
137.	movieDao.deleteExpired(exp)
138.	detailDao.deleteExpired(exp)
139.	Timber.d("Expired cache cleared")
140.	}
141.	
142.	suspend fun clearAllCache() {
143.	movieDao.clearAll()
144.	detailDao.clearAll()
145.	Timber.d("All cache cleared")
146.	}
147.	
148.	var lastCategory: String
149.	get() = prefs.lastMovieCategory
150.	set(v) { prefs.lastMovieCategory = v }
151.	}

Tabel 53. Source Code AppContainer.kt

1.	package com.example.clickanime.di
2.	
3.	import android.content.Context
4.	import com.example.clickanime.data.local.AppPreferences
5.	import com.example.clickanime.data.local.MovieDatabase
6.	import com.example.clickanime.data.remote.NetworkClient
7.	import com.example.clickanime.data.repository.MovieRepository
8.	import com.example.clickanime.viewmodel.MovieViewModelFactory
9.	
10.	class AppContainer(context: Context) {

11.	
12.	val appPreferences = AppPreferences(context)
13.	
14.	private val db = MovieDatabase.getInstance(context)
15.	
16.	private val repo = MovieRepository(api = NetworkClient.tmdbApiService, movieDao = db.movieDao(), detailDao = db.movieDetailDao(), favDao = db.favoriteMovieDao(), prefs = appPreferences)
23.	
24.	val movieViewModelFactory = MovieViewModelFactory(repo, appPreferences)
25.	}

Tabel 54. Source Code Movie.kt

1.	package com.example.clickanime.model
2.	
3.	private const val IMG_BASE = "https://image.tmdb.org/t/p/"
4.	
5.	data class Movie(val id: Int, val title: String, val originalTitle: String, val overview: String, val posterPath: String?, val backdropPath: String?, val voteAverage: Double, val voteCount: Int, val releaseDate: String, val genreIds: List<Int>,

```

16.         val popularity: Double,
17.         val originalLanguage: String,
18.         val category: String
19.     ) {
20.         val posterUrl: String?    get() = posterPath?.let
    { "${IMG_BASE}w342$it" }
21.         val     backdropUrl:      String?    get()      =
    backdropPath?.let { "${IMG_BASE}w780$it" }
22.         val     releaseYear:      String      get()      =
    releaseDate.take(4).isEmpty { "N/A" }
23.         val     formattedRating:   String      get()      =
    String.format("%.1f", voteAverage)
24.     }
25.
26. data class MovieDetail(
27.     val id: Int,
28.     val title: String,
29.     val originalTitle: String,
30.     val overview: String,
31.     val posterPath: String?,
32.     val backdropPath: String?,
33.     val voteAverage: Double,
34.     val voteCount: Int,
35.     val releaseDate: String,
36.     val runtime: Int?,
37.     val genres: List<Genre>,
38.     val tagline: String,
39.     val status: String,
40.     val popularity: Double,
41.     val budget: Long,
42.     val revenue: Long,
43.     val homepage: String,
44.     val originalLanguage: String
45. ) {
46.     val posterUrl: String?    get() = posterPath?.let
    { "${IMG_BASE}w500$it" }

```

47.	val backdropUrl: String? get() = backdropPath?.let { "\${IMG_BASE}w780\$it" }
48.	val releaseYear: String get() = releaseDate.take(4).isEmpty { "N/A" }
49.	val formattedRating: String get() = String.format("%.1f", voteAverage)
50.	val formattedRuntime: String get() = runtime?.let { "\${it / 60}h \${it % 60}m" } ?: "N/A"
51.	val genreNames: String get() = genres.joinToString(", ") { it.name }
52.	}
53.	
54.	data class Genre(val id: Int, val name: String)
55.	
56.	data class FavoriteMovie(
57.	val movieId: Int,
58.	val title: String,
59.	val posterPath: String?,
60.	val voteAverage: Double,
61.	val releaseDate: String,
62.	val overview: String,
63.	val addedAt: Long
64.	) {
65.	val posterUrl: String? get() = posterPath?.let { "\${IMG_BASE}w342\$it" }
66.	val releaseYear: String get() = releaseDate.take(4).isEmpty { "N/A" }
67.	val formattedRating: String get() = String.format("%.1f", voteAverage)
68.	}

Tabel 55. Source Code MovieUiState.kt

1.	package com.example.clickanime.viewmodel
2.	
3.	import com.example.clickanime.model.Movie
4.	import com.example.clickanime.model.MovieDetail
5.	

6.	sealed class MovieListUiState {
7.	object Loading : MovieListUiState()
8.	data class Success(val movies: List<Movie>) : MovieListUiState()
9.	data class Error(val message: String) : MovieListUiState()
10.	}
11.	
12.	sealed class MovieDetailUiState {
13.	object Idle : MovieDetailUiState()
14.	object Loading : MovieDetailUiState()
15.	data class Success(val movie: MovieDetail) : MovieDetailUiState()
16.	data class Error(val message: String) : MovieDetailUiState()
17.	}
18.	
19.	sealed class MovieSearchUiState {
20.	object Idle : MovieSearchUiState()
21.	object Loading : MovieSearchUiState()
22.	data class Success(val movies: List<Movie>) : MovieSearchUiState()
23.	data class Error(val message: String) : MovieSearchUiState()
24.	object Empty : MovieSearchUiState()
25.	}

Tabel 56. Source Code MovieViewModel.kt

1.	package com.example.clickanime.viewmodel
2.	
3.	import androidx.lifecycle.ViewModel
4.	import androidx.lifecycle.ViewModelProvider
5.	import androidx.lifecycle.viewModelScope
6.	import com.example.clickanime.data.local.AppPreferences
7.	import com.example.clickanime.data.remote.model.ApiResponse

```

8.      import
        com.example.clickanime.data.repository.MovieRepository
9.
10.     import com.example.clickanime.model.FavoriteMovie
11.     import com.example.clickanime.model.Movie
12.     import com.example.clickanime.model.MovieDetail
13.     import kotlinx.coroutines.Job
14.     import kotlinx.coroutines.delay
15.     import kotlinx.coroutines.flow.*
16.     import kotlinx.coroutines.launch
17.     import timber.log.Timber
18.
19.     class MovieViewModel(
20.         private val repo: MovieRepository,
21.         private val prefs: AppPreferences
22.     ) : ViewModel() {
23.
24.         private val _selectedCategory = MutableStateFlow(
25.             prefs.lastMovieCategory.ifEmpty {
26.                 MovieRepository.CAT_POPULAR
27.             }
28.         )
29.         val selectedCategory: StateFlow<String> =
30.             _selectedCategory.asStateFlow()
31.
32.         private val _listState =
33.             MutableStateFlow<MovieListUiState>(MovieListUiState.Loading)
34.         val listState: StateFlow<MovieListUiState> =
35.             _listState.asStateFlow()
36.
37.         private val _detailState =
38.             MutableStateFlow<MovieDetailUiState>(MovieDetailUiState.Idle)
39.         val detailState: StateFlow<MovieDetailUiState> =
40.             _detailState.asStateFlow()
41.
42.         private val _searchQuery = MutableStateFlow("")

```

35.	val searchQuery: StateFlow<String> =
	_searchQuery.asStateFlow()
36.	
37.	private val _searchState =
	MutableStateFlow<MovieSearchUiState>(MovieSearchUiState.Idle)
38.	val searchState: StateFlow<MovieSearchUiState> =
	_searchState.asStateFlow()
39.	
40.	val favorites: StateFlow<List<FavoriteMovie>> =
	repo.getAllFavorites()
41.	.stateIn(viewModelScope,
	SharingStarted.WhileSubscribed(5_000), emptyList())
42.	
43.	val favoriteCount: StateFlow<Int> =
	repo.getFavoriteCount()
44.	.stateIn(viewModelScope,
	SharingStarted.WhileSubscribed(5_000), 0)
45.	
46.	private var listJob: Job? = null
47.	private var searchJob: Job? = null
48.	
49.	init {
50.	loadMovies(_selectedCategory.value)
51.	viewModelScope.launch {
	repo.clearExpiredCache() }
52.	}
53.	
54.	fun selectCategory(cat: String) {
55.	if (_selectedCategory.value == cat) return
56.	_selectedCategory.value = cat
57.	prefs.lastMovieCategory = cat // Simpan ke
	SharedPreferences
58.	loadMovies(cat)
59.	Timber.d("Category → \$cat")
60.	}
61.	

```

62.     fun loadMovies(category: String =
        _selectedCategory.value, forceRefresh: Boolean =
        false) {
63.         listJob?.cancel()
64.         listJob = viewModelScope.launch {
65.             _listState.value =
        MovieListUiState.Loading
66.             repo.getMovies(category, forceRefresh)
67.                 .catch { e ->
68.                     _listState.value =
        MovieListUiState.Error(e.message ?: "Unknown error")
69.                 }
70.                 .collect { response ->
71.                     _listState.value = when
        (response) {
72.                         is ApiResponse.Success -> {
73.                             if
        (response.data.isEmpty()) MovieListUiState.Loading
74.                             else
        MovieListUiState.Success(response.data)
75.                         }
76.                         is ApiResponse.Error ->
        MovieListUiState.Error("Error ${response.code}:
        ${response.message}")
77.                         is ApiResponse.Exception ->
        MovieListUiState.Error(response.e.message ?:
        "Network error")
78.                     }
79.                 }
80.             }
81.         }
82.
83.         fun refresh() = loadMovies(forceRefresh = true)
84.
85.         fun loadDetail(movieId: Int) {
86.             _detailState.value =
        MovieDetailUiState.Loading
87.             viewModelScope.launch {

```



```

88.         _detailState.value = when (val r =
repo.getMovieDetail(movieId)) {
89.             is ApiResponse.Success      ->
MovieDetailUiState.Success(r.data)
90.             is ApiResponse.Error        ->
MovieDetailUiState.Error("Error          ${r.code}:
${r.message}")
91.             is ApiResponse.Exception    ->
MovieDetailUiState.Error(r.e.message ?: "Failed to
load")
92.         }
93.     }
94. }
95.
96. fun clearDetail() { _detailState.value =
MovieDetailUiState.Idle }
97.
98. fun updateSearch(query: String) {
99.     _searchQuery.value = query
100.    searchJob?.cancel()
101.    if (query.isBlank()) { _searchState.value =
MovieSearchUiState.Idle; return }
102.    searchJob = viewModelScope.launch {
103.        _searchState.value                =
MovieSearchUiState.Loading
104.        delay(500)    // debounce
105.        _searchState.value = when (val r =
repo.searchMovies(query)) {
106.            is ApiResponse.Success      -> if
(r.data.isEmpty()) MovieSearchUiState.Empty
107.            else
MovieSearchUiState.Success(r.data)
108.            is ApiResponse.Error        ->
MovieSearchUiState.Error("Error          ${r.code}:
${r.message}")
109.            is ApiResponse.Exception    ->
MovieSearchUiState.Error(r.e.message  ?:  "Search
failed")
110.        }
111.    }

```

```

112.     }
113.
114.     fun clearSearch() {
115.         _searchQuery.value = ""
116.         _searchState.value = MovieSearchUiState.Idle
117.         searchJob?.cancel()
118.     }
119.
120.     fun isFavoriteFlow(movieId: Int): Flow<Boolean> =
repo.isFavorite(movieId)
121.
122.     fun toggleFavorite(movie: Movie) {
123.         viewModelScope.launch {
124.             val isFav =
repo.isFavorite(movie.id).first()
125.             if (isFav) repo.removeFavorite(movie.id)
else repo.addFavorite(movie)
126.         }
127.     }
128.
129.     fun toggleFavorite(detail: MovieDetail) {
130.         viewModelScope.launch {
131.             val isFav =
repo.isFavorite(detail.id).first()
132.             if (isFav) repo.removeFavorite(detail.id)
else repo.addFavorite(detail)
133.         }
134.     }
135.
136.     fun removeFavorite(movieId: Int) {
137.         viewModelScope.launch {
repo.removeFavorite(movieId) }
138.     }
139.
140.     fun clearAllCache() {

```

141.	viewModelScope.launch {
142.	repo.clearAllCache()
143.	loadMovies(forceRefresh = true)
144.	}
145.	}
146.	}
147.	
148.	class MovieViewModelFactory(
149.	private val repo: MovieRepository,
150.	private val prefs: AppPreferences
151.	) : ViewModelProvider.Factory {
152.	override fun <T : ViewModel> create(modelClass: Class<T>): T {
153.	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java)) {
154.	@Suppress("UNCHECKED_CAST")
155.	return MovieViewModel(repo, prefs) as T
156.	}
157.	throw IllegalArgumentException("Unknown ViewModel: \${modelClass.name}")
158.	}
159.	}

Tabel 57. Source Code NavGraph.kt

1.	package com.example.clickanime.navigation
2.	
3.	import androidx.compose.runtime.Composable
4.	import androidx.navigation.NavHostController
5.	import androidx.navigation.NavType
6.	import androidx.navigation.compose.NavHost
7.	import androidx.navigation.compose.composable
8.	import androidx.navigation.navArgument
9.	import
	com.example.clickanime.data.local.AppPreferences

```

10. import com.example.clickanime.ui.detail.DetailScreen
11. import
    com.example.clickanime.ui.favorites.FavoritesScreen
12. import com.example.clickanime.ui.home.HomeScreen
13. import
    com.example.clickanime.ui.language.LanguageScreen
14. import com.example.clickanime.ui.search.SearchScreen
15. import
    com.example.clickanime.ui.settings.SettingsScreen
16. import
    com.example.clickanime.viewmodel.MovieViewModel
17.
18. @Composable
19. fun NavGraph(
20.     navController: NavHostController,
21.     movieViewModel: MovieViewModel,
22.     appPreferences: AppPreferences,
23.     onLocaleChange: (String) -> Unit
24. ) {
25.     NavHost(
26.         navController = navController,
27.         startDestination = Screen.Home.route
28.     ) {
29.         composable(Screen.Home.route) {
30.             HomeScreen(
31.                 viewModel = movieViewModel,
32.                 onMovieClick = { id ->
navController.navigate(Screen.Detail.createRoute(id))
                },
33.                 onSearchClick = {
navController.navigate(Screen.Search.route) },
34.                 onFavoritesClick = {
navController.navigate(Screen.Favorites.route) },
35.                 onLanguageClick = {
navController.navigate(Screen.Language.route) },
36.                 onSettingsClick = {
navController.navigate(Screen.Settings.route) }

```

```

37.         )
38.     }
39.
40.     composable(
41.         route = Screen.Detail.route,
42.         arguments = listOf(navArgument("movieId")
43. { type = NavType.IntType })
44.     ) { back ->
45.         val movieId =
46. back.arguments?.getInt("movieId") ?:
47. return@composable
48.         DetailScreen(
49.             movieId = movieId,
50.             viewModel = movieViewModel,
51.             onBack = {
52. navController.popBackStack() }
53.         )
54.     }
55.
56.     composable(Screen.Search.route) {
57.         SearchScreen(
58.             viewModel = movieViewModel,
59.             onMovieClick = { id ->
60. navController.navigate(Screen.Detail.createRoute(id))
61. },
62.             onBack = {
63. navController.popBackStack() }
64.         )
65.     }
66.
67.     composable(Screen.Favorites.route) {
68.         FavoritesScreen(
69.             viewModel = movieViewModel,
70.             onMovieClick = { id ->
71. navController.navigate(Screen.Detail.createRoute(id))
72. },

```

64.	onBack	=	{
	navController.popBackStack() }		
65.	)		
66.	}		
67.			
68.	composable(Screen.Language.route) {		
69.	LanguageScreen(		
70.	onLanguageSelected = { lang ->		
71.	onLocaleChange(lang)		
72.	navController.popBackStack()		
73.	},		
74.	onBack	=	{
	navController.popBackStack() }		
75.	)		
76.	}		
77.			
78.	composable(Screen.Settings.route) {		
79.	SettingsScreen(		
80.	viewModel = movieViewModel,		
81.	preferences = appPreferences,		
82.	onBack	=	{
	navController.popBackStack() }		
83.	)		
84.	}		
85.	}		
86.	}		

Tabel 58. Source Code Screen.kt

1.	package com.example.clickanime.navigation
2.	
3.	sealed class Screen(val route: String) {
4.	object Home : Screen("home")
5.	object Detail : Screen("detail/{movieId}") {
6.	fun createRoute(movieId: Int) =
	"detail/\$movieId"

7.	}
8.	object Search : Screen("search")
9.	object Favorites : Screen("favorites")
10.	object Settings : Screen("settings")
11.	object Language : Screen("language")
12.	}

Tabel 59. Source Code HomeScreen.kt

1.	package com.example.clickanime.ui.home
2.	
3.	import androidx.compose.foundation.background
4.	import androidx.compose.foundation.clickable
5.	import androidx.compose.foundation.layout.*
6.	import androidx.compose.foundation.lazy.LazyColumn
7.	import androidx.compose.foundation.lazy.LazyRow
8.	import androidx.compose.foundation.lazy.items
9.	import androidx.compose.foundation.shape.RoundedCornerShape
10.	import androidx.compose.material.icons.Icons
11.	import androidx.compose.material.icons.filled.*
12.	import androidx.compose.material3.*
13.	import androidx.compose.runtime.*
14.	import androidx.compose.ui.Alignment
15.	import androidx.compose.ui.Modifier
16.	import androidx.compose.ui.draw.clip
17.	import androidx.compose.ui.graphics.Brush
18.	import androidx.compose.ui.graphics.Color
19.	import androidx.compose.ui.layout.ContentScale
20.	import androidx.compose.ui.res.stringResource
21.	import androidx.compose.ui.text.font.FontWeight
22.	import androidx.compose.ui.text.style.TextOverflow
23.	import androidx.compose.ui.unit.dp
24.	import androidx.compose.ui.unit.sp
25.	import coil.compose.AsyncImage

```

26. import com.example.clickanime.R
27. import
   com.example.clickanime.data.repository.MovieRepository
28. import com.example.clickanime.model.Movie
29. import
   com.example.clickanime.viewmodel.MovieListUiState
30. import
   com.example.clickanime.viewmodel.MovieViewModel
31.
32. private data class CategoryTab(val key: String, val
   labelRes: Int)
33.
34. private val CATEGORIES = listOf(
35.     CategoryTab(MovieRepository.CAT_POPULAR,
   R.string.category_popular),
36.     CategoryTab(MovieRepository.CAT_NOW_PLAYING,
   R.string.category_now_playing),
37.     CategoryTab(MovieRepository.CAT_TOP_RATED,
   R.string.category_top_rated),
38.     CategoryTab(MovieRepository.CAT_UPCOMING,
   R.string.category_upcoming),
39. )
40.
41. @OptIn(ExperimentalMaterial3Api::class)
42. @Composable
43. fun HomeScreen(
44.     viewModel: MovieViewModel,
45.     onMovieClick: (Int) -> Unit,
46.     onSearchClick: () -> Unit,
47.     onFavoritesClick: () -> Unit,
48.     onLanguageClick: () -> Unit,
49.     onSettingsClick: () -> Unit
50. ) {
51.     val listState by
   viewModel.listState.collectAsState()
52.     val selectedCategory by
   viewModel.selectedCategory.collectAsState()

```


53.	val favCount	by
	viewModel.favoriteCount.collectAsState()	
54.		
55.	Scaffold(	
56.	topBar = {	
57.	TopAppBar(	
58.	title = {	
	Text(stringResource(R.string.app_title), fontWeight =	
	FontWeight.Bold) },	
59.	actions = {	
60.	IconButton(onClick =	
	onSearchClick) {	
61.	Icon(Icons.Default.Search,	
	contentDescription =	
	stringResource(R.string.nav_search))	
62.	}	
63.	BadgedBox(badge = {	
64.	if (favCount > 0) Badge {	
	Text(favCount.toString()) }	
65.	}) {	
66.	IconButton(onClick =	
	onFavoritesClick) {	
67.	Icon(Icons.Default.Favorite, contentDescription =	
	stringResource(R.string.nav_favorites))	
68.	}	
69.	}	
70.	IconButton(onClick =	
	onLanguageClick) {	
71.	Icon(Icons.Default.Language,	
	contentDescription =	
	stringResource(R.string.change_language))	
72.	}	
73.	IconButton(onClick =	
	onSettingsClick) {	
74.	Icon(Icons.Default.Settings,	
	contentDescription =	
	stringResource(R.string.nav_settings))	
75.	}	
76.	},	

```

77.         colors =
TopAppBarDefaults.topAppBarColors(
78.             containerColor =
MaterialTheme.colorScheme.primary,
79.             titleContentColor =
MaterialTheme.colorScheme.onPrimary,
80.             actionIconContentColor =
MaterialTheme.colorScheme.onPrimary
81.         )
82.     )
83. }
84. ) { padding ->
85.     Column(
86.         modifier = Modifier
87.             .fillMaxSize()
88.             .padding(padding)
89.     ) {
90.         val selectedIdx = CATEGORIES.indexOfFirst
{ it.key == selectedCategory }.coerceAtLeast(0)
91.         ScrollableTabRow(
92.             selectedTabIndex = selectedIdx,
93.             edgePadding = 8.dp,
94.             containerColor =
MaterialTheme.colorScheme.surface,
95.             contentColor =
MaterialTheme.colorScheme.primary
96.         ) {
97.             CATEGORIES.forEach { cat ->
98.                 val selected = selectedCategory
== cat.key
99.                 Tab(
100.                     selected = selected,
101.                     onClick = {
viewModel.selectCategory(cat.key) },
102.                     text = {
103.                         Text(

```

```

104.         text =
stringResource(cat.labelRes),
105.         fontWeight = if
(selected) FontWeight.Bold else FontWeight.Normal
106.     )
107. }
108. )
109. }
110. }
111.
112.     when (val state = listState) {
113.         is MovieListUiState.Loading ->
LoadingContent()
114.         is MovieListUiState.Error ->
ErrorContent(state.message) { viewModel.refresh() }
115.         is MovieListUiState.Success ->
MovieContent(
116.             movies = state.movies,
117.             viewModel = viewModel,
118.             onMovieClick = onMovieClick
119.         )
120.     }
121. }
122. }
123. }
124.
125. @Composable
126. private fun MovieContent(
127.     movies: List<Movie>,
128.     viewModel: MovieViewModel,
129.     onMovieClick: (Int) -> Unit
130. ) {
131.     LazyColumn(
132.         contentPadding = PaddingValues(bottom =
16.dp),
133.         modifier = Modifier.fillMaxSize()

```

```

134.         ) {
135.             if (movies.isNotEmpty()) {
136.                 item {
137.                     Text(
138.                         text =
stringResource(R.string.featured),
139.                         style =
MaterialTheme.typography.titleMedium,
140.                         fontWeight = FontWeight.Bold,
141.                         modifier = Modifier.padding(start
= 16.dp, top = 12.dp, bottom = 8.dp)
142.                     )
143.                     MovieCarousel(movies =
movies.take(6), onMovieClick = onMovieClick) =
144.                     Spacer(Modifier.height(12.dp))
145.                     HorizontalDivider(modifier =
Modifier.padding(horizontal = 16.dp)) =
146.                     Text(
147.                         text =
"${stringResource(R.string.all_movies)}
(${movies.size})",
148.                         style =
MaterialTheme.typography.titleMedium, =
149.                         fontWeight = FontWeight.Bold,
150.                         modifier = Modifier.padding(start
= 16.dp, top = 10.dp, bottom = 4.dp)
151.                     )
152.                 }
153.             }
154.
155.             items(movies, key = { it.id }) { movie ->
156.                 MovieListItem(
157.                     movie = movie,
158.                     onMovieClick = {
onMovieClick(movie.id) },
159.                     onFavoriteClick = {
viewModel.toggleFavorite(movie) },

```

```

160.         isFavoriteFlow                                     =
viewModel.isFavoriteFlow(movie.id)
161.     )
162. }
163. }
164. }
165.
166. @Composable
167. private fun MovieCarousel(movies: List<Movie>,
onMovieClick: (Int) -> Unit) {
168.     LazyRow(
169.         contentPadding = PaddingValues(horizontal
= 16.dp),
170.         horizontalArrangement =
Arrangement.spacedBy(12.dp)
171.     ) {
172.         items(movies, key = { it.id }) { movie ->
173.             CarouselCard(movie = movie, onClick = {
onMovieClick(movie.id) })
174.         }
175.     }
176. }
177.
178. @Composable
179. private fun CarouselCard(movie: Movie, onClick: () ->
Unit) {
180.     Card(
181.         modifier = Modifier
182.             .width(160.dp)
183.             .height(220.dp)
184.             .clickable { onClick() },
185.         shape = RoundedCornerShape(16.dp),
186.         elevation = CardDefaults.cardElevation(6.dp)
187.     ) {
188.         Box(Modifier.fillMaxSize()) {
189.             AsyncImage(

```

```

190.         model = movie.posterUrl,
191.         contentDescription = movie.title,
192.         contentScale = ContentScale.Crop,
193.         modifier =
Modifier.fillMaxSize()
194.     )
195.     Box(
196.         Modifier.fillMaxSize().background(
197.             Brush.verticalGradient(
198.                 colors =
listOf(Color.Transparent, Color.Black.copy(.80f)),
199.                 startY = 120f
200.             )
201.         )
202.     )
203.     Column(
204.         Modifier.align(Alignment.BottomStart).padding(10.dp)
205.     ) {
206.         Text(movie.title, color =
Color.White, fontWeight = FontWeight.Bold,
207.             fontSize = 12.sp, maxLines = 2,
overflow = TextOverflow.Ellipsis)
208.         Row(verticalAlignment =
Alignment.CenterVertically) {
209.             Icon(Icons.Default.Star, null,
tint = Color(0xFFFFD700), modifier =
Modifier.size(12.dp))
210.             Spacer(Modifier.width(3.dp))
211.             Text(movie.formattedRating, color
= Color.White, fontSize = 11.sp)
212.             Spacer(Modifier.width(6.dp))
213.             Text(movie.releaseYear, color =
Color.White.copy(.7f), fontSize = 11.sp)
214.         }
215.     }
216. }

```

```

217.     }
218. }
219.
220. @Composable
221. private fun MovieListItem(
222.     movie: Movie,
223.     onMovieClick: () -> Unit,
224.     onFavoriteClick: () -> Unit,
225.     isFavoriteFlow:
kotlinx.coroutines.flow.Flow<Boolean>
226. ) {
227.     val isFav by
isFavoriteFlow.collectAsState(initial = false)
228.
229.     Card(
230.         modifier = Modifier
231.             .fillMaxWidth()
232.             .padding(horizontal = 16.dp, vertical =
5.dp)
233.             .clickable { onMovieClick() },
234.         shape = RoundedCornerShape(14.dp),
235.         elevation = CardDefaults.cardElevation(3.dp),
236.         colors =
CardDefaults.cardColors(containerColor
=
MaterialTheme.colorScheme.surface)
237.     ) {
238.         Row(
239.             modifier =
Modifier.fillMaxWidth().padding(10.dp),
240.             verticalAlignment = Alignment.Top
241.         ) {
242.             AsyncImage(
243.                 model = movie.posterUrl,
244.                 contentDescription = movie.title,
245.                 contentScale =
ContentScale.Crop,

```

```

246.         modifier = Modifier
247.             .width(72.dp).height(105.dp)
248.             .clip(RoundedCornerShape(10.dp))
249.         )
250.         Spacer(Modifier.width(12.dp))
251.         Column(Modifier.weight(1f)) {
252.             Row(
253.                 Modifier.fillMaxWidth(),
254.                 horizontalArrangement =
Arrangement.SpaceBetween,
255.                 verticalAlignment =
Alignment.CenterVertically
256.             ) {
257.                 Text(
258.                     movie.title,
259.                     style =
MaterialTheme.typography.titleSmall,
260.                     fontWeight = FontWeight.Bold,
261.                     maxLines = 2,
262.                     overflow =
TextOverflow.Ellipsis,
263.                     modifier =
Modifier.weight(1f)
264.                 )
265.                 IconButton(onClick =
onFavoriteClick, modifier = Modifier.size(36.dp)) {
266.                     Icon(
267.                         imageVector = if
(isFav) Icons.Default.Favorite else
Icons.Default.FavoriteBorder,
268.                         contentDescription = if
(isFav)
stringResource(R.string.remove_from_favorites)
269.                         else
stringResource(R.string.add_to_favorites),
270.                         tint = if
(isFav) Color.Red else
MaterialTheme.colorScheme.onSurfaceVariant,

```


271.	modifier	=
	Modifier.size(20.dp)	
272.	)	
273.	}	
274.	}	
275.	Spacer(Modifier.height(3.dp))	
276.	Row(verticalAlignment	=
	Alignment.CenterVertically) {	
277.	Icon(Icons.Default.Star, null,	
	tint = Color(0xFFFFD700), modifier	=
	Modifier.size(14.dp))	
278.	Spacer(Modifier.width(3.dp))	
279.	Text("\${movie.formattedRating}	
	("\${movie.voteCount})",	
280.	style	=
	MaterialTheme.typography.bodySmall,	
281.	color	=
	MaterialTheme.colorScheme.onSurfaceVariant)	
282.	Spacer(Modifier.width(8.dp))	
283.	Icon(Icons.Default.CalendarToday, null,	
284.	tint	=
	MaterialTheme.colorScheme.primary, modifier	=
	Modifier.size(12.dp))	
285.	Spacer(Modifier.width(2.dp))	
286.	Text(movie.releaseYear, style	=
	MaterialTheme.typography.bodySmall,	
287.	color	=
	MaterialTheme.colorScheme.primary)	
288.	}	
289.	Spacer(Modifier.height(5.dp))	
290.	Text(	
291.	movie.overview,	
292.	style	=
	MaterialTheme.typography.bodySmall,	
293.	color	=
	MaterialTheme.colorScheme.onSurfaceVariant,	
294.	maxLines = 3, overflow	=
	TextOverflow.Ellipsis,	

```

295.             lineHeight = 18.sp
296.         )
297.     }
298. }
299. }
300. }
301.
302. @Composable
303. private fun LoadingContent() {
304.     Box(Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
305.         Column(horizontalAlignment =
Alignment.CenterHorizontally) {
306.             CircularProgressIndicator(color =
MaterialTheme.colorScheme.primary)
307.             Spacer(Modifier.height(16.dp))
308.             Text(stringResource(R.string.loading_movies), color =
MaterialTheme.colorScheme.onSurfaceVariant)
309.         }
310.     }
311. }
312.
313. @Composable
314. private fun ErrorContent(message: String, onRetry: ()
-> Unit) {
315.     Box(Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
316.         Column(horizontalAlignment =
Alignment.CenterHorizontally, modifier =
Modifier.padding(32.dp)) {
317.             Icon(Icons.Default.ErrorOutline, null,
318.                 tint =
MaterialTheme.colorScheme.error, modifier =
Modifier.size(64.dp))
319.             Spacer(Modifier.height(16.dp))
320.             Text(stringResource(R.string.failed_to_load), style =
MaterialTheme.typography.titleMedium,

```

321.	fontWeight = FontWeight.Bold)
322.	Spacer (Modifier.height (8.dp))
323.	Text (message, style = MaterialTheme.typography.bodySmall,
324.	color = MaterialTheme.colorScheme.onSurfaceVariant)
325.	Spacer (Modifier.height (20.dp))
326.	Button (onClick = onRetry) {
327.	Icon (Icons.Default.Refresh, null)
328.	Spacer (Modifier.width (8.dp))
329.	Text (stringResource (R.string.retry))
330.	}
331.	}
332.	}
333.	}

Tabel 60. Source Code DetailScreen.kt

1.	package com.example.clickanime.ui.detail
2.	
3.	import androidx.compose.foundation.background
4.	import androidx.compose.foundation.layout.*
5.	import androidx.compose.foundation.rememberScrollState
6.	import androidx.compose.foundation.shape.RoundedCornerShape
7.	import androidx.compose.foundation.verticalScroll
8.	import androidx.compose.material.icons.Icons
9.	import androidx.compose.material.icons.automirrored.filled.A rrowBack
10.	import androidx.compose.material.icons.filled.*
11.	import androidx.compose.material3.*
12.	import androidx.compose.runtime.*
13.	import androidx.compose.ui.Alignment
14.	import androidx.compose.ui.Modifier
15.	import androidx.compose.ui.draw.clip

```

16. import androidx.compose.ui.graphics.Brush
17. import androidx.compose.ui.graphics.Color
18. import
   androidx.compose.ui.graphics.vector.ImageVector
19. import androidx.compose.ui.layout.ContentScale
20. import androidx.compose.ui.res.stringResource
21. import androidx.compose.ui.text.font.FontWeight
22. import androidx.compose.ui.text.style.TextAlign
23. import androidx.compose.ui.unit.dp
24. import androidx.compose.ui.unit.sp
25. import coil.compose.AsyncImage
26. import com.example.clickanime.R
27. import com.example.clickanime.model.MovieDetail
28. import
   com.example.clickanime.viewmodel.MovieDetailUiState
29. import
   com.example.clickanime.viewmodel.MovieViewModel
30.
31. @OptIn(ExperimentalMaterial3Api::class)
32. @Composable
33. fun DetailScreen(
34.     movieId: Int,
35.     viewModel: MovieViewModel,
36.     onBack: () -> Unit
37. ) {
38.     val uiState by
   viewModel.detailState.collectAsState()
39.
40.     LaunchedEffect(movieId) {
   viewModel.loadDetail(movieId) }
41.     DisposableEffect(Unit) { onDispose {
   viewModel.clearDetail() } }
42.
43.     Scaffold(
44.         topBar = {
45.             TopAppBar(

```

```

46.         title = {
47.             val title = (uiState as?
MovieDetailUiState.Success)?.movie?.title ?: ""
48.             Text(title, fontWeight =
FontWeight.Bold, maxLines = 1, overflow =
androidx.compose.ui.text.style.TextOverflow.Ellipsis)
49.         },
50.         navigationIcon = {
51.             IconButton(onClick = onBackPressed) {
52.
Icon(Icons.AutoMirrored.Filled.ArrowBack,
stringResource(R.string.back))
53.         }
54.     },
55.     actions = {
56.         if (uiState is
MovieDetailUiState.Success) {
57.             val movie = (uiState as
MovieDetailUiState.Success).movie
58.             val isFav by
viewModel.isFavoriteFlow(movie.id).collectAsState(fal
se)
59.             IconButton(onClick = {
viewModel.toggleFavorite(movie) }) {
60.                 Icon(
61.                     imageVector =
if (isFav) Icons.Default.Favorite else
Icons.Default.FavoriteBorder,
62.                     contentDescription =
if (isFav)
stringResource(R.string.remove_from_favorites)
63.                     else
stringResource(R.string.add_to_favorites),
64.                     tint =
if (isFav) Color.Red else
MaterialTheme.colorScheme.onPrimary
65.                 )
66.             }
67.         }
68.     },

```

69.	colors	=
	TopAppBarDefaults.topAppBarColors(	
70.	containerColor	=
	MaterialTheme.colorScheme.primary,	
71.	titleContentColor	=
	MaterialTheme.colorScheme.onPrimary,	
72.	navigationIconContentColor	=
	MaterialTheme.colorScheme.onPrimary	
73.	)	
74.	)	
75.	}	
76.	) { padding ->	
77.	when (val state = uiState) {	
78.	is MovieDetailUiState.Loading,	
	MovieDetailUiState.Idle -> {	
79.	Box(Modifier.fillMaxSize().padding(padding),	
	contentAlignment = Alignment.Center) {	
80.	CircularProgressIndicator()	
81.	}	
82.	}	
83.	is MovieDetailUiState.Error -> {	
84.	Box(Modifier.fillMaxSize().padding(padding),	
	contentAlignment = Alignment.Center) {	
85.	Column(horizontalAlignment	=
	Alignment.CenterHorizontally, modifier	=
	Modifier.padding(24.dp)) {	
86.	Icon(Icons.Default.ErrorOutline, null, tint	=
	MaterialTheme.colorScheme.error, modifier	=
	Modifier.size(56.dp))	
87.	Spacer(Modifier.height(12.dp))	
88.	Text(state.message, textAlign	
	= TextAlign.Center)	
89.	Spacer(Modifier.height(16.dp))	
90.	Button(onClick	= {
	viewModel.loadDetail(movieId) }	{
	Text(stringResource(R.string.retry)) }	

```

91.         }
92.     }
93. }
94.     is MovieDetailUiState.Success -> {
95.         DetailContent(movie = state.movie,
paddingValues = padding)
96.     }
97. }
98. }
99. }
100.
101. @Composable
102. private fun DetailContent(movie: MovieDetail,
paddingValues: PaddingValues) {
103.     Column(
104.         modifier = Modifier
105.             .fillMaxSize()
106.             .padding(paddingValues)
107.             .verticalScroll(rememberScrollState())
108.     ) {
109.         Box(Modifier.fillMaxWidth().height(260.dp)) {
110.             AsyncImage(
111.                 model = movie.backdropUrl
?: movie.posterUrl,
112.                 contentDescription = movie.title,
113.                 contentScale = ContentScale.Crop,
114.                 modifier =
Modifier.fillMaxSize()
115.             )
116.             Box(Modifier.fillMaxSize().background(
117.                 Brush.verticalGradient(listOf(Color.Transparent,
MaterialTheme.colorScheme.background))
118.             ))
119.         Column(Modifier.align(Alignment.BottomStart).padding(
16.dp)) {

```

```

120.         Text(movie.title, color =
MaterialTheme.colorScheme.onBackground,
121.             fontWeight =
FontWeight.ExtraBold, fontSize = 22.sp)
122.         if (movie.tagline.isNotBlank()) {
123.             Text("\${movie.tagline}\\"",
color = MaterialTheme.colorScheme.primary,
124.             fontSize = 13.sp, modifier =
Modifier.padding(top = 2.dp))
125.         }
126.     }
127. }
128.
129. Row(Modifier.fillMaxWidth().padding(horizontal =
16.dp).offset(y = (-20).dp)) {
130.     AsyncImage(
131.         model = movie.posterUrl,
132.         contentDescription = movie.title,
133.         contentScale = ContentScale.Crop,
134.         modifier =
Modifier.width(100.dp).height(148.dp).clip(RoundedCor
nerShape(12.dp))
135.     )
136.     Spacer(Modifier.width(14.dp))
137.     Card(
138.         modifier =
Modifier.weight(1f).align(Alignment.Bottom),
139.         shape = RoundedCornerShape(12.dp),
140.         colors =
CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.primaryContainer)
141.     ) {
142.         Column(Modifier.padding(14.dp),
verticalArrangement = Arrangement.spacedBy(8.dp)) {
143.             StatRow(Icons.Default.Star,
stringResource(R.string.rating),
"\${movie.formattedRating}/10", Color(0xFFFFD700))
144.             StatRow(Icons.Default.Schedule,
stringResource(R.string.runtime),

```


	movie.formattedRuntime,	
	MaterialTheme.colorScheme.primary)	
145.	StatRow(Icons.Default.CalendarToday,	
	stringResource(R.string.year), movie.releaseYear,	
	MaterialTheme.colorScheme.secondary)	
146.	StatRow(Icons.Default.HowToVote,	
	stringResource(R.string.votes),	
	movie.voteCount.toString(),	
	MaterialTheme.colorScheme.tertiary)	
147.	}	
148.	}	
149.	}	
150.		
151.	Column(Modifier.fillMaxWidth().padding(horizontal =	=
	16.dp)) {	
152.	if (movie.genres.isNotEmpty()) {	
153.	Row(horizontalArrangement	=
	Arrangement.spacedBy(8.dp), modifier	=
	Modifier.padding(bottom = 10.dp)) {	
154.	movie.genres.take(4).forEach {	
	genre ->	
155.	Surface(shape	=
	RoundedCornerShape(20.dp), color	=
	MaterialTheme.colorScheme.secondaryContainer) {	
156.	Text(genre.name, modifier	
	= Modifier.padding(horizontal = 12.dp, vertical =	
	5.dp),	
157.	style	=
	MaterialTheme.typography.labelSmall,	
158.	color	=
	MaterialTheme.colorScheme.onSecondaryContainer,	
159.	fontWeight	=
	FontWeight.SemiBold)	
160.	}	
161.	}	
162.	}	
163.	}	
164.		
165.	if (movie.status.isNotBlank()) {	

```

166.         Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.padding(bottom = 12.dp)) {
167.             Icon(Icons.Default.Info, null, Modifier.size(14.dp), tint = MaterialTheme.colorScheme.primary)
168.             Spacer(Modifier.width(4.dp))
169.             Text("${stringResource(R.string.status)}: ${movie.status}",
170.                 style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.primary)
171.             Spacer(Modifier.width(12.dp))
172.             Text("${stringResource(R.string.language)}: ${movie.originalLanguage.uppercase()}",
173.                 style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
174.         }
175.     }
176.
177.     Text(stringResource(R.string.overview), style = MaterialTheme.typography.titleMedium,
178.         fontWeight = FontWeight.Bold, modifier = Modifier.padding(bottom = 8.dp))
179.     Text(
180.         text = movie.overview.ifBlank { stringResource(R.string.no_overview) },
181.         style = MaterialTheme.typography.bodyMedium,
182.         color = MaterialTheme.colorScheme.onSurfaceVariant,
183.         lineHeight = 22.sp,
184.         modifier = Modifier.padding(bottom = 20.dp)
185.     )
186.
187.     if (movie.budget > 0 || movie.revenue > 0)
{

```

```

188. Text(stringResource(R.string.box_office), style =
    MaterialTheme.typography.titleMedium,
189.         fontWeight = FontWeight.Bold,
    modifier = Modifier.padding(bottom = 8.dp))
190.         Card(
191.             modifier =
    Modifier.fillMaxWidth().padding(bottom = 20.dp),
192.             shape =
    RoundedCornerShape(12.dp),
193.             colors =
    CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant)
194.         ) {
195.     Row(Modifier.fillMaxWidth().padding(16.dp),
    horizontalArrangement = Arrangement.SpaceEvenly) {
196.     BoxOfficeCol(stringResource(R.string.budget),
    formatCurrency(movie.budget))
197.     VerticalDivider(Modifier.height(40.dp))
198.     BoxOfficeCol(stringResource(R.string.revenue),
    formatCurrency(movie.revenue))
199.     }
200.     }
201.     }
202.
203.     Spacer(Modifier.height(24.dp))
204.     }
205.     }
206. }
207.
208. @Composable
209. private fun StatRow(icon: ImageVector, label: String,
    value: String, tint: Color) {
210.     Row(verticalAlignment =
    Alignment.CenterVertically) {

```

211.	Icon(icon, null, tint = tint, modifier = Modifier.size(16.dp))	
212.	Spacer(Modifier.width(6.dp))	
213.	Column {	
214.	Text(label, style = MaterialTheme.typography.labelSmall, color = MaterialTheme.colorScheme.onSurfaceVariant)	
215.	Text(value, style = MaterialTheme.typography.bodySmall, fontWeight = FontWeight.SemiBold)	
216.	}	
217.	}	
218.	}	
219.		
220.	@Composable	
221.	private fun BoxOfficeCol(label: String, value: String) {	
222.	Column(horizontalAlignment = Alignment.CenterHorizontally) {	
223.	Text(label, style = MaterialTheme.typography.labelMedium, color = MaterialTheme.colorScheme.onSurfaceVariant)	
224.	Text(value, fontWeight = FontWeight.Bold)	
225.	}	
226.	}	
227.		
228.	private fun formatCurrency(amount: Long): String = when {	
229.	amount == 0L -> "N/A"	
230.	amount >= 1_000_000_000 -> "\$%.1fB".format(amount / 1_000_000_000.0)	
231.	amount >= 1_000_000 -> "\$%.1fM".format(amount / 1_000_000.0)	
232.	else -> "\$\$amount"	
233.	}	

Tabel 61. Source Code LanguageScreen.kt

1.	package com.example.clickanime.ui.language
----	--

```

2.
3. import androidx.compose.foundation.border
4. import androidx.compose.foundation.clickable
5. import androidx.compose.foundation.layout.*
6. import
  androidx.compose.foundation.shape.RoundedCornerShape
7. import androidx.compose.material.icons.Icons
8. import
  androidx.compose.material.icons.automirrored.filled.A
  rrowBack
9. import androidx.compose.material.icons.filled.Check
10. import
  androidx.compose.material.icons.filled.Language
11. import androidx.compose.material3.*
12. import androidx.compose.runtime.*
13. import androidx.compose.ui.Alignment
14. import androidx.compose.ui.Modifier
15. import androidx.compose.ui.platform.LocalContext
16. import androidx.compose.ui.res.stringResource
17. import androidx.compose.ui.text.font.FontWeight
18. import androidx.compose.ui.unit.dp
19. import androidx.compose.ui.unit.sp
20. import com.example.clickanime.R
21.
22. private data class LangOption(
23.     val code: String,
24.     val nativeName: String,
25.     val labelRes: Int,
26.     val flag: String
27. )
28.
29. private val LANGUAGES = listOf(
30.     LangOption("en", "English",
  R.string.lang_english, "GB"),
31.     LangOption("id", "Bahasa Indonesia",
  R.string.lang_indonesian, "ID"),

```

```

32. )
33.
34. @OptIn(ExperimentalMaterial3Api::class)
35. @Composable
36. fun LanguageScreen(
37.     onLanguageSelected: (String) -> Unit,
38.     onBack: () -> Unit
39. ) {
40.     val context = LocalContext.current
41.     val currentLang = context.resources.configuration.locales[0].language =
42.
43.     var selected by remember {
44.         mutableStateOf(currentLang) }
45.
46.     Scaffold(
47.         topBar = {
48.             TopAppBar(
49.                 title = {
50.                     Text(
51.                         text = stringResource(R.string.change_language), =
52.                         fontWeight = FontWeight.Bold
53.                     )
54.                 },
55.                 navigationIcon = {
56.                     IconButton(onClick = onBack) {
57.                         Icon(
58.                             imageVector = Icons.AutoMirrored.Filled.ArrowBack, =
59.                             contentDescription = stringResource(R.string.back) =
60.                         )
61.                     }
62.                 },

```

62.	colors	=
	TopAppBarDefaults.topAppBarColors(	
63.	containerColor	=
	MaterialTheme.colorScheme.primary,	
64.	titleContentColor	=
	MaterialTheme.colorScheme.onPrimary,	
65.	navigationIconContentColor	=
	MaterialTheme.colorScheme.onPrimary	
66.	)	
67.	)	
68.	}	
69.	) { padding ->	
70.	Column(	
71.	modifier = Modifier	
72.	.fillMaxSize()	
73.	.padding(padding)	
74.	.padding(24.dp)	
75.	) {	
76.	Icon(	
77.	imageVector = Icons.Default.Language,	
78.	contentDescription = null,	
79.	modifier = Modifier	
80.	.size(64.dp)	
81.	.align(Alignment.CenterHorizontally),	
82.	tint	=
	MaterialTheme.colorScheme.primary	
83.	)	
84.	Spacer(Modifier.height(8.dp))	
85.	Text(	
86.	text	=
	stringResource(R.string.select_language),	
87.	style	=
	MaterialTheme.typography.titleLarge,	
88.	fontWeight = FontWeight.Bold,	
89.	modifier	=
	Modifier.align(Alignment.CenterHorizontally)	

```

90.         )
91.         Spacer (Modifier.height (28.dp))
92.
93.         LANGUAGES.forEach { lang ->
94.             val isSelected = selected == lang.code
95.             Card(
96.                 modifier = Modifier
97.                     .fillMaxWidth()
98.                     .padding(vertical = 6.dp)
99.                     .then(
100.                        if (isSelected)
101.                            Modifier.border(
102.                                width = 2.dp,
103.                                color =
104.                                    MaterialTheme.colorScheme.primary,
105.                                shape =
106.                                    RoundedCornerShape (14.dp)
107.                                ) else Modifier
108.                            )
109.                        .clickable { selected =
110.                            lang.code },
111.                        shape =
112.                            RoundedCornerShape (14.dp),
113.                        colors = CardDefaults.cardColors(
114.                            containerColor = if
115.                                (isSelected)
116.                                    MaterialTheme.colorScheme.primaryContainer
117.                                    else
118.                                    MaterialTheme.colorScheme.surface
119.                                ),
120.                        elevation =
121.                            CardDefaults.cardElevation(
122.                                defaultElevation = if
123.                                    (isSelected) 4.dp else 1.dp
124.                                )

```



```

117.         ) {
118.             Row(
119.                 modifier = Modifier
120.                     .fillMaxWidth()
121.                     .padding(18.dp),
122.                 verticalAlignment                =
Alignment.CenterVertically
123.             ) {
124.                 Text(
125.                     text = lang.flag,
126.                     fontSize = 32.sp,
127.                     modifier                =
Modifier.padding(end = 16.dp)
128.                 )
129.                 Column(Modifier.weight(1f)) {
130.                     Text(
131.                         text                =
lang.nativeName,
132.                         fontWeight         =
FontWeight.Bold,
133.                         style              =
MaterialTheme.typography.bodyLarge
134.                     )
135.                     Text(
136.                         text                =
stringResource(lang.labelRes),
137.                         style              =
MaterialTheme.typography.bodySmall,
138.                         color              =
MaterialTheme.colorScheme.onSurfaceVariant
139.                     )
140.                 }
141.                 if (isSelected) {
142.                     Icon(
143.                         imageVector        =
Icons.Default.Check,

```

144.	contentDescription =
145.	MaterialTheme.colorScheme.primary,
146.	Modifier.size(24.dp)
147.	)
148.	}
149.	}
150.	}
151.	}
152.	
153.	Spacer(Modifier.weight(1f))
154.	
155.	Button(
156.	onClick = {
157.	onLanguageSelected(selected) },
158.	modifier = Modifier
159.	.fillMaxWidth()
160.	.height(52.dp),
161.	shape = RoundedCornerShape(14.dp)
162.	) {
163.	Text(
164.	text =
165.	stringResource(R.string.apply_language),
166.	fontWeight = FontWeight.Bold,
167.	fontSize = 16.sp
168.	)
169.	}
170.	}

Tabel 62. Source Code FavoritesScreen.kt

1.	package com.example.clickanime.ui.favorites
----	---

```

2.
3. import androidx.compose.foundation.clickable
4. import androidx.compose.foundation.layout.*
5. import androidx.compose.foundation.lazy.LazyColumn
6. import androidx.compose.foundation.lazy.items
7. import
  androidx.compose.foundation.shape.RoundedCornerShape
8. import androidx.compose.material.icons.Icons
9. import
  androidx.compose.material.icons.automirrored.filled.A
  rrowBack
10. import androidx.compose.material.icons.filled.*
11. import androidx.compose.material3.*
12. import androidx.compose.runtime.*
13. import androidx.compose.ui.Alignment
14. import androidx.compose.ui.Modifier
15. import androidx.compose.ui.draw.clip
16. import androidx.compose.ui.graphics.Color
17. import androidx.compose.ui.layout.ContentScale
18. import androidx.compose.ui.res.stringResource
19. import androidx.compose.ui.text.font.FontWeight
20. import androidx.compose.ui.text.style.TextAlign
21. import androidx.compose.ui.text.style.TextOverflow
22. import androidx.compose.ui.unit.dp
23. import androidx.compose.ui.unit.sp
24. import coil.compose.AsyncImage
25. import com.example.clickanime.R
26. import com.example.clickanime.model.FavoriteMovie
27. import
  com.example.clickanime.viewmodel.MovieViewModel
28.
29. @OptIn(ExperimentalMaterial3Api::class)
30. @Composable
31. fun FavoritesScreen(
32.     viewModel: MovieViewModel,

```

```

33.     onMovieClick: (Int) -> Unit,
34.     onBack: () -> Unit
35. ) {
36.     val favorites by
viewModel.favorites.collectAsState()
37.
38.     Scaffold(
39.         topBar = {
40.             TopAppBar(
41.                 title = {
42.                     Text(
43.                         text =
"$${stringResource(R.string.nav_favorites)}
(${favorites.size})",
44.                         fontWeight = FontWeight.Bold
45.                     )
46.                 },
47.                 navigationIcon = {
48.                     IconButton(onClick = onBack) {
49.                         Icon(
50.                             imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
51.                             contentDescription =
stringResource(R.string.back)
52.                         )
53.                     }
54.                 },
55.                 colors =
TopAppBarDefaults.topAppBarColors(
56.                     containerColor =
MaterialTheme.colorScheme.primary,
57.                     titleContentColor =
MaterialTheme.colorScheme.onPrimary,
58.                     navigationIconContentColor =
MaterialTheme.colorScheme.onPrimary
59.                 )
60.             )

```

61.	}	
62.	) { padding ->	
63.	if (favorites.isEmpty()) {	
64.	EmptyFavoritesContent(modifier	=
	Modifier.padding(padding))	
65.	} else {	
66.	LazyColumn(	
67.	modifier = Modifier	
68.	.fillMaxSize()	
69.	.padding(padding),	
70.	contentPadding	=
	PaddingValues(vertical = 8.dp)	
71.	) {	
72.	item {	
73.	Text(	
74.	text	=
	stringResource(R.string.favorites_saved_hint),	
75.	style	=
	MaterialTheme.typography.bodySmall,	
76.	color	=
	MaterialTheme.colorScheme.onSurfaceVariant,	
77.	modifier	=
	Modifier.padding(horizontal = 16.dp, vertical = 6.dp)	
78.	)	
79.	}	
80.	items(items = favorites, key = {	
	it.movieId }) { movie ->	
81.	FavoriteItem(	
82.	movie = movie,	
83.	onClick	= {
	onMovieClick(movie.movieId) },	
84.	onRemove	= {
	viewModel.removeFavorite(movie.movieId) }	
85.	)	
86.	}	
87.	item { Spacer(Modifier.height(16.dp))	
	}	

```

88.         }
89.     }
90. }
91. }
92.
93. @Composable
94. private fun EmptyFavoritesContent(modifier: Modifier =
Modifier) {
95.     Box(
96.         modifier = modifier.fillMaxSize(),
97.         contentAlignment = Alignment.Center
98.     ) {
99.         Column(
100.             horizontalAlignment                =
Alignment.CenterHorizontally,
101.             modifier = Modifier.padding(32.dp)
102.         ) {
103.             Icon(
104.                 imageVector                =
Icons.Default.FavoriteBorder,
105.                 contentDescription = null,
106.                 modifier = Modifier.size(96.dp),
107.                 tint                =
MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha
= 0.35f)
108.             )
109.             Spacer(Modifier.height(20.dp))
110.             Text(
111.                 text                =
stringResource(R.string.favorites_empty_title),
112.                 style                =
MaterialTheme.typography.titleLarge,
113.                 fontWeight = FontWeight.Bold
114.             )
115.             Spacer(Modifier.height(10.dp))
116.             Text(

```

```

117.         text                                     =
stringResource(R.string.favorites_empty_desc),
118.         style                                     =
MaterialTheme.typography.bodyMedium,
119.         color                                     =
MaterialTheme.colorScheme.onSurfaceVariant,
120.         textAlign = TextAlign.Center,
121.         lineHeight = 22.sp
122.     )
123. }
124. }
125. }
126.
127. @Composable
128. private fun FavoriteItem(
129.     movie: FavoriteMovie,
130.     onClick: () -> Unit,
131.     onRemove: () -> Unit
132. ) {
133.     var showDialog by remember { mutableStateOf(false) }
134.
135.     if (showDialog) {
136.         AlertDialog(
137.             onDismissRequest = { showDialog = false },
138.             icon = Icon(Icons.Default.DeleteOutline, contentDescription =
= null) },
139.             title = Text(stringResource(R.string.remove_favorite)) },
140.             text = {
141.                 Text(
142.                     String.format(stringResource(R.string.remove_favorite
_confirm), movie.title)
143.                 )
144.             },

```

```

145.         confirmButton = {
146.             TextButton(onClick = {
147.                 onRemove()
148.                 showDialog = false
149.             }) {
150.                 Text(
151.                     stringResource(R.string.remove),
152.                     color =
MaterialTheme.colorScheme.error
153.                 )
154.             }
155.         },
156.         dismissButton = {
157.             TextButton(onClick = { showDialog =
false }) {
158.                 Text(stringResource(R.string.cancel))
159.             }
160.         }
161.     )
162. }
163.
164. Card(
165.     modifier = Modifier
166.         .fillMaxWidth()
167.         .padding(horizontal = 16.dp, vertical =
5.dp)
168.         .clickable { onClick() },
169.     shape = RoundedCornerShape(14.dp),
170.     elevation = CardDefaults.cardElevation(3.dp),
171.     colors =
CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
172. ) {
173.     Row(

```


174.	modifier = Modifier	
175.	.fillMaxWidth()	
176.	.padding(10.dp),	
177.	verticalAlignment	=
	Alignment.CenterVertically	
178.	) {	
179.	AsyncImage(	
180.	model = movie.posterUrl,	
181.	contentDescription = movie.title,	
182.	contentScale = ContentScale.Crop,	
183.	modifier = Modifier	
184.	.width(68.dp)	
185.	.height(100.dp)	
186.	.clip(RoundedCornerShape(10.dp))	
187.	)	
188.		
189.	Spacer(Modifier.width(12.dp))	
190.		
191.	Column(Modifier.weight(1f)) {	
192.	Text(	
193.	text = movie.title,	
194.	fontWeight = FontWeight.Bold,	
195.	fontSize = 14.sp,	
196.	maxLines = 2,	
197.	overflow = TextOverflow.Ellipsis	
198.	)	
199.	Spacer(Modifier.height(5.dp))	
200.	Row(verticalAlignment	=
	Alignment.CenterVertically) {	
201.	Icon(	
202.	Icons.Default.Star, null,	
203.	tint = Color(0xFFFFD700),	
204.	modifier	=
	Modifier.size(14.dp)	
205.	)	

```

206.                Spacer(Modifier.width(3.dp))
207.                Text(
208.                    text = movie.formattedRating,
209.                    style                                =
MaterialTheme.typography.bodySmall
210.                )
211.                Spacer(Modifier.width(10.dp))
212.                Icon(
213.                    Icons.Default.CalendarToday,
null,
214.                    tint                                =
MaterialTheme.colorScheme.primary,
215.                    modifier                            =
Modifier.size(12.dp)
216.                )
217.                Spacer(Modifier.width(3.dp))
218.                Text(
219.                    text = movie.releaseYear,
220.                    style                                =
MaterialTheme.typography.bodySmall,
221.                    color                                =
MaterialTheme.colorScheme.primary
222.                )
223.            }
224.            Spacer(Modifier.height(5.dp))
225.            Text(
226.                text = movie.overview,
227.                style                                =
MaterialTheme.typography.bodySmall,
228.                color                                =
MaterialTheme.colorScheme.onSurfaceVariant,
229.                maxLines = 2,
230.                overflow = TextOverflow.Ellipsis,
231.                lineHeight = 18.sp
232.            )
233.        }
234.

```

235.	IconButton(
236.	onClick = { showDialog = true },
237.	modifier = Modifier.size(40.dp)
238.	) {
239.	Icon(
240.	imageVector =
241.	Icons.Default.Favorite,
242.	contentDescription =
243.	stringResource(R.string.remove_from_favorites),
244.	tint = Color.Red,
245.	modifier = Modifier.size(22.dp)
246.	)
247.	}
248.	}

Tabel 63. Source Code SearchScreen.kt

1.	package com.example.clickanime.ui.search
2.	
3.	import androidx.compose.foundation.clickable
4.	import androidx.compose.foundation.layout.*
5.	import androidx.compose.foundation.lazy.LazyColumn
6.	import androidx.compose.foundation.lazy.items
7.	import
8.	androidx.compose.foundation.shape.RoundedCornerShape
9.	import
10.	androidx.compose.foundation.text.KeyboardActions
11.	import
12.	androidx.compose.foundation.text.KeyboardOptions
13.	import androidx.compose.material.icons.Icons
14.	import
15.	androidx.compose.material.icons.automirrored.filled.A
16.	rrowBack
17.	import androidx.compose.material.icons.filled.*
18.	import androidx.compose.material3.*

```

14. import androidx.compose.runtime.*
15. import androidx.compose.ui.Alignment
16. import androidx.compose.ui.Modifier
17. import androidx.compose.ui.draw.clip
18. import androidx.compose.ui.graphics.Color
19. import androidx.compose.ui.layout.ContentScale
20. import androidx.compose.ui.platform.LocalFocusManager
21. import androidx.compose.ui.res.stringResource
22. import androidx.compose.ui.text.font.FontWeight
23. import androidx.compose.ui.text.input.ImeAction
24. import androidx.compose.ui.text.style.TextAlign
25. import androidx.compose.ui.text.style.TextOverflow
26. import androidx.compose.ui.unit.dp
27. import androidx.compose.ui.unit.sp
28. import coil.compose.AsyncImage
29. import com.example.clickanime.R
30. import com.example.clickanime.model.Movie
31. import
  com.example.clickanime.viewmodel.MovieSearchUiState
32. import
  com.example.clickanime.viewmodel.MovieViewModel
33.
34. @OptIn(ExperimentalMaterial3Api::class)
35. @Composable
36. fun SearchScreen(
37.     viewModel: MovieViewModel,
38.     onMovieClick: (Int) -> Unit,
39.     onBack: () -> Unit
40. ) {
41.     val query by
  viewModel.searchQuery.collectAsState()
42.     val searchState by
  viewModel.searchState.collectAsState()
43.     val focusMgr = LocalFocusManager.current
44.

```

45.	DisposableEffect(Unit) { onDispose {	
	viewModel.clearSearch() } }	
46.		
47.	Scaffold(	
48.	topBar = {	
49.	TopAppBar(	
50.	navigationIcon = {	
51.	IconButton(onClick = onBackPressed) {	
52.	Icon(Icons.AutoMirrored.Filled.ArrowBack,	
53.	stringResource(R.string.back), tint =	
	MaterialTheme.colorScheme.onPrimary)	
54.	}	
55.	},	
56.	title = {	
57.	OutlinedTextField(	
58.	value = query,	
59.	onValueChange = {	
	viewModel.updateSearch(it) },	
60.	placeholder = {	
	Text(stringResource(R.string.search_hint),	
61.	style =	
	MaterialTheme.typography.bodyMedium) },	
62.	modifier =	
	Modifier.fillMaxWidth(),	
63.	singleLine = true,	
64.	shape =	
	RoundedCornerShape(24.dp),	
65.	colors =	
	OutlinedTextFieldDefaults.colors(	
66.	focusedBorderColor =	
	MaterialTheme.colorScheme.onPrimary,	
67.	unfocusedBorderColor =	
	MaterialTheme.colorScheme.onPrimary.copy(.5f),	
68.	focusedTextColor =	
	MaterialTheme.colorScheme.onPrimary,	
69.	unfocusedTextColor =	
	MaterialTheme.colorScheme.onPrimary,	

```

70.         cursorColor =
MaterialTheme.colorScheme.onPrimary,
71.         focusedPlaceholderColor
= MaterialTheme.colorScheme.onPrimary.copy(.7f),
72.         unfocusedPlaceholderColor=
MaterialTheme.colorScheme.onPrimary.copy(.7f)
73.     ),
74.     trailingIcon = {
75.         if (query.isNotBlank()) {
76.             IconButton(onClick =
{ viewModel.clearSearch() }) {
77.                 Icon(Icons.Default.Clear,
stringResource(R.string.clear_search),
78.                 tint =
MaterialTheme.colorScheme.onPrimary)
79.             }
80.         }
81.     },
82.     keyboardOptions =
KeyboardOptions(imeAction = ImeAction.Search),
83.     keyboardActions =
KeyboardActions(onSearch = { focusMgr.clearFocus() })
84. )
85. },
86.     colors =
TopAppBarDefaults.topAppBarColors(
87.         containerColor =
MaterialTheme.colorScheme.primary
88.     )
89. )
90. }
91. ) { padding ->
92.     Box(Modifier.fillMaxSize().padding(padding))
93.     {
94.         when (val state = searchState) {

```

95.	EmptyPlaceholder(Icons.Default.Search,	
	stringResource(R.string.search_empty_hint))	
96.	}	
97.	is MovieSearchUiState.Loading -> {	
98.	Box(Modifier.fillMaxSize(),	
	contentAlignment = Alignment.Center) {	
99.	CircularProgressIndicator()	
100.	}	
101.	}	
102.	is MovieSearchUiState.Empty -> {	
103.	EmptyPlaceholder(Icons.Default.SearchOff,	
104.	"\${stringResource(R.string.search_no_results)}"	
	\"\$query\"")	
105.	}	
106.	is MovieSearchUiState.Error -> {	
107.	Box(Modifier.fillMaxSize(),	
	contentAlignment = Alignment.Center) {	
108.	Column(horizontalAlignment =	=
	Alignment.CenterHorizontally, modifier	=
	Modifier.padding(24.dp)) {	
109.	Icon(Icons.Default.ErrorOutline, null,	
110.	tint	=
	MaterialTheme.colorScheme.error, modifier	=
	Modifier.size(56.dp))	
111.	Spacer(Modifier.height(12.dp))	
112.	Text(state.message,	
	textAlign = TextAlign.Center)	
113.	}	
114.	}	
115.	}	
116.	is MovieSearchUiState.Success -> {	
117.	LazyColumn(contentPadding	=
	PaddingValues(vertical = 8.dp)) {	
118.	item {	

```

119.                 Text(
120.                     text =
121.                         "${state.movies.size}
122.                         ${stringResource(R.string.search_results)}
123.                         \"$query\"",
124.                     style =
125.                         MaterialTheme.typography.bodySmall,
126.                     color =
127.                         MaterialTheme.colorScheme.onSurfaceVariant,
128.                     modifier =
129.                         Modifier.padding(horizontal = 16.dp, vertical = 6.dp)
130.                     )
131.                 }
132.                 items(state.movies, key = {
133.                     it.id }) { movie ->
134.                     SearchResultItem(movie =
135.                         movie, onClick = { onMovieClick(movie.id) })
136.                 }
137.             }
138.         }
139.     }
140. }
141.
142. @Composable
143. private fun EmptyPlaceholder(icon:
144.     androidx.compose.ui.graphics.vector.ImageVector,
145.     label: String) {
146.     Box(Modifier.fillMaxSize(), contentAlignment =
147.         Alignment.Center) {
148.         Column(horizontalAlignment =
149.             Alignment.CenterHorizontally, modifier =
150.             Modifier.padding(32.dp)) {
151.             Icon(icon, null, modifier =
152.                 Modifier.size(80.dp),
153.                 tint =
154.                     MaterialTheme.colorScheme.onSurfaceVariant.copy(.35f)
155.                 )
156.             Spacer(Modifier.height(16.dp))

```



```

143.         Text(label, style =
MaterialTheme.typography.bodyLarge,
144.             color =
MaterialTheme.colorScheme.onSurfaceVariant.copy(.6f),
textAlign = TextAlign.Center)
145.     }
146. }
147. }
148.
149. @Composable
150. private fun SearchResultItem(movie: Movie, onClick: ()
-> Unit) {
151.     Card(
152.         modifier =
Modifier.fillMaxWidth().padding(horizontal = 16.dp,
vertical = 5.dp).clickable { onClick() },
153.         shape = RoundedCornerShape(12.dp),
154.         elevation = CardDefaults.cardElevation(2.dp)
155.     ) {
156.         Row(Modifier.fillMaxWidth().padding(10.dp),
verticalAlignment = Alignment.CenterVertically) {
157.             AsyncImage(
158.                 model = movie.posterUrl,
159.                 contentDescription = movie.title,
160.                 contentScale = ContentScale.Crop,
161.                 modifier =
Modifier.width(55.dp).height(80.dp).clip(RoundedCorne
rShape(8.dp))
162.             )
163.             Spacer(Modifier.width(12.dp))
164.             Column(Modifier.weight(1f)) {
165.                 Text(movie.title, fontWeight =
FontWeight.SemiBold, fontSize = 14.sp,
166.                     maxLines = 2, overflow =
TextOverflow.Ellipsis)
167.                 Spacer(Modifier.height(4.dp))
168.                 Row(verticalAlignment =
Alignment.CenterVertically) {

```

169.	Icon(Icons.Default.Star, null, tint = Color(0xFFFFD700), modifier = Modifier.size(13.dp))
170.	Spacer(Modifier.width(3.dp))
171.	Text(movie.formattedRating, style = MaterialTheme.typography.bodySmall)
172.	Spacer(Modifier.width(8.dp))
173.	Text(movie.releaseYear, style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.primary)
174.	}
175.	Spacer(Modifier.height(4.dp))
176.	Text(movie.overview, style = MaterialTheme.typography.bodySmall,
177.	color = MaterialTheme.colorScheme.onSurfaceVariant,
178.	maxLines = 2, overflow = TextOverflow.Ellipsis)
179.	}
180.	Icon(Icons.Default.ChevronRight, null, tint = MaterialTheme.colorScheme.onSurfaceVariant)
181.	}
182.	}
183.	}

Tabel 64. Source Code Settings Screen.kt

1.	package com.example.clickanime.ui.settings
2.	
3.	import androidx.compose.foundation.layout.*
4.	import androidx.compose.foundation.rememberScrollState
5.	import androidx.compose.foundation.shape.RoundedCornerShape
6.	import androidx.compose.foundation.verticalScroll
7.	import androidx.compose.material.icons.Icons
8.	import androidx.compose.material.icons.automirrored.filled.A rrowBack

```

9. import androidx.compose.material.icons.filled.*
10. import androidx.compose.material3.*
11. import androidx.compose.runtime.*
12. import androidx.compose.ui.Alignment
13. import androidx.compose.ui.Modifier
14. import
    androidx.compose.ui.graphics.vector.ImageVector
15. import androidx.compose.ui.res.stringResource
16. import androidx.compose.ui.text.font.FontWeight
17. import androidx.compose.ui.unit.dp
18. import com.example.clickanime.R
19. import
    com.example.clickanime.data.local.AppPreferences
20. import
    com.example.clickanime.viewmodel.MovieViewModel
21.
22. @OptIn(ExperimentalMaterial3Api::class)
23. @Composable
24. fun SettingsScreen(
25.     viewModel: MovieViewModel,
26.     preferences: AppPreferences,
27.     onBack: () -> Unit
28. ) {
29.     var notificationsEnabled by remember {
30. mutableStateOf(preferences.isNotificationsEnabled)
31.     }
32.     var isGridView by remember {
33.         mutableStateOf(preferences.isGridView)
34.     }
35.     var showClearDialog by remember {
        mutableStateOf(false) }
36.     var showCacheClearedMsg by remember {
        mutableStateOf(false) }
37.
38.     if (showClearDialog) {

```

```

39.         AlertDialog(
40.             onDismissRequest = { showClearDialog =
false },
41.             icon = {
42.                 Icon(Icons.Default.DeleteSweep,
contentDescription = null)
43.             },
44.             title = {
Text(stringResource(R.string.settings_clear_cache_tit
le)) },
45.             text = {
Text(stringResource(R.string.settings_clear_cache_msg
)) },
46.             confirmButton = {
47.                 TextButton(onClick = {
48.                     viewModel.clearAllCache()
49.                     showCacheClearedMsg = true
50.                     showClearDialog = false
51.                 }) {
52.                     Text(
53.                         text =
stringResource(R.string.settings_clear_cache),
54.                         color =
MaterialTheme.colorScheme.error
55.                     )
56.                 }
57.             },
58.             dismissButton = {
59.                 TextButton(onClick = {
showClearDialog = false }) {
60.                     Text(stringResource(R.string.cancel))
61.                 }
62.             }
63.         )
64.     }
65.

```

```

66. Scaffold(
67.     topBar = {
68.         TopAppBar(
69.             title = {
70.                 Text(
71.                     text =
stringResource(R.string.settings_title),
72.                     fontWeight = FontWeight.Bold
73.                 )
74.             },
75.             navigationIcon = {
76.                 IconButton(onClick = onBackPressed) {
77.                     Icon(
78.                         imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
79.                         contentDescription =
stringResource(R.string.back)
80.                     )
81.                 }
82.             },
83.             colors =
TopAppBarDefaults.topAppBarColors(
84.                 containerColor =
MaterialTheme.colorScheme.primary,
85.                 titleContentColor =
MaterialTheme.colorScheme.onPrimary,
86.                 navigationIconContentColor =
MaterialTheme.colorScheme.onPrimary
87.             )
88.         )
89.     }
90. ) { padding ->
91.     Column(
92.         modifier = Modifier
93.             .fillMaxSize()
94.             .padding(padding)

```

```

95.         .verticalScroll(rememberScrollState())
96.             .padding(16.dp),
97.         verticalArrangement                        =
Arrangement.spacedBy(4.dp)
98.     ) {
99.
100.     SectionHeader(stringResource(R.string.settings_app))
101.
102.     ToggleRow(
103.         icon = Icons.Default.Notifications,
104.         title                        =
stringResource(R.string.settings_notifications),
105.         subtitle                    =
stringResource(R.string.settings_notifications_sub),
106.         checked = notificationsEnabled,
107.         onCheckedChange = {
108.             notificationsEnabled = it
109.         }
110.     )
111.
112.     ToggleRow(
113.         icon = Icons.Default.GridView,
114.         title                        =
stringResource(R.string.settings_grid_view),
115.         subtitle                    =
stringResource(R.string.settings_grid_view_sub),
116.         checked = isGridView,
117.         onCheckedChange = {
118.             isGridView = it
119.             preferences.isGridView = it
120.         }
121.     )
122.
123.     Spacer(Modifier.height(8.dp))

```

```

124.         HorizontalDivider()
125.         Spacer (Modifier.height (8.dp))
126.
127.     SectionHeader (stringResource (R.string.settings_data))
128.
129.     Card (
130.         modifier = Modifier.fillMaxWidth(),
131.         shape = RoundedCornerShape (12.dp),
132.         colors = CardDefaults.cardColors (
133.             containerColor                =
MaterialTheme.colorScheme.surfaceVariant
134.         )
135.     ) {
136.         Column (Modifier.padding (14.dp)) {
137.             Row (verticalAlignment        =
Alignment.CenterVertically) {
138.                 Icon (
139.                     imageVector          =
Icons.Default.Storage,
140.                     contentDescription    =
null,
141.                     tint                  =
MaterialTheme.colorScheme.primary,
142.                     modifier              =
Modifier.size (20.dp)
143.                 )
144.                 Spacer (Modifier.width (10.dp))
145.                 Column (Modifier.weight (1f)) {
146.                     Text (
147.                         text              =
stringResource (R.string.settings_cache_strategy),
148.                         fontWeight         =
FontWeight.SemiBold,
149.                         style              =
MaterialTheme.typography.bodyMedium
150.                     )

```

```

151.                 Text(
152.                     text                =
stringResource(R.string.settings_cache_strategy_value
),
153.                     style              =
MaterialTheme.typography.bodySmall,
154.                     color              =
MaterialTheme.colorScheme.primary
155.                 )
156.             }
157.         }
158.         Spacer(Modifier.height(8.dp))
159.         Text(
160.             text                =
stringResource(R.string.settings_cache_desc),
161.             style              =
MaterialTheme.typography.bodySmall,
162.             color              =
MaterialTheme.colorScheme.onSurfaceVariant,
163.             lineHeight         =
MaterialTheme.typography.bodySmall.lineHeight
164.         )
165.     }
166. }
167.
168.     Spacer(Modifier.height(8.dp))
169.
170.     OutlinedButton(
171.         onClick = { showClearDialog = true },
172.         modifier = Modifier.fillMaxWidth(),
173.         shape = RoundedCornerShape(10.dp),
174.         colors                =
ButtonDefaults.outlinedButtonColors(
175.             contentColor      =
MaterialTheme.colorScheme.error
176.         )
177.     ) {

```



```

178.             Icon(
179.                 imageVector                                =
Icons.Default.DeleteSweep,
180.                 contentDescription = null,
181.                 modifier = Modifier.size(18.dp)
182.             )
183.             Spacer(Modifier.width(8.dp))
184.             Text(
185.                 text                                =
stringResource(R.string.settings_clear_cache),
186.                 fontWeight = FontWeight.Medium
187.             )
188.         }
189.
190.         if (showCacheClearedMsg) {
191.             Spacer(Modifier.height(4.dp))
192.             Row(verticalAlignment                    =
Alignment.CenterVertically) {
193.                 Icon(
194.                     imageVector                                =
Icons.Default.CheckCircle,
195.                     contentDescription = null,
196.                     tint                                =
MaterialTheme.colorScheme.primary,
197.                     modifier                                =
Modifier.size(16.dp)
198.                 )
199.                 Spacer(Modifier.width(6.dp))
200.                 Text(
201.                     text                                =
stringResource(R.string.settings_cache_cleared),
202.                     style                                =
MaterialTheme.typography.bodySmall,
203.                     color                                =
MaterialTheme.colorScheme.primary
204.                 )
205.             }

```

```

206.         }
207.
208.         Spacer (Modifier.height (8.dp))
209.         HorizontalDivider()
210.         Spacer (Modifier.height (8.dp))
211.
212.     SectionHeader (stringResource (R.string.settings_about)
213. )
214.
215.         Card (
216.             modifier = Modifier.fillMaxWidth(),
217.             shape = RoundedCornerShape (12.dp),
218.             colors = CardDefaults.cardColors (
219.                 containerColor =
220. MaterialTheme.colorScheme.surfaceVariant
221.             )
222.         ) {
223.             Column (
224.                 Modifier.padding (14.dp),
225.                 verticalArrangement =
226. Arrangement.spacedBy (4.dp)
227.             ) {
228.                 Text (
229.                     text =
230. stringResource (R.string.settings_app_label),
231.                     fontWeight = FontWeight.Bold,
232.                     style =
233. MaterialTheme.typography.bodyMedium
234.                 )
235.                 HorizontalDivider (
236.                     modifier =
237. Modifier.padding (vertical = 4.dp),
238.                     color =
239. MaterialTheme.colorScheme.onSurfaceVariant.copy (alpha
240. = 0.2f)
241.                 )

```

```

234.         AboutRow(Icons.Default.School,
stringResource(R.string.settings_about_module))
235.         AboutRow(Icons.Default.Movie,
stringResource(R.string.settings_about_tmdb))
236.
AboutRow(Icons.Default.NetworkCheck,
stringResource(R.string.settings_about_networking))
237.         AboutRow(Icons.Default.Storage,
stringResource(R.string.settings_about_storage))
238.     }
239. }
240.
241.     Spacer(Modifier.height(24.dp))
242. }
243. }
244. }
245.
246. @Composable
247. private fun SectionHeader(title: String) {
248.     Text(
249.         text = title,
250.         style = MaterialTheme.typography.labelLarge,
251.         color = MaterialTheme.colorScheme.primary,
252.         fontWeight = FontWeight.Bold,
253.         modifier = Modifier.padding(top = 4.dp, bottom
= 6.dp)
254.     )
255. }
256.
257. @Composable
258. private fun ToggleRow(
259.     icon: ImageVector,
260.     title: String,
261.     subtitle: String,
262.     checked: Boolean,
263.     onCheckedChange: (Boolean) -> Unit

```

```

264. ) {
265.     Row(
266.         modifier = Modifier
267.             .fillMaxWidth()
268.             .padding(vertical = 6.dp),
269.         verticalAlignment                    =
Alignment.CenterVertically
270.     ) {
271.         Icon(
272.             imageVector = icon,
273.             contentDescription = null,
274.             tint = MaterialTheme.colorScheme.primary,
275.             modifier = Modifier.size(22.dp)
276.         )
277.         Spacer(Modifier.width(12.dp))
278.         Column(Modifier.weight(1f)) {
279.             Text(
280.                 text = title,
281.                 style                    =
MaterialTheme.typography.bodyMedium,
282.                 fontWeight = FontWeight.Medium
283.             )
284.             Text(
285.                 text = subtitle,
286.                 style                    =
MaterialTheme.typography.bodySmall,
287.                 color                    =
MaterialTheme.colorScheme.onSurfaceVariant
288.             )
289.         }
290.         Switch(
291.             checked = checked,
292.             onCheckedChange = onCheckedChange
293.         )
294.     }

```

295.	}
296.	
297.	@Composable
298.	private fun AboutRow(icon: ImageVector, text: String)
	{
299.	Row(
300.	verticalAlignment =
	Alignment.CenterVertically,
301.	modifier = Modifier.padding(vertical = 2.dp)
302.	) {
303.	Icon(
304.	imageVector = icon,
305.	contentDescription = null,
306.	tint =
	MaterialTheme.colorScheme.onSurfaceVariant,
307.	modifier = Modifier.size(14.dp)
308.	)
309.	Spacer(Modifier.width(8.dp))
310.	Text(
311.	text = text,
312.	style =
	MaterialTheme.typography.bodySmall,
313.	color =
	MaterialTheme.colorScheme.onSurfaceVariant
314.	)
315.	}
316.	}

Tabel 65. Source Code strings.xml English

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<resources>
3.	<string name="app_name">ClickAnime</string>
4.	<string name="app_title">Movie Explorer</string>
5.	
6.	<!-- Navigation -->

7.	<string name="nav_home">Home</string>
8.	<string name="nav_search">Search</string>
9.	<string name="nav_favorites">Favorites</string>
10.	<string name="nav_settings">Settings</string>
11.	
12.	<!-- Home -->
13.	<string name="featured">Featured</string>
14.	<string name="all_movies">All Movies</string>
15.	<string name="category_popular">Popular</string>
16.	<string name="category_now_playing">Now Playing</string>
17.	<string name="category_topRated">Top Rated</string>
18.	<string name="category_upcoming">Upcoming</string>
19.	<string name="loading_movies">Loading movies...</string>
20.	<string name="refresh">Refresh</string>
21.	<string name="retry">Retry</string>
22.	<string name="failed_to_load">Failed to load movies</string>
23.	
24.	<!-- Detail -->
25.	<string name="overview">Overview</string>
26.	<string name="no_overview">No overview available.</string>
27.	<string name="rating">Rating</string>
28.	<string name="runtime">Runtime</string>
29.	<string name="year">Year</string>
30.	<string name="votes">Votes</string>
31.	<string name="box_office">Box Office</string>
32.	<string name="budget">Budget</string>
33.	<string name="revenue">Revenue</string>
34.	<string name="status">Status</string>
35.	<string name="language">Language</string>

36.	<code><string name="add_to_favorites">Add to Favorites</string></code>
37.	<code><string name="remove_from_favorites">Remove from Favorites</string></code>
38.	
39.	<code><!-- Search --></code>
40.	<code><string name="search_hint">Search movies...</string></code>
41.	<code><string name="search_empty_hint">Search for your favorite movies</string></code>
42.	<code><string name="search_no_results">No movies found for</string></code>
43.	<code><string name="search_results">results for</string></code>
44.	<code><string name="clear_search">Clear search</string></code>
45.	
46.	<code><!-- Favorites --></code>
47.	<code><string name="favorites_empty_title">No favorites yet</string></code>
48.	<code><string name="favorites_empty_desc">Browse movies and tap ♥ to save them here permanently.</string></code>
49.	<code><string name="favorites_saved_hint">Saved to your device • Stays even offline</string></code>
50.	<code><string name="remove_favorite">Remove Favorite</string></code>
51.	<code><string name="remove_favorite_confirm">Remove \"%s\" from your favorites?</string></code>
52.	<code><string name="remove">Remove</string></code>
53.	<code><string name="cancel">Cancel</string></code>
54.	
55.	<code><!-- Settings --></code>
56.	<code><string name="settings_title">Settings</string></code>
57.	<code><string name="settings_app">App Settings</string></code>
58.	<code><string name="settings_notifications">Enable Notifications</string></code>
59.	<code><string name="settings_notifications_sub">Receive updates about new movies</string></code>
60.	<code><string name="settings_grid_view">Grid View</string></code>

61.	<code><string name="settings_grid_view_sub">Display movies in a grid layout</string></code>
62.	<code><string name="settings_data">Data & amp; Cache</string></code>
63.	<code><string name="settings_cache_strategy">Movie Cache Strategy</string></code>
64.	<code><string name="settings_cache_strategy_value">Cache-First with 30-minute TTL</string></code>
65.	<code><string name="settings_cache_desc">Movies are cached locally and refreshed every 30 minutes. Favorites are stored permanently.</string></code>
66.	<code><string name="settings_clear_cache">Clear Movie Cache</string></code>
67.	<code><string name="settings_clear_cache_title">Clear Movie Cache</string></code>
68.	<code><string name="settings_clear_cache_msg">This will delete all cached movie data. The app will re-fetch from the internet. Your favorites will NOT be deleted.</string></code>
69.	<code><string name="settings_cache_cleared">✓ Cache cleared. Movies will be refreshed from the internet.</string></code>
70.	<code><string name="settings_about">About</string></code>
71.	<code><string name="settings_app_label">ClickAnime - Movie Explorer</string></code>
72.	<code><string name="settings_about_module">Modul 5 - Connect to the Internet</string></code>
73.	<code><string name="settings_about_tmdb">Data provided by The Movie Database (TMDB)</string></code>
74.	<code><string name="settings_about_networking">Networking: Retrofit + KotlinX Serialization</string></code>
75.	<code><string name="settings_about_storage">Storage: Room (cache) + SharedPreferences (settings)</string></code>
76.	
77.	<code><!-- Language --></code>
78.	<code><string name="change_language">Change Language</string></code>
79.	<code><string name="select_language">Select Language</string></code>
80.	<code><string name="apply_language">Apply</string></code>

81.	<code><string name="lang_english">English</string></code>
82.	<code><string name="lang_indonesian">Indonesian</string></code>
83.	<code><string name="back">Back</string></code>
84.	<code></resources></code>

Tabel 66. Source Code strings.xml Indonesia

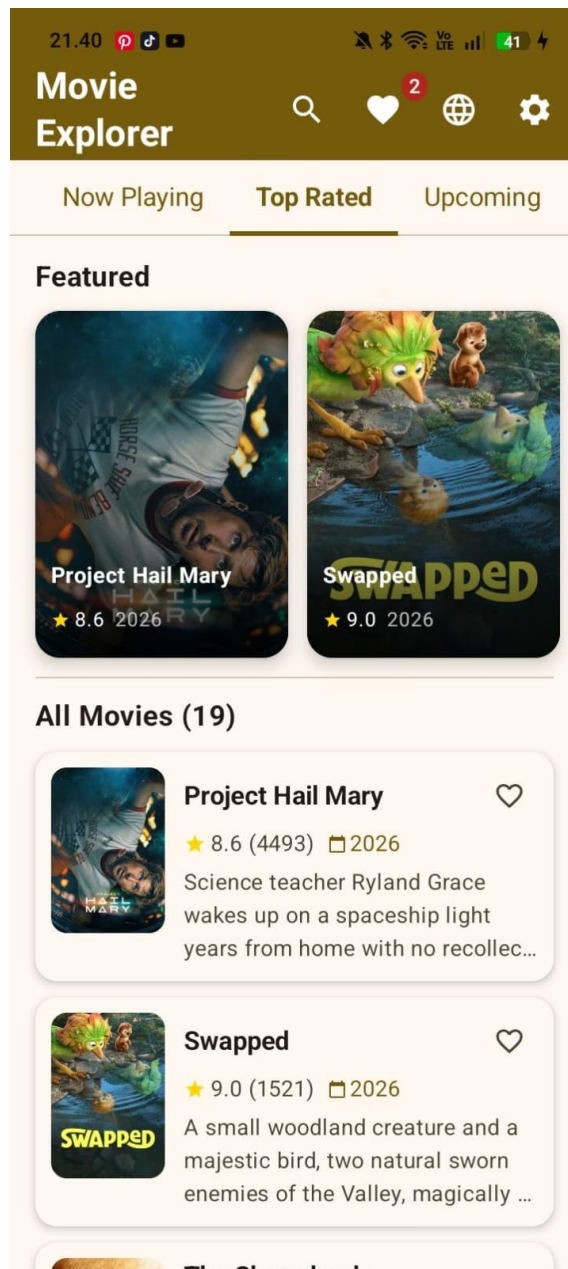
1.	<code><?xml version="1.0" encoding="utf-8"?></code>
2.	<code><resources></code>
3.	<code><string name="app_name">ClickAnime</string></code>
4.	<code><string name="app_title">Penjelajah Film</string></code>
5.	
6.	<code><string name="nav_home">Beranda</string></code>
7.	<code><string name="nav_search">Cari</string></code>
8.	<code><string name="nav_favorites">Favorit</string></code>
9.	<code><string name="nav_settings">Pengaturan</string></code>
10.	
11.	<code><string name="featured">Unggulan</string></code>
12.	<code><string name="all_movies">Semua Film</string></code>
13.	<code><string name="category_popular">Populer</string></code>
14.	<code><string name="category_now_playing">Sedang Tayang</string></code>
15.	<code><string name="category_top_rated">Nilai Tertinggi</string></code>
16.	<code><string name="category_upcoming">Akan Datang</string></code>
17.	<code><string name="loading_movies">Memuat film...</string></code>
18.	<code><string name="refresh">Perbarui</string></code>
19.	<code><string name="retry">Coba Lagi</string></code>
20.	<code><string name="failed_to_load">Gagal memuat film</string></code>
21.	
22.	<code><string name="overview">Sinopsis</string></code>
23.	<code><string name="no_overview">Tidak ada sinopsis tersedia.</string></code>

24.	<string name="rating">Nilai</string>
25.	<string name="runtime">Durasi</string>
26.	<string name="year">Tahun</string>
27.	<string name="votes">Suara</string>
28.	<string name="box_office">Box Office</string>
29.	<string name="budget">Anggaran</string>
30.	<string name="revenue">Pendapatan</string>
31.	<string name="status">Status</string>
32.	<string name="language">Bahasa</string>
33.	<string name="add_to_favorites">Tambah ke Favorit</string>
34.	<string name="remove_from_favorites">Hapus dari Favorit</string>
35.	
36.	<string name="search_hint">Cari film...</string>
37.	<string name="search_empty_hint">Cari film favoritmu</string>
38.	<string name="search_no_results">Tidak ada film ditemukan untuk</string>
39.	<string name="search_results">hasil untuk</string>
40.	<string name="clear_search">Bersihkan pencarian</string>
41.	
42.	<string name="favorites_empty_title">Belum ada favorit</string>
43.	<string name="favorites_empty_desc">Jelajahi film dan ketuk ♥ untuk menyimpannya di sini secara permanen.</string>
44.	<string name="favorites_saved_hint">Tersimpan di perangkat • Tetap ada saat offline</string>
45.	<string name="remove_favorite">Hapus Favorit</string>
46.	<string name="remove_favorite_confirm">Hapus \"%s\" dari favorit?</string>
47.	<string name="remove">Hapus</string>
48.	<string name="cancel">Batal</string>
49.	

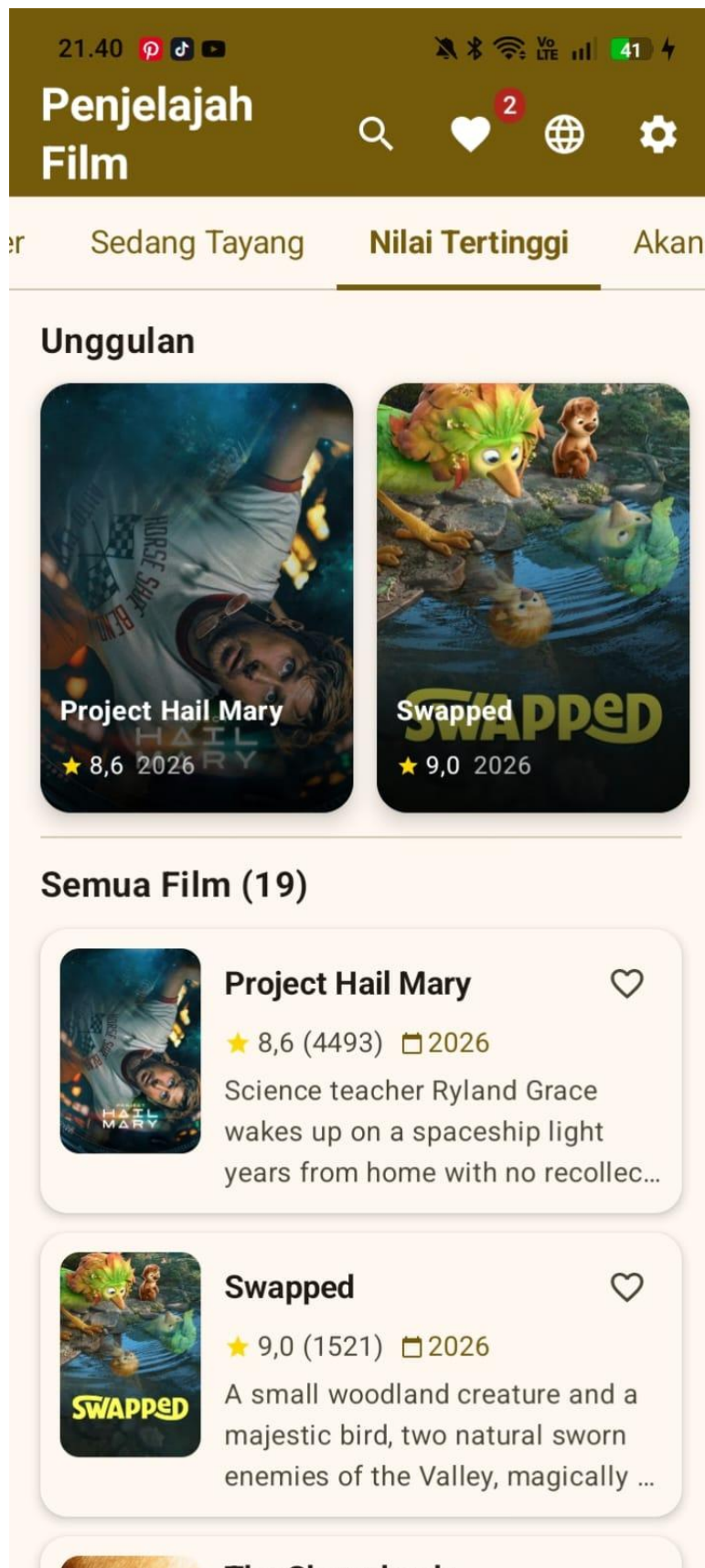
50.	<code><string name="settings_title">Pengaturan</string></code>
51.	<code><string name="settings_app">Pengaturan Aplikasi</string></code>
52.	<code><string name="settings_notifications">Aktifkan Notifikasi</string></code>
53.	<code><string name="settings_notifications_sub">Terima pembaruan tentang film baru</string></code>
54.	<code><string name="settings_grid_view">Tampilan Grid</string></code>
55.	<code><string name="settings_grid_view_sub">Tampilkan film dalam tata letak grid</string></code>
56.	<code><string name="settings_data">Data & Cache</string></code>
57.	<code><string name="settings_cache_strategy">Strategi Cache Film</string></code>
58.	<code><string name="settings_cache_strategy_value">Cache-First dengan TTL 30 menit</string></code>
59.	<code><string name="settings_cache_desc">Film di-cache secara lokal dan diperbarui setiap 30 menit. Favorit disimpan permanen.</string></code>
60.	<code><string name="settings_clear_cache">Bersihkan Cache Film</string></code>
61.	<code><string name="settings_clear_cache_title">Bersihkan Cache Film</string></code>
62.	<code><string name="settings_clear_cache_msg">Ini akan menghapus semua data cache film. Aplikasi akan mengambil ulang dari internet. Favorit Anda TIDAK akan dihapus.</string></code>
63.	<code><string name="settings_cache_cleared">✓ Cache dibersihkan. Film akan diperbarui dari internet.</string></code>
64.	<code><string name="settings_about">Tentang</string></code>
65.	<code><string name="settings_app_label">ClickAnime – Penjelajah Film</string></code>
66.	<code><string name="settings_about_module">Modul 5 – Connect to the Internet</string></code>
67.	<code><string name="settings_about_tmdb">Data disediakan oleh The Movie Database (TMDB)</string></code>
68.	<code><string name="settings_about_networking">Jaringan: Retrofit + KotlinX Serialization</string></code>

69.	<code><string name="settings_about_storage">Penyimpanan: Room (cache) + SharedPreferences (pengaturan)</string></code>
70.	
71.	<code><string name="change_language">Ganti Bahasa</string></code>
72.	<code><string name="select_language">Pilih Bahasa</string></code>
73.	<code><string name="apply_language">Terapkan</string></code>
74.	<code><string name="lang_english">Bahasa Inggris</string></code>
75.	<code><string name="lang_indonesian">Bahasa Indonesia</string></code>
76.	<code><string name="back">Kembali</string></code>
77.	<code></resources></code>

B. OUTPUT CODE



Gambar 25. Homepage English



Gambar 26. Home Page Indonesia

22.11









46


Project Hail Mary




Project Hail Mary

"Believe in the Hail Mary."




Nilai
8,6/10


Durasi
2h 37m


Tahun
2026


Suara
4496

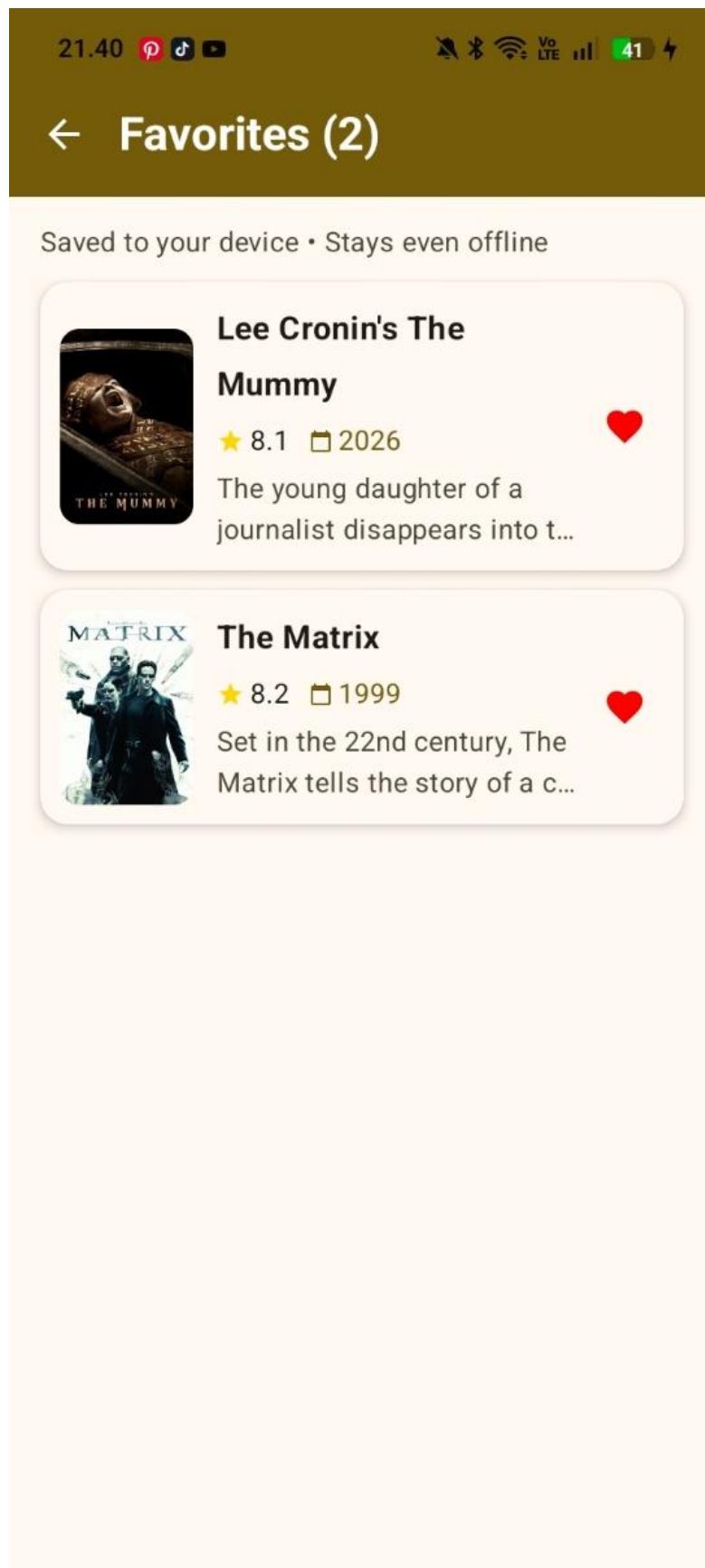
Science Fiction
Adventure

i Status: Released Bahasa: EN

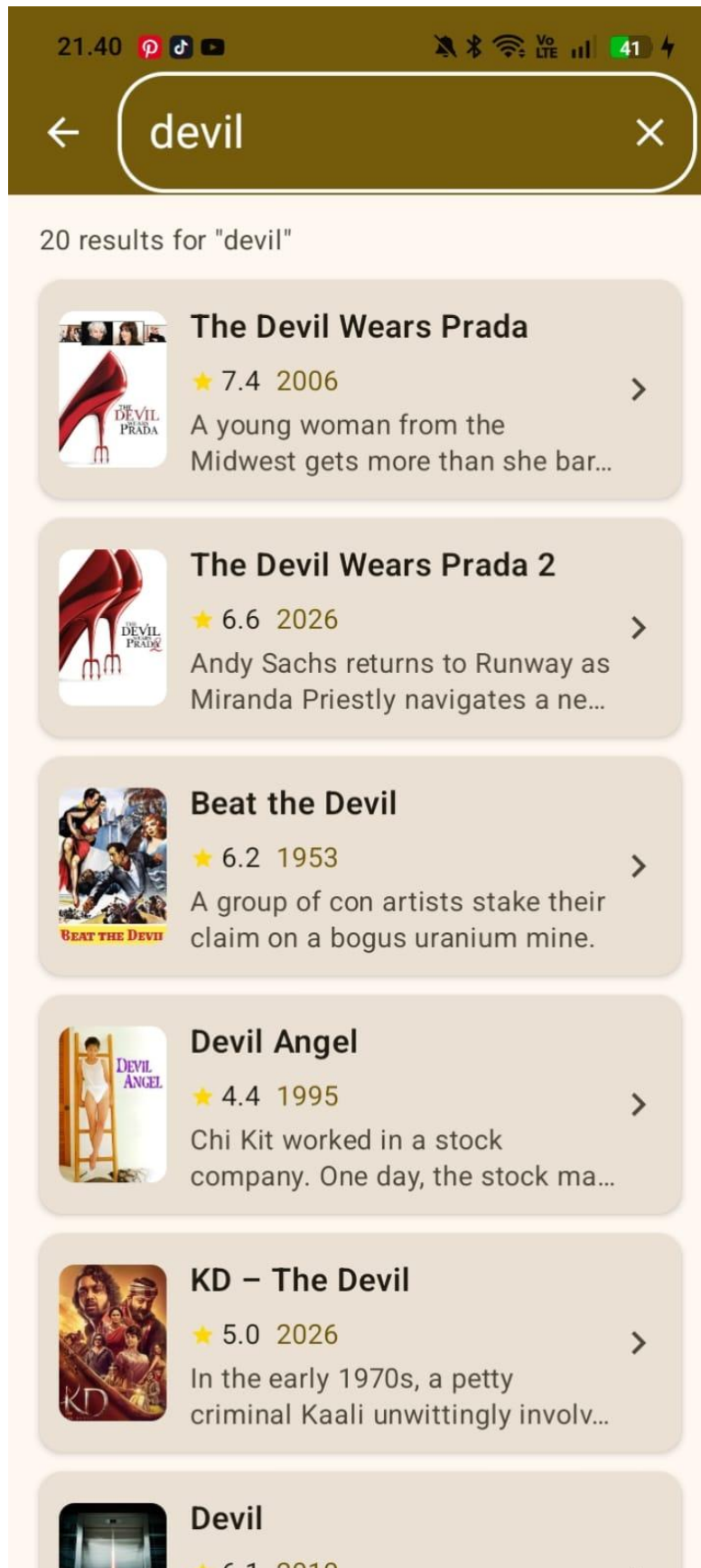
Sinopsis

Science teacher Ryland Grace wakes up on a spaceship light years from home with no recollection of who he is or how he got there. As his memory returns, he begins to uncover his mission: solve the riddle of the mysterious substance causing the sun to

Gambar 27. Detail Page Indonesia



Gambar 28. Favorites Page



Gambar 29. Search Page

C. PEMBAHASAN

MainActivity.kt

MainActivity berperan sebagai titik masuk utama (entry point) aplikasi. Kelas ini mengatur tampilan ke layar penuh (edge-to-edge) dan mengelola pengaturan bahasa (Locale) dengan mengambil data dari SharedPreferences sebelum Context di attach. Di dalam fungsi onCreate, aktivitas ini membangun antarmuka menggunakan Jetpack Compose, menginisialisasi MovieViewModel dari *factory* yang ada di AppContainer, dan meneruskan kontrol navigasi utama ke NavGraph.

MovieApp.kt

MovieApp mewarisi kelas Application dan merupakan titik inisialisasi awal sebelum komponen aplikasi lain dijalankan. File ini berfungsi sebagai tempat inisialisasi global untuk AppContainer yang mengatur *Dependency Injection* manual. Selain itu, modul ini juga mengonfigurasi library Timber untuk sistem pencatatan log (logging) otomatis yang hanya aktif pada mode debug, sehingga memudahkan pelacakan tanpa membebani versi rilis.

MovieDao.kt

MovieDao merupakan *Data Access Object* (DAO) untuk library Room yang mengatur operasi database lokal. File ini mendefinisikan kueri-kueri SQL untuk mengelola data cache film, detail film, dan daftar film favorit. Fungsi di dalamnya memanfaatkan *Coroutines* dan *Flow* secara asinkron untuk operasi seperti menyimpan (insert), mengambil data berdasarkan kategori, mencari data, hingga membersihkan cache yang sudah kedaluwarsa.

MovieEntity.kt

MovieEntity mendefinisikan representasi tabel di dalam database Room melalui anotasi @Entity. File ini berisi struktur *Data Class* untuk film umum (movies), detail film (movie_details), dan film favorit (favorite_movies). Entitas-entitas ini bertindak sebagai skema basis data lokal yang menyimpan cache dari API sehingga aplikasi dapat menampilkan data kembali meski tanpa koneksi internet.

AppPreferences.kt

AppPreferences adalah kelas utilitas (wrapper) yang mengelola penyimpanan persisten data ringan menggunakan SharedPreferences. File ini menangani status pengaturan pengguna yang sifatnya global, seperti pilihan bahasa (language), preferensi tampilan antarmuka (grid view), status notifikasi, serta menyimpan state terakhir dari kategori film yang diakses agar aplikasi mengingat pilihan pengguna di sesi berikutnya.

MovieDatabase.kt

MovieDatabase adalah kelas abstrak turunan RoomDatabase yang berfungsi sebagai pengelola utama database lokal. File ini mendefinisikan seluruh entitas tabel dan menyambungkannya dengan DAO yang relevan. Kelas ini mengimplementasikan pola *Singleton* untuk memastikan hanya ada satu instance database yang dibuat dan beroperasi dalam satu memori berjalan untuk menghindari bentrok pada koneksi SQLite.

MovieMapper.kt

MovieMapper berfungsi sebagai pemetaan (mapper) arsitektur data. File ini berisi *extension functions* untuk mengubah objek dari satu lapisan ke lapisan lain. Ia merombak data *Data Transfer Object* (DTO) hasil *response* API menjadi *Entity* untuk disimpan di database lokal Room, dan kemudian mengubah entitas tersebut menjadi model *Domain* yang bersih untuk siap disajikan ke antarmuka pengguna (UI).

TmdbResponse.kt

TmdbResponse menampung struktur *Data Class* khusus untuk menerjemahkan JSON dari TMDB API. Menggunakan anotasi dari *KotlinX Serialization* (*@SerializedName*), file ini memastikan kunci JSON di *parsing* dengan presisi ke variabel Kotlin. Selain itu, terdapat *sealed class* *ApiResponse* untuk membungkus hasil permintaan jaringan menjadi tiga state fundamental: *Success*, *Error*, dan *Exception*.

ApiHandler.kt

ApiHandler menyediakan fungsi utilitas *safeApiCall* yang dirancang untuk menjalankan pemanggilan Retrofit API dalam blok *try-catch* secara aman. File ini menstandarkan semua respons jaringan dan *parsing error*, sehingga alih-alih melempar interupsi secara langsung yang membuat aplikasi mogok (*crash*), fungsi ini akan selalu mengembalikan objek tipe *ApiResponse* yang terprediksi.

NetworkClient.kt

NetworkClient mengonfigurasi klien HTTP dan Retrofit. Di dalamnya terdapat *OkHttpClient* yang diinjeksi dengan *Interceptor* khusus untuk selalu menyematkan *API Key* TMDB pada *header* setiap *request*. File ini juga mengintegrasikan pemformatan respons JSON berbasis *KotlinX Serialization* dan *logging interceptor* untuk menampilkan rekam jejak lalu lintas jaringan pada Android Studio saat mode debug.

TmdbApiService.kt

TmdbApiService adalah antarmuka (interface) khusus Retrofit yang memetakan URL dan *endpoint* API dari The Movie Database. File ini menggunakan anotasi HTTP seperti *@GET* untuk menarik daftar film dari berbagai kategori (populer, *now playing*, *top rated*), mencari film spesifik, dan memanggil rincian lengkap profil satu film secara asinkron (menggunakan parameter *suspend*).

MovieRepository.kt

MovieRepository bertindak sebagai *Single Source of Truth* pengelola aliran data dengan menerapkan *Caching Strategy*. File ini terlebih dahulu mengecek waktu kedaluwarsa cache di Room; jika valid, aplikasi hanya memuat data lokal agar cepat (*offline-first*). Apabila kedaluwarsa, *Repository* ini akan menarik respons terbaru dari *NetworkClient*, menyimpan perubahannya di *Database*, lalu memancarkannya kembali ke *ViewModel* menggunakan Flow.

AppContainer.kt

AppContainer menerapkan arsitektur injeksi dependensi manual untuk level aplikasi. File ini membuat, mengelola, dan membagikan satu instansi tunggal (*singleton*) komponen inti yang berat (seperti koneksi database, *Retrofit*, dan *Repository*) ke kelas-kelas lain secara terpusat, sehingga mencegah memori bengkak akibat pembuatan objek (instansiasi) yang berulang-ulang di berbagai layar UI.

Movie.kt

Movie.kt merupakan definisi model tingkat *Domain* yang murni dan bersih dari *framework* API atau Database. Model data (seperti *Movie* dan *MovieDetail*) didesain sangat spesifik untuk kebutuhan UI, termasuk mengimplementasikan properti komputasi (*computed properties*) yang langsung merangkai URL format poster, mengambil potongan tahun rilis, dan melakukan penyesuaian string rating secara otomatis.

MovieUiState.kt

MovieUiState menggunakan hierarki kelas tertutup (*sealed classes*) untuk memodelkan berbagai kemungkinan status pada halaman UI (Loading, Success, Error, atau Idle). File ini menjamin bahwa komposisi di *Jetpack Compose* bersifat prediktif dan hanya merender antarmuka berdasarkan kepastian data yang dilempar, menjaga UI tidak me-render status yang tumpang tindih.

MovieViewModel.kt

MovieViewModel menjadi jembatan utama logika interaksi di sisi klien. Kelas ini menampung data UI reaktif (StateFlow) terkait detail, kategori, daftar pencarian, maupun koleksi favorit. Semua operasi asinkron dipanggil melalui *viewModelScope* agar tidak memblokir antarmuka, sekaligus mengatur delegasi ke *MovieRepository* (seperti saat kategori di tap, pencarian diketik, hingga interaksi simpan favorit).

NavGraph.kt

NavGraph menampung topologi perpindahan setiap antarmuka (*Screen*) dalam satu kendali komposisi via *NavHost*. Skema ini menarik setiap layar (seperti *HomeScreen*, *DetailScreen*, *SettingsScreen*) serta mendelegasikan perpindahan rute dan penangkapan argumen seperti *movieId*. Sistem fungsi navigasi dilempar sebagai parameter *lambda* untuk meminimalisasi ketergantungan antar komponen visual langsung.

Screen.kt

Screen membentuk skema objek berkonsep *sealed class* yang mewakili semua identitas rute (rute dasar berupa teks string). File ini mengeliminasi penulisan label teks rute yang rentan *typo* (*hardcoded strings*), dan mencakup fungsi helper (*createRoute*) yang secara sistematis menyisipkan ID film spesifik ketika aplikasi perlu melompat dari daftar ke halaman *DetailScreen*.

HomeScreen.kt

HomeScreen bertugas melukis visual papan kemudi depan dengan menanamkan tata letak *Scaffold*. Halaman ini memonitor status *ViewModel* via *collectAsState()* yang segera merender daftar film vertikal atau korsel film horisontal (*carousel*). Terdapat juga

integrasi navigasi interaktif (*TopAppBar*) pada *header* yang memberikan akses mulus untuk pencarian, favorit, maupun menuju bagian panel pengaturan.

DetailScreen.kt

DetailScreen menciptakan komposisi antarmuka yang mengkonsumsi *StateFlow* detail profil film secara utuh. Ketika sebuah rute *movieId* ditangkap, file ini akan memicu *ViewModel* mengunduh data TMDb secara terperinci. Elemen UI menampilkan lapisan poster grafis bertumpuk (*backdrop*), durasi tayang, status rilis, dan ikhtisar sinopsis lengkap yang dilengkapi *Floating Action Button* berfitur interaksi favorit *real-time*.

LanguageScreen.kt

LanguageScreen merepresentasikan elemen UI tempat pengguna menentukan prioritas bahasa lokalisasi. Aksi klik dalam *layout* ini akan mengeksekusi parameter *callback* ke arah komponen induk agar memutakhirkan preferensi basis *AppPreferences* dan segera memulai ulang paksa struktur aktivitas dasar, sehingga *resource* aplikasi terganti sepenuhnya di tingkat sistem.

FavoritesScreen.kt

FavoritesScreen merender area layar perpustakaan pribadi yang diamati sistem dari status *Flow* koleksi *database* lokal (*Room*). Layar ini menjamin visual list disinkronisasi langsung jika pengguna mencabut status hati (*unfavorite*). UI menangani logika mandiri untuk menampilkan pesan indikator kosong ketika basis data *favorite* belum memiliki satu baris entri pun.

SearchScreen.kt

SearchScreen memberikan wadah kolom pencarian (input form) dan menangani aliran teks dinamis *real-time*. Dengan pola *debounce* pada *ViewModel* untuk menjaga agar laju pengetikan (query) tidak membebani limit API, halaman ini langsung merespons barisan hasil pencarian spesifik, sembari menukar *state* tampilan seketika menjadi ikon muat ulang (*loading spinner*) hingga sukses melukis rentetan *card* film.

SettingsScreen.kt

SettingsScreen adalah panel UI minimalis yang menyajikan fitur manajemen aplikasi tingkat lanjut kepada pengguna, yang dirancang menggunakan *modifier Jetpack Compose* yang responsif. Tampilan ini menampilkan informasi modul/developer, sekaligus memiliki opsi pembersihan file sampah (Clear Cache) yang saat ditekan akan menstimulasi *Repository* via *ViewModel* untuk merombak serta menghapus histori tabel *database*.

strings.xml English

strings.xml English berperan sebagai fondasi *resource* tekstual (hardcode text centralization) standar dalam lingkup sistem Android. Semua istilah tombol, dialog, deskripsi film, dan *placeholder* error digabung di satu repositori *XML*, sehingga kode Kotlin murni bebas dari *string* mentah dan pemeliharaan kosakata menjadi lebih rapi di satu titik modifikasi.

strings.xml Indonesia

`strings.xml` Indonesia adalah implementasi dukungan multibahasa (lokalisasi i18n). Berkas sumber alternatif (berada di jalur folder `values-id`) ini menjabarkan seluruh terjemahan yang sinkron dengan *file string* Inggris. Ketika modul `MainActivity` menyuntikkan *Locale* bahasa Indonesia ke *base context*, aplikasi seketika mendeteksi dan menggunakan seluruh matriks bahasa di sini untuk merombak UI.

TAUTAN GIT

<https://github.com/ERISCHA24/PRAKTIKUMMOBILE.git>